

DESIGN PATTERN

April 10, 2025

Contents

1	Patterns	2
1.1	Factory Method - creational	2
1.2	Strategy pattern - behavioral	7
1.3	Template Method - behavioral	10
1.4	Decorator Pattern - structural	12
1.5	Composite Pattern - structural	21
1.6	Adapter Pattern - structural	29
1.7	Singleton Pattern - creational	33
1.8	Facade Pattern - structural	36
1.9	Flyweight Pattern - structural	42
1.10	proxy Pattern - Structural	48
1.11	Command Pattern - behavioral	54
1.12	Prototype Pattern - creational	65
1.13	Observer Pattern - behavioral	76
2	Combinations	86
2.1	Strategy and Factory	86
2.2	Strategy and Template	88
2.3	Strategy and Decorator	90
2.4	Strategy and Composite	93
2.5	Strategy and Adapter	96
2.6	Strategy and Singleton	98
2.7	Strategy, Factory and Template	99
2.8	Decorator, Composite and Adapter	102
2.9	Factory, Singleton and Adapter	106
2.10	Template, Factory and Strategy	108
2.11	Strategy, Decorator and Factory	111
2.12	Observer + Command	115
2.13	proxy + Flyweight	117
2.14	Facade + Prototype	119
2.15	Command + Prototype	121
2.16	Proxy + Observer	123
2.17	Factory + Decorator	125
2.18	Facade + Singleton	127
2.19	Command + Composite	129
2.20	Composite + Decorator	130
3	Differences	132

1 Patterns

1.1 Factory Method - creational

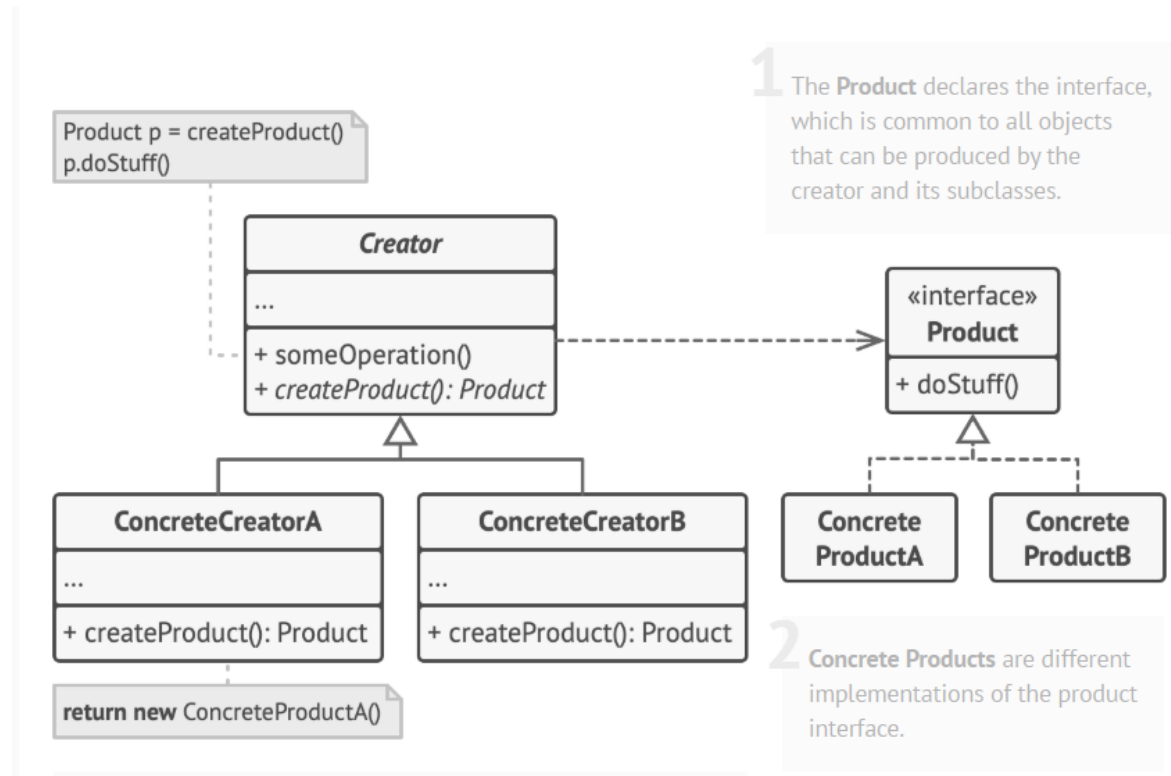


Figure 1: Factory structure

a scenario where we have different types of logistics, such as `RoadTransport` and `SeaTransport`. Each type of transport provides a method to deliver goods. The Factory Method pattern allows us to define a method to create an object, but lets subclasses alter the type of object that will be created.

```
1 //Step 1: Define the Product Interface
2 public interface ITransport
3 {
4     void Deliver();
5 }
6 //Step 2: Create Concrete Products
7 public class RoadTransport : ITransport
8 {
9     public void Deliver()
10    {
11        Console.WriteLine("Delivering by road in a truck.");
12    }
13 }
14
15 public class SeaTransport : ITransport
16 {
17     public void Deliver()
18    {
```

```

19         Console.WriteLine("Delivering by sea in a ship.");
20     }
21 }
22 //Step 3: Define the Creator Abstract Class
23 public abstract class Logistics
24 {
25     // Factory Method
26     public abstract ITransport CreateTransport();
27
28     public void PlanDelivery()
29     {
30         ITransport transport = CreateTransport();
31         transport.Deliver();
32     }
33 }
34 //Step 4: Create Concrete Creators
35 public class RoadLogistics : Logistics
36 {
37     public override ITransport CreateTransport()
38     {
39         return new RoadTransport();
40     }
41 }
42
43 public class SeaLogistics : Logistics
44 {
45     public override ITransport CreateTransport()
46     {
47         return new SeaTransport();
48     }
49 }
50 //Step 5: Using the Factory Method
51 class Program
52 {
53     static void Main(string[] args)
54     {
55         Logistics roadLogistics = new RoadLogistics();
56         roadLogistics.PlanDelivery(); // Output: Delivering by road in a truck.
57
58         Logistics seaLogistics = new SeaLogistics();
59         seaLogistics.PlanDelivery(); // Output: Delivering by sea in a ship.
60     }
61 }
62 \\output
63 Delivering by road in a truck.
64 Delivering by sea in a ship.

```

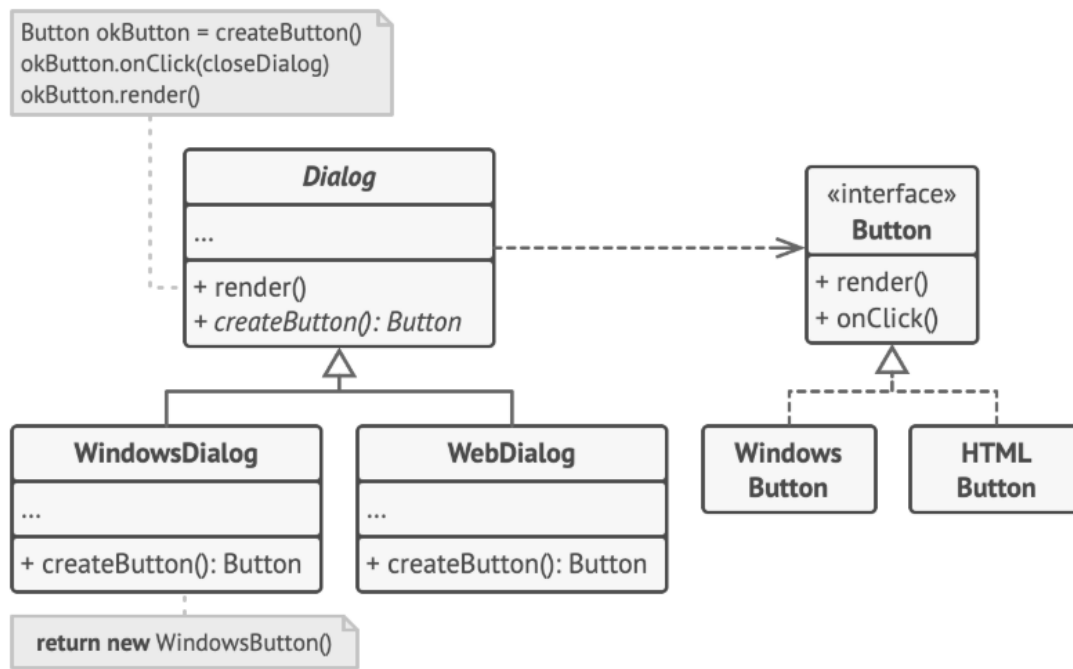


Figure 2: Factory

```

1      using System;
2
3      // The Product interface
4      public interface IButton
5      {
6          void Render();
7          void OnClick(Action action);
8      }
9
10     // Concrete Products
11     public class WindowsButton : IButton
12     {
13         public void Render()
14         {
15             Console.WriteLine("Rendering a Windows-style button");
16         }
17
18         public void OnClick(Action action)
19         {
20             Console.WriteLine("Windows button clicked");
21             action?.Invoke();
22         }
23     }
24
25     public class HTMLButton : IButton
26     {
27         public void Render()
28         {

```

```

29         Console.WriteLine("Rendering an HTML button");
30     }
31
32     public void OnClick(Action action)
33     {
34         Console.WriteLine("HTML button clicked");
35         action?.Invoke();
36     }
37 }
38
39 // The Creator abstract class
40 public abstract class Dialog
41 {
42     public void Render()
43     {
44         Console.WriteLine("Rendering dialog...");
45         IButton okButton = CreateButton();
46         okButton.OnClick(CloseDialog);
47         okButton.Render();
48     }
49
50     private void CloseDialog()
51     {
52         Console.WriteLine("Dialog closed");
53     }
54
55     // Factory method
56     public abstract IButton CreateButton();
57 }
58
59 // Concrete Creators
60 public class WindowsDialog : Dialog
61 {
62     public override IButton CreateButton()
63     {
64         return new WindowsButton();
65     }
66 }
67
68 public class WebDialog : Dialog
69 {
70     public override IButton CreateButton()
71     {
72         return new HTMLButton();
73     }
74 }
75
76 // Client code
77 class Program
78 {
79     static void Main(string[] args)
80     {
81         Console.WriteLine("Creating Windows dialog:");
82         Dialog windowsDialog = new WindowsDialog();
83         windowsDialog.Render();
84
85         Console.WriteLine("\nCreating Web dialog:");
86         Dialog webDialog = new WebDialog();
87         webDialog.Render();

```

```
88
89      /* Output:
90      Creating Windows dialog:
91      Rendering dialog...
92      Windows button clicked
93      Dialog closed
94      Rendering a Windows-style button
95
96      Creating Web dialog:
97      Rendering dialog...
98      HTML button clicked
99      Dialog closed
100     Rendering an HTML button
101     */
102 }
103 }
```

1.2 Strategy pattern - behavioral

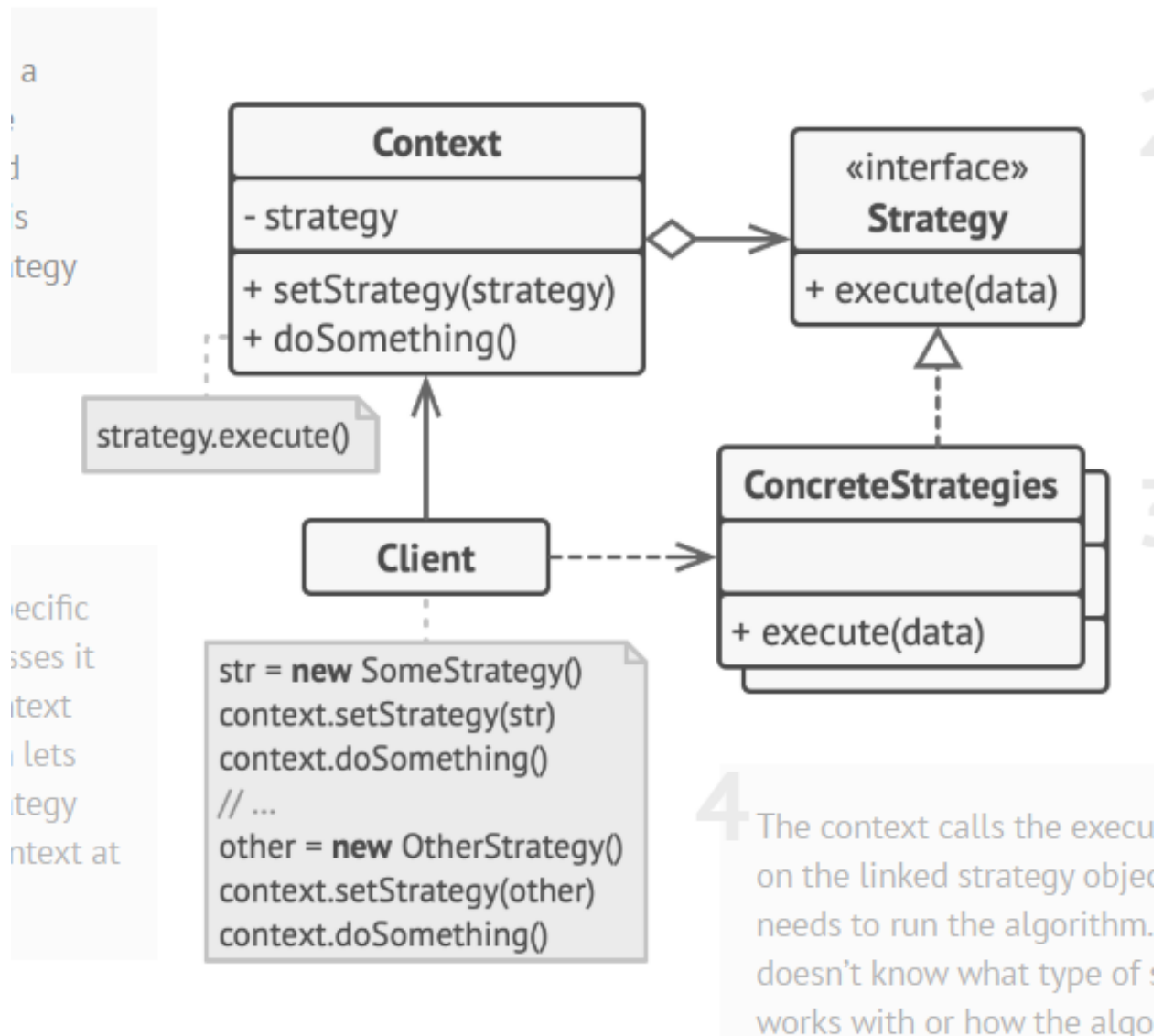


Figure 3: strategy structure

You can swap algorithms used inside an object at runtime. A scenario where different types of payment methods (Credit Card, PayPal, and Bank Transfer) are used for an online shopping platform. The `PaymentStrategy` interface defines a strategy for processing payments, and each concrete class implements a different payment method.

```

1 //Step 1: Define the Strategy Interface
2 public interface IPaymentStrategy
3 {
4     void Pay(decimal amount);
5 }
6 //Step 2: Create Concrete Strategies
  
```

```

7 public class CreditCardPayment : IPaymentStrategy
8 {
9     private string _cardNumber;
10    private string _cardHolderName;
11    private string _expiryDate;
12
13    public CreditCardPayment(string cardNumber, string cardHolderName, string
        expiryDate)
14    {
15        _cardNumber = cardNumber;
16        _cardHolderName = cardHolderName;
17        _expiryDate = expiryDate;
18    }
19
20    public void Pay(decimal amount)
21    {
22        Console.WriteLine($"Paid {amount:C} using Credit Card.");
23    }
24 }
25
26 public class PayPalPayment : IPaymentStrategy
27 {
28     private string _email;
29
30     public PayPalPayment(string email)
31     {
32         _email = email;
33     }
34
35     public void Pay(decimal amount)
36     {
37         Console.WriteLine($"Paid {amount:C} using PayPal.");
38     }
39 }
40
41 public class BankTransferPayment : IPaymentStrategy
42 {
43     private string _bankAccountNumber;
44
45     public BankTransferPayment(string bankAccountNumber)
46     {
47         _bankAccountNumber = bankAccountNumber;
48     }
49
50     public void Pay(decimal amount)
51     {
52         Console.WriteLine($"Paid {amount:C} using Bank Transfer.");
53     }
54 }
55 //Step 3: Create the Context Class
56 public class ShoppingCart
57 {
58     private IPaymentStrategy _paymentStrategy;
59
60     public void SetPaymentMethod(IPaymentStrategy paymentStrategy)
61     {
62         _paymentStrategy = paymentStrategy;
63     }
64 }

```



```

65     public void Checkout(decimal amount)
66     {
67         if (_paymentStrategy == null)
68         {
69             Console.WriteLine("Please select a payment method before proceeding to
70                               checkout.");
71             return;
72         }
73         _paymentStrategy.Pay(amount);
74     }
75 //Step 4: Using the Strategy Pattern
76 class Program
77 {
78     static void Main(string[] args)
79     {
80         ShoppingCart cart = new ShoppingCart();
81
82         // Set payment method to Credit Card
83         cart.SetPaymentMethod(new CreditCardPayment("1234-5678-9012-3456", "John
84             Doe", "12/25"));
85         cart.Checkout(150.00m);
86
87         // Change payment method to PayPal
88         cart.SetPaymentMethod(new PayPalPayment("john.doe@example.com"));
89         cart.Checkout(75.50m);
90
91         // Change payment method to Bank Transfer
92         cart.SetPaymentMethod(new BankTransferPayment("9876543210"));
93         cart.Checkout(300.00m);
94     }
95 }
96
97 \\output:
98 Paid $150.00 using Credit Card.
99 Paid $75.50 using PayPal.

```

1.3 Template Method - behavioral

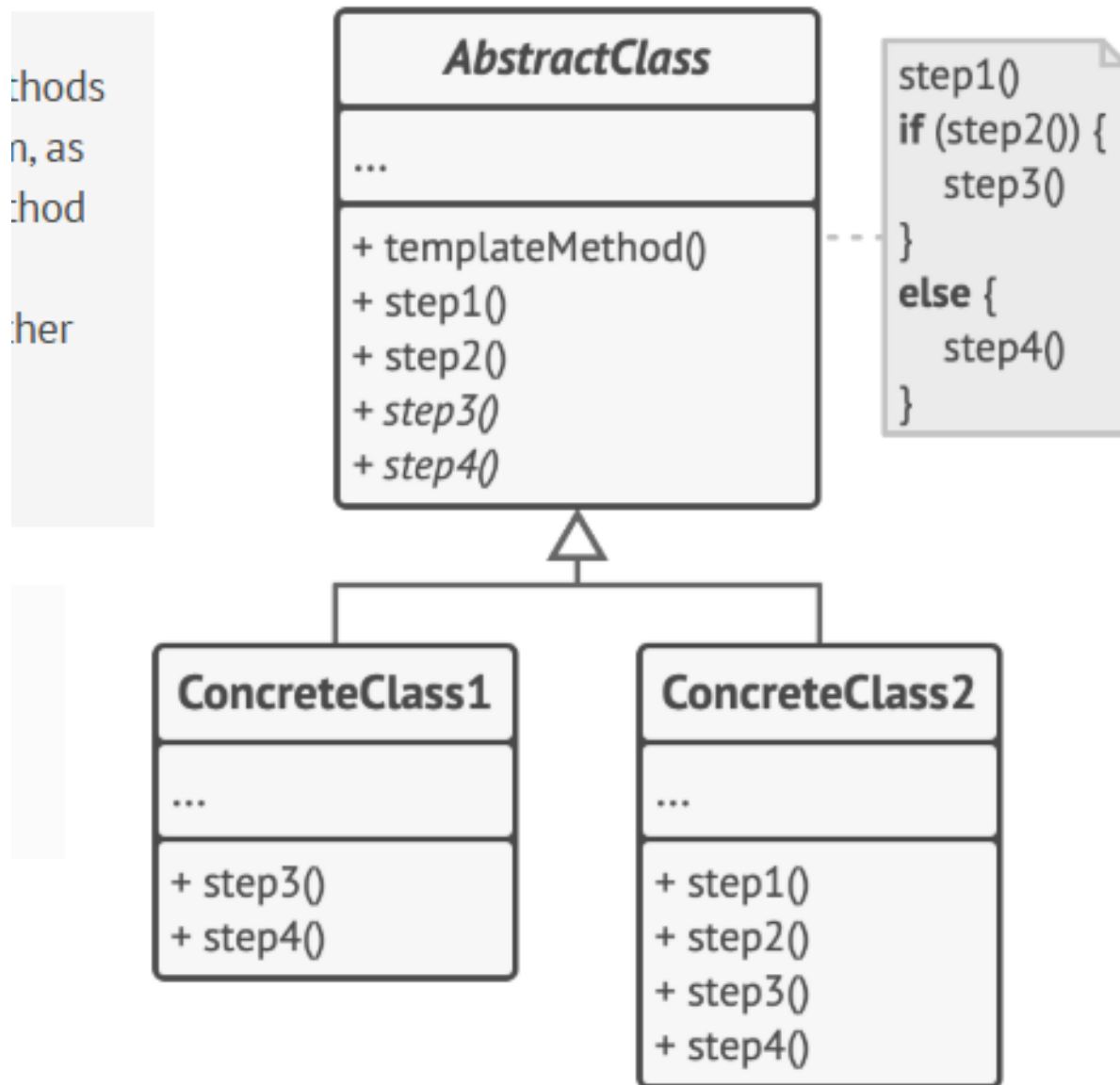


Figure 4: Template structure

a scenario where a data processing system needs to read, process, and save data. Different types of data sources (e.g., CSV files, JSON files) will follow the same general steps, but each can have its specific implementation for certain parts of the process.

1 //Step 1: Define the Abstract Class

```

2   public abstract class DataProcessor
3   {
4       // Template Method
5       public void ProcessData()
6       {
7           ReadData();
8           ProcessDataCore();
9           SaveData();
10      }
11
12      // Steps with default implementations
13      protected abstract void ReadData();
14      protected abstract void ProcessDataCore();
15
16      // Default implementation of SaveData
17      protected virtual void SaveData()
18      {
19          Console.WriteLine("Saving data to the database.");
20      }
21  }
22
23  //Step 2: Create Concrete Classes
24  public class CSVDataProcessor : DataProcessor
25  {
26      protected override void ReadData()
27      {
28          Console.WriteLine("Reading data from CSV file.");
29      }
30
31      protected override void ProcessDataCore()
32      {
33          Console.WriteLine("Processing CSV data.");
34      }
35  }
36
37  public class JSONDataProcessor : DataProcessor
38  {
39      protected override void ReadData()
40      {
41          Console.WriteLine("Reading data from JSON file.");
42      }
43
44      protected override void ProcessDataCore()
45      {
46          Console.WriteLine("Processing JSON data.");
47      }
48
49      // Override SaveData if needed
50      protected override void SaveData()
51      {
52          Console.WriteLine("Saving data to the NoSQL database.");
53      }
54  }
55  //Step 3: Using the Template Method
56  class Program
57  {
58      static void Main(string[] args)
59      {
60          DataProcessor csvProcessor = new CSVDataProcessor();

```

```
61         csvProcessor.ProcessData();
62
63         Console.WriteLine();
64
65         DataProcessor jsonProcessor = new JSONDataProcessor();
66         jsonProcessor.ProcessData();
67     }
68 }
69 //output
70 Reading data from CSV file.
71 Processing CSV data.
72 Saving data to the database.
73
74 Reading data from JSON file.
75 Processing JSON data.
76 Saving data to the NoSQL database.
```

1.4 Decorator Pattern - structural

Use the Decorator pattern when you need to be able to assign extra behaviors to objects at runtime without breaking the code that uses these objects.

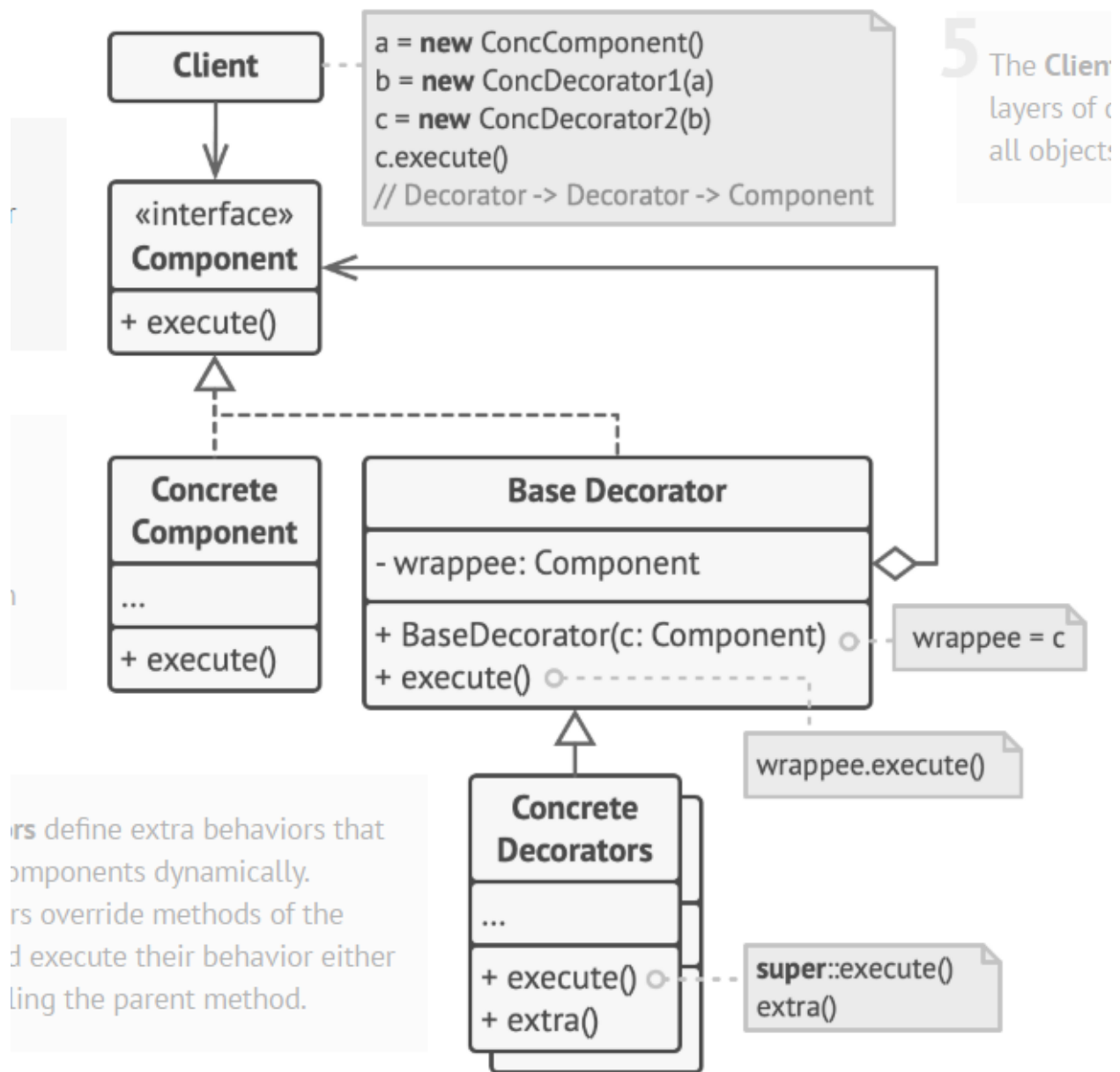


Figure 5: Decorator structure

a scenario where different beverages (like coffee) can have additional ingredients (like milk, sugar, or whipped cream) added dynamically. Each of these ingredients will be treated as decorators, enhancing the base beverage.

```

1  //Step 1: Define the Component Interface
2  public interface IBeverage
3  {
4      string GetDescription();

```

```

5     decimal GetCost();
6 }
7 //Step 2: Create Concrete Components
8 public class Espresso : IBeverage
9 {
10     public string GetDescription()
11     {
12         return "Espresso";
13     }
14
15     public decimal GetCost()
16     {
17         return 2.00m; // Cost of Espresso
18     }
19 }
20
21 public class HouseBlend : IBeverage
22 {
23     public string GetDescription()
24     {
25         return "House Blend Coffee";
26     }
27
28     public decimal GetCost()
29     {
30         return 1.50m; // Cost of House Blend
31     }
32 }
33 //Step 3: Create the Decorator Base Class
34 public abstract class BeverageDecorator : IBeverage
35 {
36     protected IBeverage _beverage;
37
38     public BeverageDecorator(IBeverage beverage)
39     {
40         _beverage = beverage;
41     }
42
43     public virtual string GetDescription()
44     {
45         return _beverage.GetDescription();
46     }
47
48     public virtual decimal GetCost()
49     {
50         return _beverage.GetCost();
51     }
52 }
53 //Step 4: Create Concrete Decorators
54 public class Milk : BeverageDecorator
55 {
56     public Milk(IBeverage beverage) : base(beverage) { }
57
58     public override string GetDescription()
59     {
60         return _beverage.GetDescription() + ", Milk";
61     }
62
63     public override decimal GetCost()

```

```

64     {
65         return _beverage.GetCost() + 0.50m; // Additional cost for Milk
66     }
67 }
68
69 public class Sugar : BeverageDecorator
70 {
71     public Sugar(IBeverage beverage) : base(beverage) { }
72
73     public override string GetDescription()
74     {
75         return _beverage.GetDescription() + ", Sugar";
76     }
77
78     public override decimal GetCost()
79     {
80         return _beverage.GetCost() + 0.25m; // Additional cost for Sugar
81     }
82 }
83
84 public class WhippedCream : BeverageDecorator
85 {
86     public WhippedCream(IBeverage beverage) : base(beverage) { }
87
88     public override string GetDescription()
89     {
90         return _beverage.GetDescription() + ", Whipped Cream";
91     }
92
93     public override decimal GetCost()
94     {
95         return _beverage.GetCost() + 0.75m; // Additional cost for Whipped Cream
96     }
97 }
98 //Step 5: Using the Decorator Pattern
99 class Program
100 {
101     static void Main(string[] args)
102     {
103         // Order an Espresso with Milk and Sugar
104         IBeverage beverage1 = new Espresso();
105         beverage1 = new Milk(beverage1);
106         beverage1 = new Sugar(beverage1);
107         Console.WriteLine($"{beverage1.GetDescription()} - Cost: {beverage1.
            GetCost():C}");
108
109         // Order a House Blend with Whipped Cream
110         IBeverage beverage2 = new HouseBlend();
111         beverage2 = new WhippedCream(beverage2);
112         Console.WriteLine($"{beverage2.GetDescription()} - Cost: {beverage2.
            GetCost():C}");
113     }
114 }
115 //output
116 Espresso, Milk, Sugar - Cost: $2.75
117 House Blend Coffee, Whipped Cream - Cost: $2.25

```

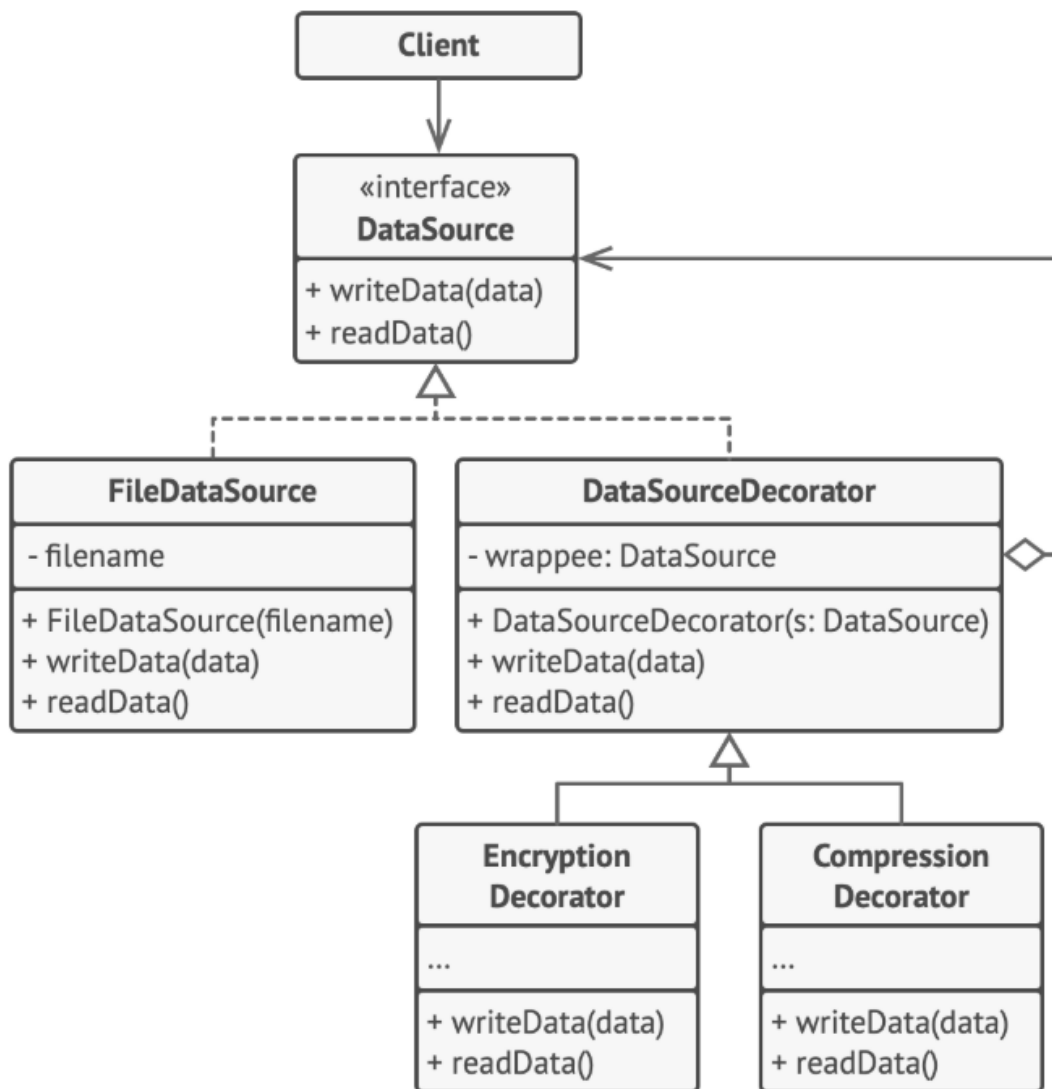


Figure 6: Decorator

```

1  using System;
2  using System.Text;
3  using System.IO;
4  using System.IO.Compression;
5  using System.Security.Cryptography;
6
7  // Component interface
8  public interface IDataSource
9  {
10     void WriteData(string data);
11     string ReadData();
12 }
13
14 // Concrete component - File data source
  
```



```

15 public class FileDataSource : IDataSource
16 {
17     private readonly string _filename;
18
19     public FileDataSource(string filename)
20     {
21         _filename = filename;
22     }
23
24     public void WriteData(string data)
25     {
26         File.WriteAllText(_filename, data);
27         Console.WriteLine($"Writing plain data to file: {_filename}");
28     }
29
30     public string ReadData()
31     {
32         string data = File.ReadAllText(_filename);
33         Console.WriteLine($"Reading plain data from file: {_filename}");
34         return data;
35     }
36 }
37
38 // Base decorator class
39 public abstract class DataSourceDecorator : IDataSource
40 {
41     protected IDataSource _wrapped;
42
43     public DataSourceDecorator(IDataSource source)
44     {
45         _wrapped = source;
46     }
47
48     public virtual void WriteData(string data)
49     {
50         _wrapped.WriteData(data);
51     }
52
53     public virtual string ReadData()
54     {
55         return _wrapped.ReadData();
56     }
57 }
58
59 // Concrete decorator - Encryption
60 public class EncryptionDecorator : DataSourceDecorator
61 {
62     private readonly byte[] _key = Encoding.UTF8.GetBytes("16-byte-enc-key!"); //
        Should be secure in production
63
64     public EncryptionDecorator(IDataSource source) : base(source) { }
65
66     public override void WriteData(string data)
67     {
68         using (Aes aes = Aes.Create())
69         {
70             aes.Key = _key;
71             aes.GenerateIV();
72

```

```

73         using (MemoryStream ms = new MemoryStream())
74         {
75             ms.Write(aes.IV, 0, aes.IV.Length);
76             using (CryptoStream cs = new CryptoStream(ms, aes.CreateEncryptor
77                 (), CryptoStreamMode.Write))
78                 using (StreamWriter sw = new StreamWriter(cs))
79                 {
80                     sw.Write(data);
81                 }
82             string encryptedData = Convert.ToBase64String(ms.ToArray());
83             Console.WriteLine("Encrypting data before writing");
84             _wrapped.WriteData(encryptedData);
85         }
86     }
87 }
88
89 public override string ReadData()
90 {
91     string encryptedData = _wrapped.ReadData();
92     byte[] encryptedBytes = Convert.FromBase64String(encryptedData);
93
94     using (Aes aes = Aes.Create())
95     {
96         aes.Key = _key;
97         byte[] iv = new byte[aes.IV.Length];
98         Array.Copy(encryptedBytes, iv, iv.Length);
99         aes.IV = iv;
100
101         using (MemoryStream ms = new MemoryStream(encryptedBytes, iv.Length,
102             encryptedBytes.Length - iv.Length))
103             using (CryptoStream cs = new CryptoStream(ms, aes.CreateDecryptor(),
104                 CryptoStreamMode.Read))
105                 using (StreamReader sr = new StreamReader(cs))
106                 {
107                     Console.WriteLine("Decrypting data after reading");
108                     return sr.ReadToEnd();
109                 }
110     }
111 }
112
113 // Concrete decorator - Compression
114 public class CompressionDecorator : DataSourceDecorator
115 {
116     public CompressionDecorator(IDataSource source) : base(source) { }
117
118     public override void WriteData(string data)
119     {
120         using (MemoryStream ms = new MemoryStream())
121         {
122             using (GZipStream gzip = new GZipStream(ms, CompressionMode.Compress))
123                 using (StreamWriter sw = new StreamWriter(gzip))
124                 {
125                     sw.Write(data);
126                 }
127
128             string compressedData = Convert.ToBase64String(ms.ToArray());
129             Console.WriteLine("Compressing data before writing");

```

```

129         _wrapped.WriteData(compressedData);
130     }
131 }
132
133 public override string ReadData()
134 {
135     string compressedData = _wrapped.ReadData();
136     byte[] compressedBytes = Convert.FromBase64String(compressedData);
137
138     using (MemoryStream ms = new MemoryStream(compressedBytes))
139     using (GZipStream gzip = new GZipStream(ms, CompressionMode.Decompress))
140     using (StreamReader sr = new StreamReader(gzip))
141     {
142         Console.WriteLine("Decompressing data after reading");
143         return sr.ReadToEnd();
144     }
145 }
146 }
147
148 // Client code
149 class Program
150 {
151     static void Main(string[] args)
152     {
153         string filename = "test.txt";
154         string originalData = "This is some sensitive data that needs protection!"
155             ;
156
157         // Simple file data source
158         IDataSource source = new FileDataSource(filename);
159         source.WriteData(originalData);
160         Console.WriteLine($"Read data: {source.ReadData()}\n");
161
162         // File data source with compression
163         IDataSource compressedSource = new CompressionDecorator(new FileDataSource(
164             filename));
165         compressedSource.WriteData(originalData);
166         Console.WriteLine($"Read data: {compressedSource.ReadData()}\n");
167
168         // File data source with encryption
169         IDataSource encryptedSource = new EncryptionDecorator(new FileDataSource(
170             filename));
171         encryptedSource.WriteData(originalData);
172         Console.WriteLine($"Read data: {encryptedSource.ReadData()}\n");
173
174         // File data source with both compression and encryption
175         IDataSource secureSource = new EncryptionDecorator(
176             new CompressionDecorator(
177                 new FileDataSource(filename)));
178         secureSource.WriteData(originalData);
179         Console.WriteLine($"Read data: {secureSource.ReadData()}\n");
180
181         /* Sample output:
182         Writing plain data to file: test.txt
183         Reading plain data from file: test.txt
184         Read data: This is some sensitive data that needs protection!

```

```
185     Reading plain data from file: test.txt
186     Decompressing data after reading
187     Read data: This is some sensitive data that needs protection!
188
189     Encrypting data before writing
190     Writing plain data to file: test.txt
191     Reading plain data from file: test.txt
192     Decrypting data after reading
193     Read data: This is some sensitive data that needs protection!
194
195     Compressing data before writing
196     Encrypting data before writing
197     Writing plain data to file: test.txt
198     Reading plain data from file: test.txt
199     Decrypting data after reading
200     Decompressing data after reading
201     Read data: This is some sensitive data that needs protection!
202     */
203 }
204 }
```

1.5 Composite Pattern - structural

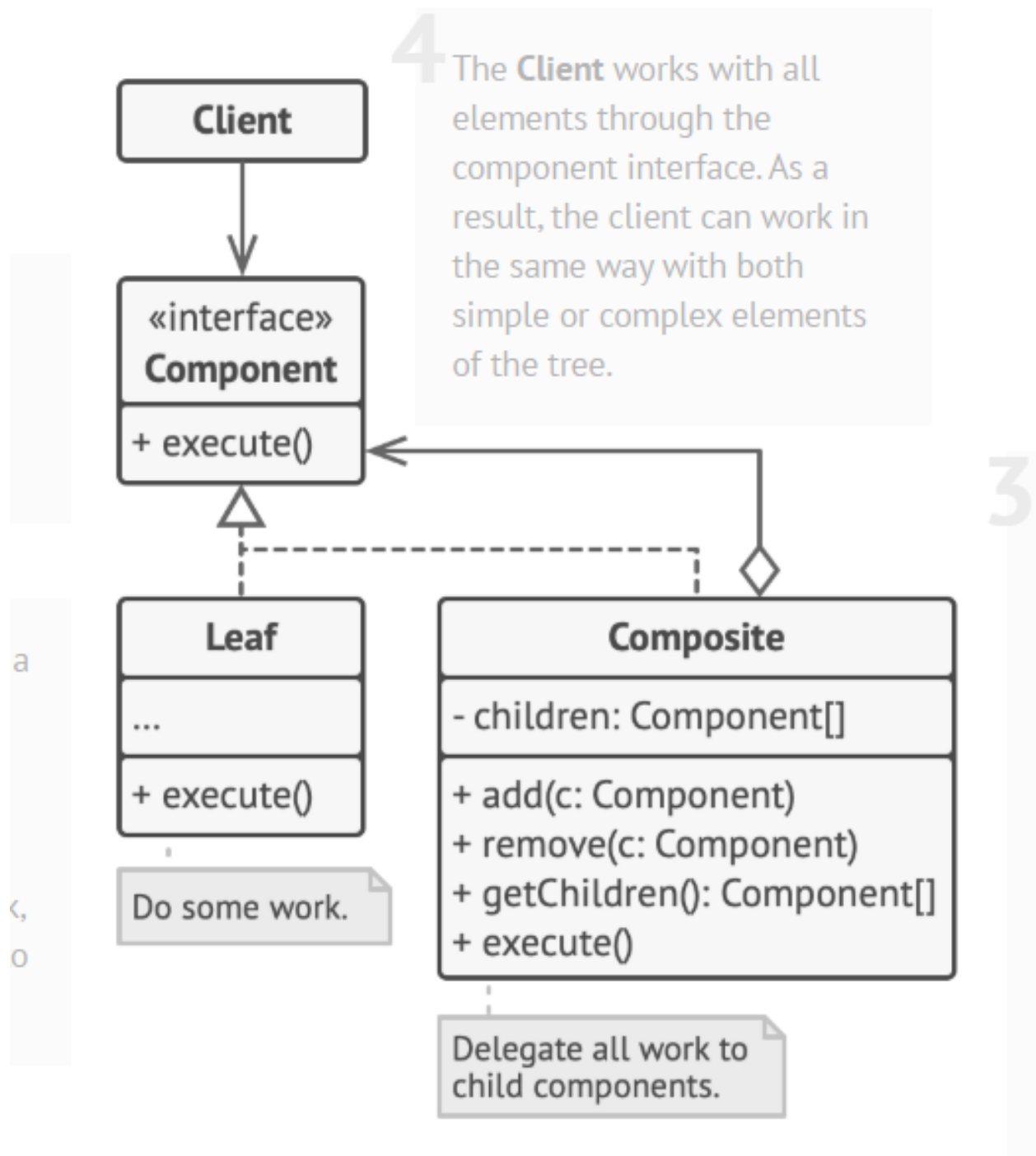


Figure 7: Composite structure

We want to represent a catalog for an e-commerce store where a **Product** can be a simple item (e.g., a Laptop, Phone) or a **Bundle** that contains multiple products (e.g., a tech bundle containing a laptop, phone,

and accessories).

```
1      //Step 1: Define the Component Interface
2      public interface IProduct
3      {
4          string GetName();
5          decimal GetPrice();
6          void Display();
7      }
8      //Step 2: Create Leaf Classes (Simple Products)
9      public class SimpleProduct : IProduct
10     {
11         private string _name;
12         private decimal _price;
13
14         public SimpleProduct(string name, decimal price)
15         {
16             _name = name;
17             _price = price;
18         }
19
20         public string GetName()
21         {
22             return _name;
23         }
24
25         public decimal GetPrice()
26         {
27             return _price;
28         }
29
30         public void Display()
31         {
32             Console.WriteLine($"Product: {_name}, Price: {GetPrice():C}");
33         }
34     }
35     //Step 3: Create Composite Class (Bundle of Products)
36     public class ProductBundle : IProduct
37     {
38         private string _name;
39         private List<IProduct> _products = new List<IProduct>();
40
41         public ProductBundle(string name)
42         {
43             _name = name;
44         }
45
46         public void AddProduct(IProduct product)
47         {
48             _products.Add(product);
49         }
50
51         public void RemoveProduct(IProduct product)
52         {
53             _products.Remove(product);
54         }
55
56         public string GetName()
57         {
```

```

58         return _name;
59     }
60
61     public decimal GetPrice()
62     {
63         decimal total = 0;
64         foreach (var product in _products)
65         {
66             total += product.GetPrice();
67         }
68         return total;
69     }
70
71     public void Display()
72     {
73         Console.WriteLine($"Bundle: {_name}, Total Price: {GetPrice():C}");
74         foreach (var product in _products)
75         {
76             product.Display();
77         }
78     }
79 }
80 //Step 4: Using the Composite Pattern
81 class Program
82 {
83     static void Main(string[] args)
84     {
85         // Create simple products
86         IProduct laptop = new SimpleProduct("Laptop", 1000.00m);
87         IProduct phone = new SimpleProduct("Smartphone", 500.00m);
88         IProduct headphones = new SimpleProduct("Headphones", 150.00m);
89
90         // Create a bundle and add simple products to it
91         ProductBundle techBundle = new ProductBundle("Tech Bundle");
92         techBundle.AddProduct(laptop);
93         techBundle.AddProduct(phone);
94         techBundle.AddProduct(headphones);
95
96         // Display individual products
97         Console.WriteLine("Individual Products:");
98         laptop.Display();
99         phone.Display();
100        headphones.Display();
101
102        Console.WriteLine("\nBundle:");
103        // Display the bundle
104        techBundle.Display();
105    }
106 }
107 \\output
108 Individual Products:
109 Product: Laptop, Price: $1,000.00
110 Product: Smartphone, Price: $500.00
111 Product: Headphones, Price: $150.00
112
113 Bundle:
114 Bundle: Tech Bundle, Total Price: $1,650.00
115 Product: Laptop, Price: $1,000.00
116 Product: Smartphone, Price: $500.00

```

117

Product: Headphones, Price: \$150.00

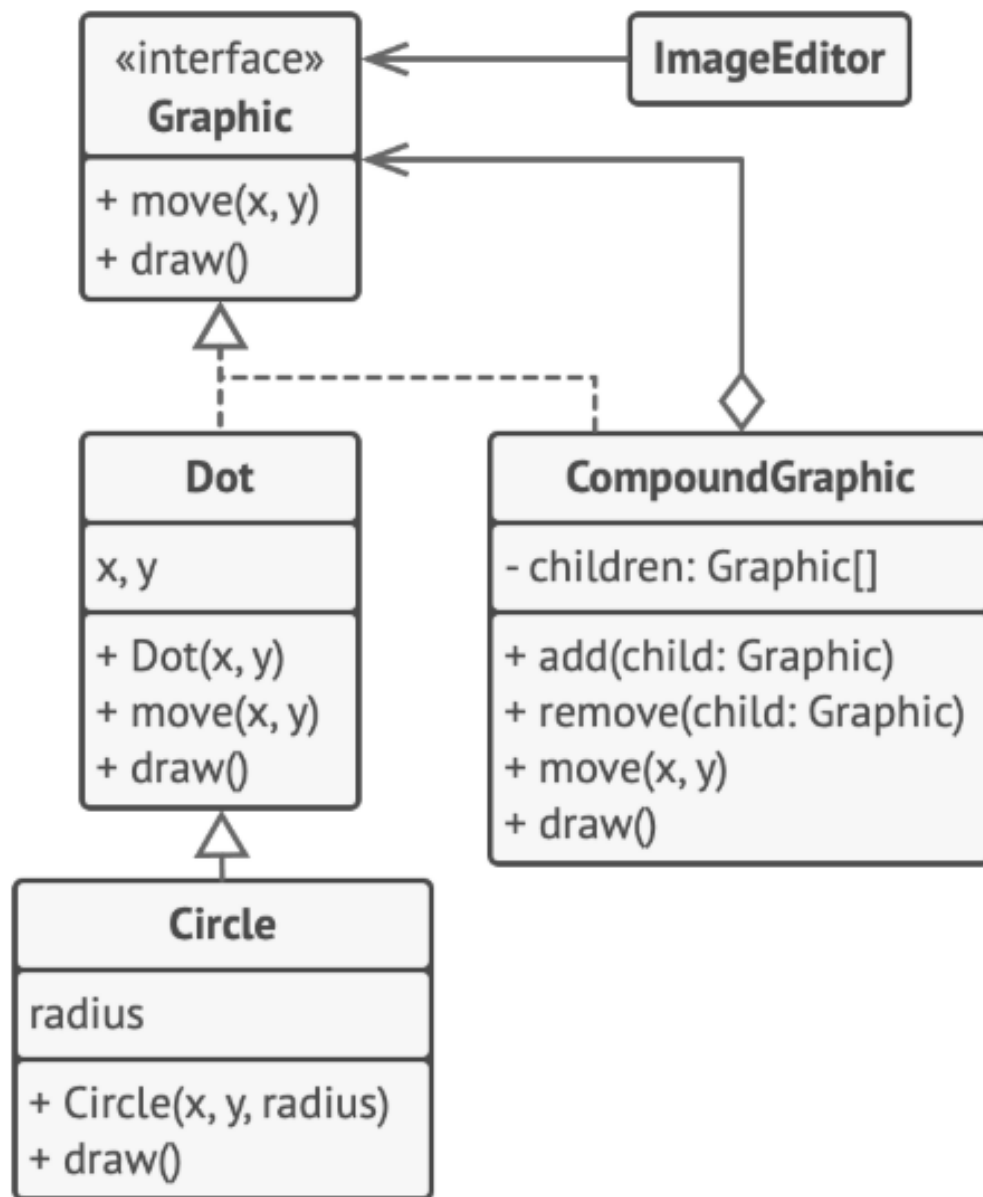


Figure 8: Composite

```

1      using System;
2      using System.Collections.Generic;
3
4      // Component interface
5      public interface IGraphic
6      {
7          void Move(int x, int y);
8          void Draw();
9      }
  
```

```

9  }
10
11  // Leaf class - Dot
12  public class Dot : IGraphic
13  {
14      public int X { get; private set; }
15      public int Y { get; private set; }
16
17      public Dot(int x, int y)
18      {
19          X = x;
20          Y = y;
21      }
22
23      public void Move(int x, int y)
24      {
25          X += x;
26          Y += y;
27          Console.WriteLine($"Dot moved to ({X}, {Y})");
28      }
29
30      public void Draw()
31      {
32          Console.WriteLine($"Drawing a dot at ({X}, {Y})");
33      }
34  }
35
36  // Leaf class - Circle
37  public class Circle : IGraphic
38  {
39      public int X { get; private set; }
40      public int Y { get; private set; }
41      public int Radius { get; private set; }
42
43      public Circle(int x, int y, int radius)
44      {
45          X = x;
46          Y = y;
47          Radius = radius;
48      }
49
50      public void Move(int x, int y)
51      {
52          X += x;
53          Y += y;
54          Console.WriteLine($"Circle moved to ({X}, {Y})");
55      }
56
57      public void Draw()
58      {
59          Console.WriteLine($"Drawing a circle at ({X}, {Y}) with radius {Radius}");
60      }
61  }
62
63  // Composite class - CompoundGraphic
64  public class CompoundGraphic : IGraphic
65  {
66      private readonly List<IGraphic> _children = new List<IGraphic>();
67

```

```

68     public void Add(IGraphic child)
69     {
70         _children.Add(child);
71         Console.WriteLine("Added child to compound graphic");
72     }
73
74     public void Remove(IGraphic child)
75     {
76         _children.Remove(child);
77         Console.WriteLine("Removed child from compound graphic");
78     }
79
80     public void Move(int x, int y)
81     {
82         foreach (var child in _children)
83         {
84             child.Move(x, y);
85         }
86         Console.WriteLine($"Moved all children by ({x}, {y})");
87     }
88
89     public void Draw()
90     {
91         Console.WriteLine("Drawing a compound graphic:");
92         foreach (var child in _children)
93         {
94             child.Draw();
95         }
96         Console.WriteLine("Finished drawing compound graphic");
97     }
98 }
99
100 // Client code - ImageEditor
101 class Program
102 {
103     static void Main(string[] args)
104     {
105         // Create simple graphics
106         var dot1 = new Dot(10, 20);
107         var dot2 = new Dot(30, 40);
108         var circle = new Circle(50, 60, 15);
109
110         // Create a compound graphic
111         var compoundGraphic = new CompoundGraphic();
112         compoundGraphic.Add(dot1);
113         compoundGraphic.Add(dot2);
114         compoundGraphic.Add(circle);
115
116         // Create another compound graphic
117         var anotherCompound = new CompoundGraphic();
118         anotherCompound.Add(new Circle(100, 100, 20));
119         anotherCompound.Add(new Dot(120, 120));
120
121         // Add the second compound to the first one
122         compoundGraphic.Add(anotherCompound);
123
124         Console.WriteLine("\nDrawing all graphics:");
125         compoundGraphic.Draw();
126

```

```

127 Console.WriteLine("\nMoving all graphics by (5, 5):");
128 compoundGraphic.Move(5, 5);
129
130 Console.WriteLine("\nDrawing after move:");
131 compoundGraphic.Draw();
132
133 /* Output:
134 Added child to compound graphic
135 Added child to compound graphic
136 Added child to compound graphic
137 Added child to compound graphic
138 Added child to compound graphic
139
140 Drawing all graphics:
141 Drawing a compound graphic:
142 Drawing a dot at (10, 20)
143 Drawing a dot at (30, 40)
144 Drawing a circle at (50, 60) with radius 15
145 Drawing a compound graphic:
146 Drawing a circle at (100, 100) with radius 20
147 Drawing a dot at (120, 120)
148 Finished drawing compound graphic
149 Finished drawing compound graphic
150
151 Moving all graphics by (5, 5):
152 Dot moved to (15, 25)
153 Dot moved to (35, 45)
154 Circle moved to (55, 65)
155 Drawing a compound graphic:
156 Circle moved to (105, 105)
157 Dot moved to (125, 125)
158 Finished drawing compound graphic
159 Moved all children by (5, 5)
160
161 Drawing after move:
162 Drawing a compound graphic:
163 Drawing a dot at (15, 25)
164 Drawing a dot at (35, 45)
165 Drawing a circle at (55, 65) with radius 15
166 Drawing a compound graphic:
167 Drawing a circle at (105, 105) with radius 20
168 Drawing a dot at (125, 125)
169 Finished drawing compound graphic
170 Finished drawing compound graphic
171 */
172 }
173 }

```

1.6 Adapter Pattern - structural

Adapter is a structural design pattern that allows objects with incompatible interfaces to collaborate.

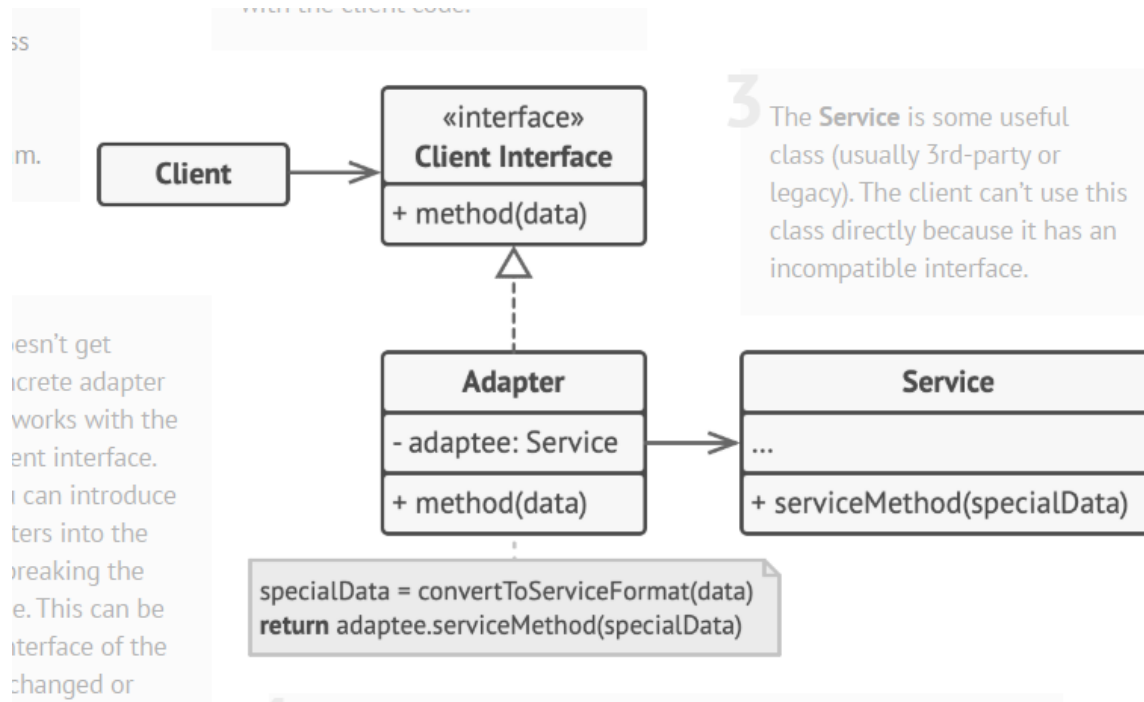


Figure 9: Adapter structure

Suppose we have an application that plays audio files. It uses a standard interface to play .mp3 files, but now we need to integrate a third-party library that plays .wav files. The third-party library has a different interface, so we'll use the Adapter Pattern to make it compatible with our existing system.

```

1  //Step 1: Define the Target Interface
2  public interface IMediaPlayer
3  {
4      void Play(string audioType, string fileName);
5  }
6  //Step 2: Create the Concrete Implementation for the Target Interface
7  public class Mp3Player : IMediaPlayer
8  {
9      public void Play(string audioType, string fileName)
10     {
11         if (audioType.Equals("mp3", StringComparison.OrdinalIgnoreCase))
12         {
13             Console.WriteLine($"Playing mp3 file. Name: {fileName}");
14         }
15         else
16         {
17             Console.WriteLine($"Invalid media type: {audioType}. mp3 format
18                 supported only.");
19         }
20     }
21 }
//Step 3: Define the Adaptee (Third-Party Library)

```

```

22 // This is the existing third-party class with a different interface
23 public class WavPlayer
24 {
25     public void PlayWavFile(string fileName)
26     {
27         Console.WriteLine($"Playing wav file. Name: {fileName}");
28     }
29 }
30 //Step 4: Create the Adapter Class
31 public class MediaAdapter : IMediaPlayer
32 {
33     private readonly WavPlayer _wavPlayer;
34
35     public MediaAdapter()
36     {
37         _wavPlayer = new WavPlayer();
38     }
39
40     public void Play(string audioType, string fileName)
41     {
42         if (audioType.Equals("wav", StringComparison.OrdinalIgnoreCase))
43         {
44             _wavPlayer.PlayWavFile(fileName);
45         }
46         else
47         {
48             Console.WriteLine($"Invalid media type: {audioType}. wav format
49                             supported only via adapter.");
50         }
51     }
52 }
53 //Step 5: Create the Client Class
54 public class AudioPlayer : IMediaPlayer
55 {
56     private readonly MediaAdapter _mediaAdapter;
57
58     public AudioPlayer()
59     {
60         _mediaAdapter = new MediaAdapter();
61     }
62
63     public void Play(string audioType, string fileName)
64     {
65         if (audioType.Equals("mp3", StringComparison.OrdinalIgnoreCase))
66         {
67             Console.WriteLine($"Playing mp3 file. Name: {fileName}");
68         }
69         else if (audioType.Equals("wav", StringComparison.OrdinalIgnoreCase))
70         {
71             // Use the adapter to play wav files
72             _mediaAdapter.Play(audioType, fileName);
73         }
74         else
75         {
76             Console.WriteLine($"Invalid media type: {audioType}. mp3 and wav
77                             formats supported.");
78         }
79     }
80 }

```

```

79 //Step 6: Using the Adapter Pattern
80 class Program
81 {
82     static void Main(string[] args)
83     {
84         IMediaPlayer audioPlayer = new AudioPlayer();
85         audioPlayer.Play("mp3", "song.mp3");
86         audioPlayer.Play("wav", "track.wav");
87         audioPlayer.Play("mp4", "video.mp4"); // Invalid type, not supported
88     }
89 }
90 \\output
91 Playing mp3 file. Name: song.mp3
92 Playing wav file. Name: track.wav
93 Invalid media type: mp4. mp3 and wav formats supported.

```

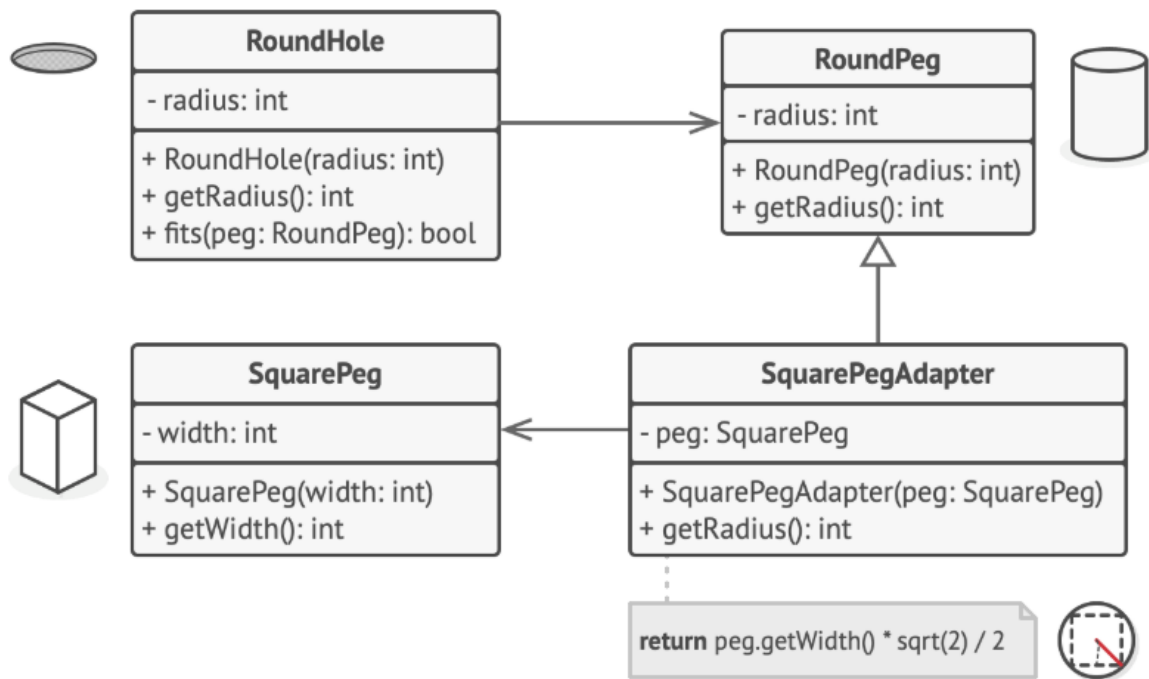


Figure 10: Adapter

```

1     using System;
2
3     // Target interface that RoundHole expects
4     public interface IRoundPeg
5     {
6         int GetRadius();
7     }
8
9     // RoundPeg is compatible with RoundHole
10    public class RoundPeg : IRoundPeg

```

```

11 {
12     private readonly int _radius;
13
14     public RoundPeg(int radius)
15     {
16         _radius = radius;
17     }
18
19     public int GetRadius()
20     {
21         return _radius;
22     }
23 }
24
25 // SquarePeg is incompatible with RoundHole
26 public class SquarePeg
27 {
28     private readonly int _width;
29
30     public SquarePeg(int width)
31     {
32         _width = width;
33     }
34
35     public int GetWidth()
36     {
37         return _width;
38     }
39 }
40
41 // Adapter makes SquarePeg compatible with RoundHole
42 public class SquarePegAdapter : IRoundPeg
43 {
44     private readonly SquarePeg _peg;
45
46     public SquarePegAdapter(SquarePeg peg)
47     {
48         _peg = peg;
49     }
50
51     public int GetRadius()
52     {
53         // Calculate minimum circle radius that can fit this square peg
54         return (int)(_peg.GetWidth() * Math.Sqrt(2) / 2);
55     }
56 }
57
58 // RoundHole class that works with IRoundPeg
59 public class RoundHole
60 {
61     private readonly int _radius;
62
63     public RoundHole(int radius)
64     {
65         _radius = radius;
66     }
67
68     public int GetRadius()
69     {

```



```

70     return _radius;
71 }
72
73 public bool Fits(IRoundPeg peg)
74 {
75     return GetRadius() >= peg.GetRadius();
76 }
77 }
78
79 // Client code
80 class Program
81 {
82     static void Main(string[] args)
83     {
84         // Create a round hole with radius 5
85         var hole = new RoundHole(5);
86
87         // Create a round peg with radius 4 (fits)
88         var roundPeg = new RoundPeg(4);
89         Console.WriteLine($"Round peg fits: {hole.Fits(roundPeg)}"); // True
90
91         // Create a square peg with width 8 (wouldn't fit without adapter)
92         var smallSquarePeg = new SquarePeg(8);
93         // hole.Fits(smallSquarePeg); // This wouldn't compile
94
95         // Use adapter to make square peg work with round hole
96         var smallSquarePegAdapter = new SquarePegAdapter(smallSquarePeg);
97         Console.WriteLine($"Small square peg fits: {hole.Fits(
98             smallSquarePegAdapter)}"); // True
99
100         var largeSquarePeg = new SquarePeg(10);
101         var largeSquarePegAdapter = new SquarePegAdapter(largeSquarePeg);
102         Console.WriteLine($"Large square peg fits: {hole.Fits(
103             largeSquarePegAdapter)}"); // False
104
105         /* Output:
106         Round peg fits: True
107         Small square peg fits: True
108         Large square peg fits: False
109         */
110     }
111 }

```

1.7 Singleton Pattern - creational

Make the default constructor private, to prevent other objects from using the new operator with the Singleton class.

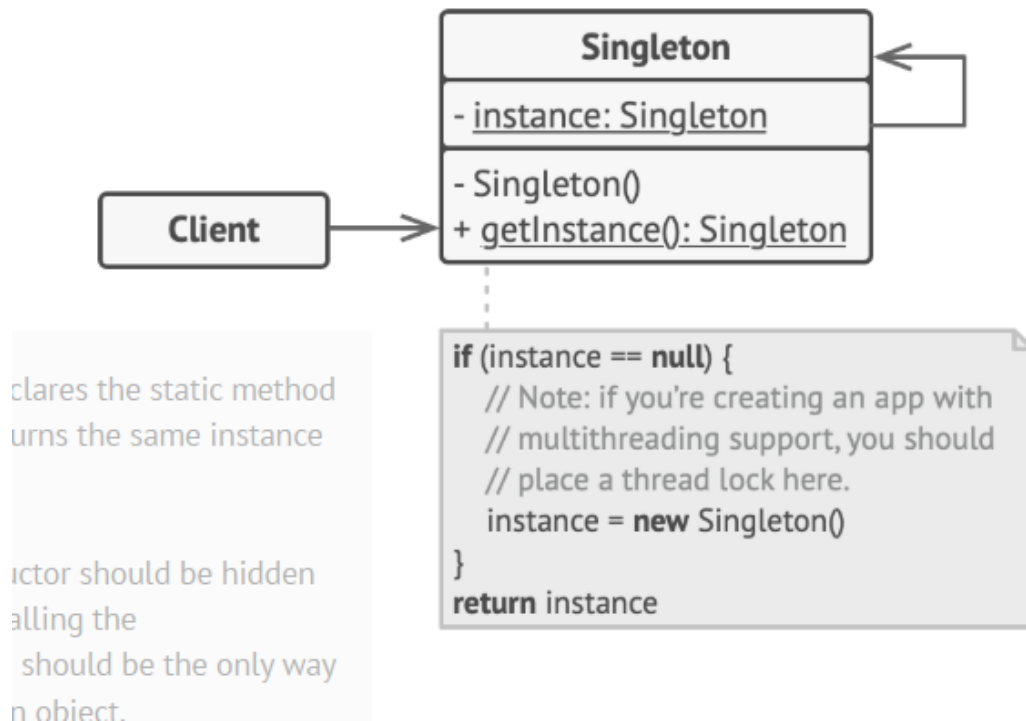


Figure 11: singleton structure

Let's create a simple logger class that ensures only one instance exists. The logger will write messages to the console, and we want to guarantee that the same logger instance is used throughout the application.

```

1      using System;
2
3      public sealed class Logger
4      {
5          // Private static readonly instance - ensures thread safety
6          private static readonly Logger _instance = new Logger();
7
8          // Private constructor prevents instantiation from other classes
9          private Logger()
10         {
11             Console.WriteLine("Logger initialized.");
12         }
13
14         // Public static property to get the instance
15         public static Logger Instance
16         {
17             get
18             {
19                 return _instance;
20             }
21         }
22
23         // Method to log messages
  
```

```

24     public void Log(string message)
25     {
26         Console.WriteLine($"[LOG]: {message}");
27     }
28 }
29 //usage example:
30 class Program
31 {
32     static void Main(string[] args)
33     {
34         Logger logger1 = Logger.Instance;
35         Logger logger2 = Logger.Instance;
36
37         logger1.Log("Application started.");
38         logger2.Log("Still using the same logger instance.");
39
40         // Check if both instances are the same
41         Console.WriteLine($"logger1 and logger2 are the same instance: {object.
            ReferenceEquals(logger1, logger2)}");
42     }
43 }
44 //output:
45 Logger initialized.
46 [LOG]: Application started.
47 [LOG]: Still using the same logger instance.
48 logger1 and logger2 are the same instance: True

```

1.8 Facade Pattern - structural

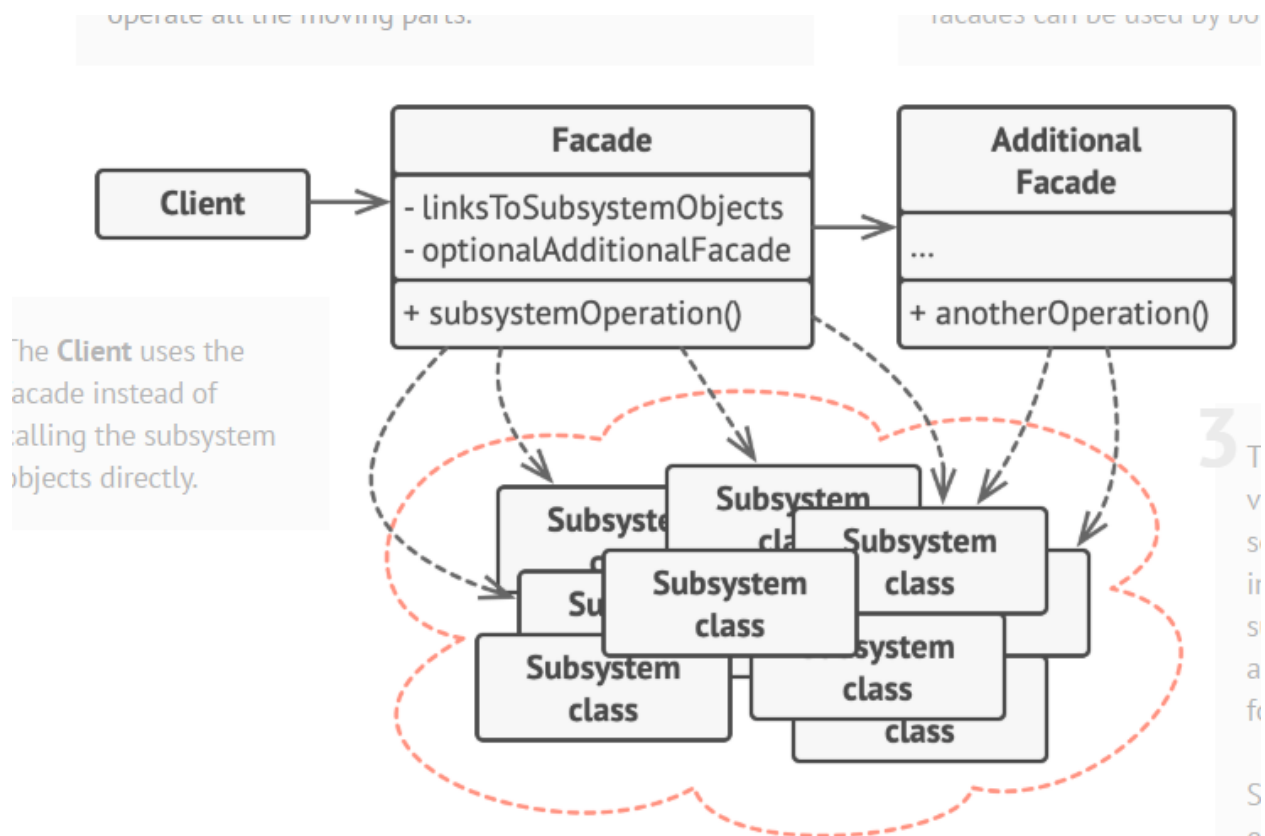


Figure 12: Facade structure

Having a facade is handy when you need to integrate your app with a sophisticated library that has dozens of features, but you just need a tiny bit of its functionality.

```

1      using System;
2
3      // Complex subsystem classes
4      public class Amplifier
5      {
6          public void TurnOn() => Console.WriteLine("Amplifier turned on");
7          public void SetVolume(int level) => Console.WriteLine($"Volume set to {level}");
8          public void TurnOff() => Console.WriteLine("Amplifier turned off");
9      }
10
11     public class DvdPlayer
12     {
13         public void TurnOn() => Console.WriteLine("DVD Player turned on");
14         public void PlayMovie(string movie) => Console.WriteLine($"Playing movie: {movie}");
15         public void TurnOff() => Console.WriteLine("DVD Player turned off");
16     }
17
18     public class Projector
19     {

```

```

20     public void TurnOn() => Console.WriteLine("Projector turned on");
21     public void SetInput(string input) => Console.WriteLine($"Input set to {input}
22         ");
23     public void TurnOff() => Console.WriteLine("Projector turned off");
24 }
25
26 public class Lights
27 {
28     public void Dim(int percentage) => Console.WriteLine($"Lights dimmed to {
29         percentage}%");
30     public void TurnOn() => Console.WriteLine("Lights turned on");
31 }
32
33 // Facade class
34 public class HomeTheaterFacade
35 {
36     private readonly Amplifier _amp;
37     private readonly DvdPlayer _dvd;
38     private readonly Projector _projector;
39     private readonly Lights _lights;
40
41     public HomeTheaterFacade(Amplifier amp, DvdPlayer dvd, Projector projector,
42         Lights lights)
43     {
44         _amp = amp;
45         _dvd = dvd;
46         _projector = projector;
47         _lights = lights;
48     }
49
50     public void WatchMovie(string movie)
51     {
52         Console.WriteLine("Get ready to watch a movie...");
53
54         _lights.Dim(10);
55         _amp.TurnOn();
56         _amp.SetVolume(5);
57         _projector.TurnOn();
58         _projector.SetInput("DVD");
59         _dvd.TurnOn();
60         _dvd.PlayMovie(movie);
61     }
62
63     public void EndMovie()
64     {
65         Console.WriteLine("Shutting down the theater...");
66
67         _dvd.TurnOff();
68         _amp.TurnOff();
69         _projector.TurnOff();
70         _lights.TurnOn();
71     }
72 }
73
74 // Client code
75 class Program
76 {
77     static void Main(string[] args)
78     {

```

```

76 // Create subsystem components
77 var amp = new Amplifier();
78 var dvd = new DvdPlayer();
79 var projector = new Projector();
80 var lights = new Lights();
81
82 // Create facade
83 var homeTheater = new HomeTheaterFacade(amp, dvd, projector, lights);
84
85 // Use simplified interface
86 homeTheater.WatchMovie("The Matrix");
87 Console.WriteLine("\nEnjoy the movie!\n");
88 homeTheater.EndMovie();
89 }
90 }

```

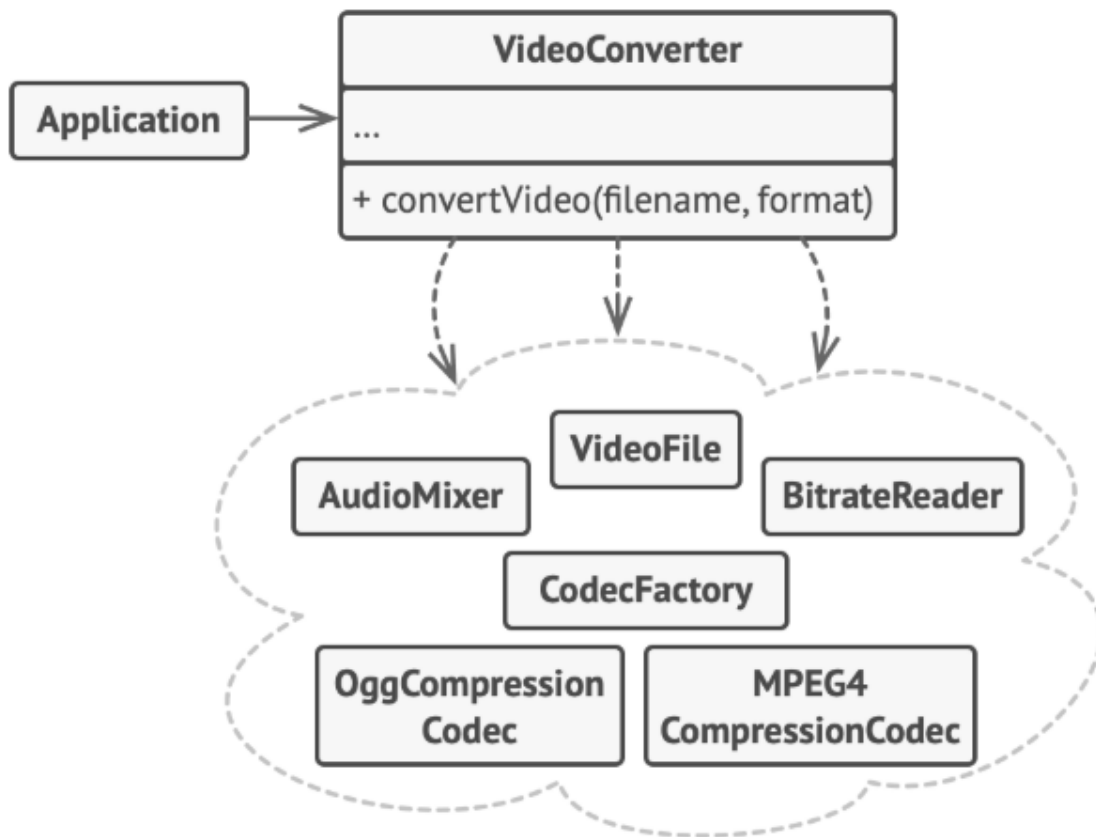


Figure 13: Facade

```

1 using System;
2
3 // Subsystem classes
4 public class VideoFile

```

```

5 {
6     private readonly string _filename;
7
8     public VideoFile(string filename)
9     {
10         _filename = filename;
11     }
12
13     public string GetFilename()
14     {
15         return _filename;
16     }
17
18     public void Save(string outputFilename)
19     {
20         Console.WriteLine($"Saving video file to: {outputFilename}");
21     }
22 }
23
24 public class CodecFactory
25 {
26     public string ExtractCodec(VideoFile file)
27     {
28         Console.WriteLine($"Extracting codec from: {file.GetFilename()}");
29         // In a real implementation, this would analyze the file to determine
30         // codec
31         return file.GetFilename().EndsWith(".mp4") ? "mpeg4" : "ogg";
32     }
33
34     public string GetDestinationCodec(string format)
35     {
36         Console.WriteLine($"Getting destination codec for format: {format}");
37         return format.ToLower() == "mp4" ? "mpeg4" : "ogg";
38     }
39 }
40
41 public class BitrateReader
42 {
43     public VideoFile Read(VideoFile file, string codec)
44     {
45         Console.WriteLine($"Reading file {file.GetFilename()} with codec {codec}");
46         ;
47         return file;
48     }
49
50     public VideoFile Convert(VideoFile buffer, string destinationCodec)
51     {
52         Console.WriteLine($"Converting to {destinationCodec} format");
53         return new VideoFile($"converted_{buffer.GetFilename()}.{destinationCodec}");
54     }
55 }
56
57 public class AudioMixer
58 {
59     public VideoFile FixAudio(VideoFile file)
60     {
61         Console.WriteLine("Fixing audio channels and normalizing volume");
62         return file;
63     }
64 }

```

```

61     }
62 }
63
64 public class CompressionCodec
65 {
66     public void Compress(VideoFile file)
67     {
68         Console.WriteLine("Compressing video file to reduce size");
69     }
70 }
71
72 // Facade class
73 public class VideoConverter
74 {
75     public VideoFile ConvertVideo(string filename, string format)
76     {
77         Console.WriteLine($"VideoConversionFacade: Starting conversion of {
78             filename} to {format}");
79
80         var file = new VideoFile(filename);
81         var codecFactory = new CodecFactory();
82
83         // Determine source and destination codecs
84         string sourceCodec = codecFactory.ExtractCodec(file);
85         string destinationCodec = codecFactory.GetDestinationCodec(format);
86
87         // Read and convert the file
88         var bitrateReader = new BitrateReader();
89         VideoFile buffer = bitrateReader.Read(file, sourceCodec);
90         VideoFile intermediateResult = bitrateReader.Convert(buffer,
91             destinationCodec);
92
93         // Process audio
94         var audioMixer = new AudioMixer();
95         VideoFile audioFixed = audioMixer.FixAudio(intermediateResult);
96
97         // Apply compression if needed
98         if (format.ToLower() == "mp4")
99         {
100             var compressor = new CompressionCodec();
101             compressor.Compress(audioFixed);
102         }
103
104         // Save the final result
105         string outputFilename = filename.Split('.')[0] + $".{format}";
106         audioFixed.Save(outputFilename);
107
108         Console.WriteLine($"VideoConversionFacade: Conversion completed to {
109             outputFilename}");
110         return audioFixed;
111     }
112 }
113
114 // Client code
115 class Program
116 {
117     static void Main(string[] args)
118     {
119         var converter = new VideoConverter();

```



```

117     Console.WriteLine("Converting video to MP4:");
118     converter.ConvertVideo("sample.ogg", "mp4");
119
120
121     Console.WriteLine("\nConverting video to OGG:");
122     converter.ConvertVideo("sample.mpeg", "ogg");
123
124     /* Output:
125     Converting video to MP4:
126     VideoConversionFacade: Starting conversion of sample.ogg to mp4
127     Extracting codec from: sample.ogg
128     Getting destination codec for format: mp4
129     Reading file sample.ogg with codec ogg
130     Converting to mpeg4 format
131     Fixing audio channels and normalizing volume
132     Compressing video file to reduce size
133     Saving video file to: sample.mp4
134     VideoConversionFacade: Conversion completed to sample.mp4
135
136     Converting video to OGG:
137     VideoConversionFacade: Starting conversion of sample.mpeg to ogg
138     Extracting codec from: sample.mpeg
139     Getting destination codec for format: ogg
140     Reading file sample.mpeg with codec mpeg4
141     Converting to ogg format
142     Fixing audio channels and normalizing volume
143     Saving video file to: sample.ogg
144     VideoConversionFacade: Conversion completed to sample.ogg
145     */
146 }
147

```

1.9 Flyweight Pattern - structural

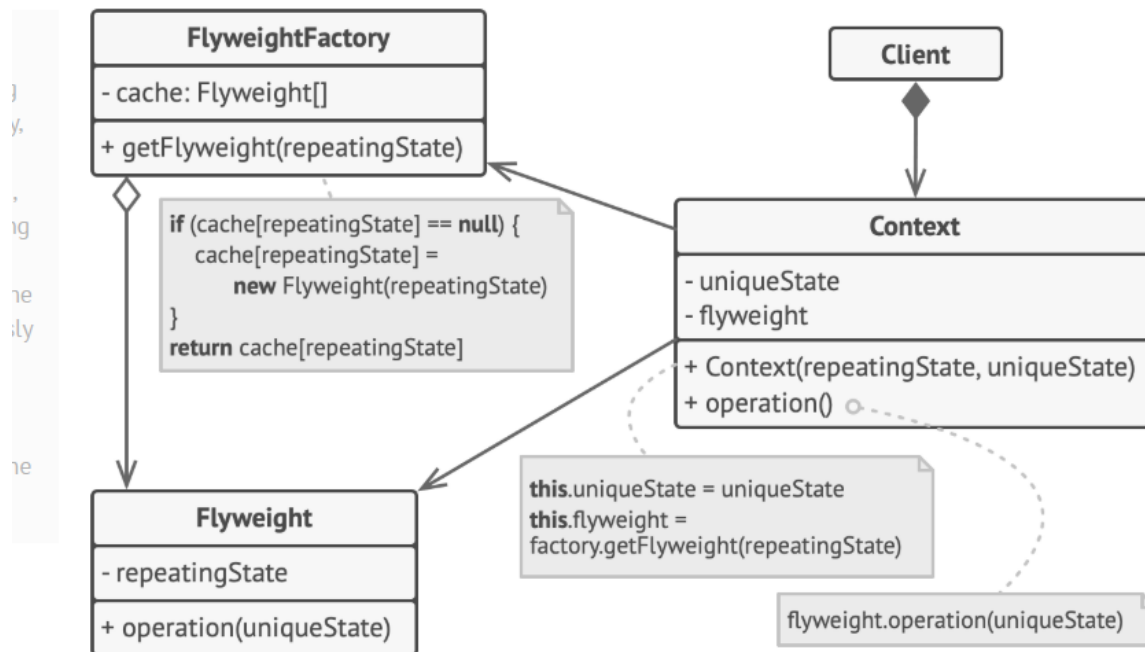


Figure 14: Flyweight structure

```

1  using System;
2  using System.Collections.Generic;
3
4  // Flyweight Factory
5  public class TextFormatFactory
6  {
7      private readonly Dictionary<string, ITextFormat> _formatCache = new Dictionary
8          <string, ITextFormat>();
9
10     public ITextFormat GetTextFormat(string fontName, int fontSize, bool isBold)
11     {
12         // Create a key for the format combination
13         string key = $"{fontName}-{fontSize}-{isBold}";
14
15         // Return existing format or create a new one
16         if (!_formatCache.ContainsKey(key))
17         {
18             _formatCache[key] = new TextFormat(fontName, fontSize, isBold);
19             Console.WriteLine($"Creating new format: {key}");
20         }
21         else
22         {
23             Console.WriteLine($"Reusing existing format: {key}");
24         }
25     }
26 }
  
```

```

25         return _formatCache[key];
26     }
27 }
28
29 // Flyweight Interface
30 public interface ITextFormat
31 {
32     void ApplyFormat(string character);
33 }
34
35 // Concrete Flyweight
36 public class TextFormat : ITextFormat
37 {
38     public string FontName { get; }
39     public int FontSize { get; }
40     public bool IsBold { get; }
41
42     public TextFormat(string fontName, int fontSize, bool isBold)
43     {
44         FontName = fontName;
45         FontSize = fontSize;
46         IsBold = isBold;
47     }
48
49     public void ApplyFormat(string character)
50     {
51         Console.WriteLine($"Character '{character}' displayed with {FontName},
52                             size {FontSize}, bold: {IsBold}");
53     }
54 }
55
56 // Client (Character) class that uses flyweights
57 public class Character
58 {
59     private readonly string _value;
60     private readonly ITextFormat _format;
61
62     public Character(string value, TextFormatFactory factory, string fontName, int
63                     fontSize, bool isBold)
64     {
65         _value = value;
66         _format = factory.GetTextFormat(fontName, fontSize, isBold);
67     }
68
69     public void Display()
70     {
71         _format.ApplyFormat(_value);
72     }
73 }
74
75 // Usage
76 class Program
77 {
78     static void Main(string[] args)
79     {
80         var formatFactory = new TextFormatFactory();
81
82         // Create characters with different formats
83         var char1 = new Character("A", formatFactory, "Arial", 12, true);

```

```

82     var char2 = new Character("B", formatFactory, "Times New Roman", 14, false
83     );
84     var char3 = new Character("C", formatFactory, "Arial", 12, true); // Same
85     format as char1
86     var char4 = new Character("D", formatFactory, "Arial", 12, false);
87     var char5 = new Character("E", formatFactory, "Times New Roman", 14, false
88     ); // Same format as char2
89
90     Console.WriteLine("\nDisplaying characters:");
91     char1.Display();
92     char2.Display();
93     char3.Display();
94     char4.Display();
95     char5.Display();
96 }
97 }
98 //output
99 Creating new format: Arial-12-True
100 Creating new format: Times New Roman-14-False
101 Reusing existing format: Arial-12-True
102 Creating new format: Arial-12-False
103 Reusing existing format: Times New Roman-14-False
104
105 Displaying characters:
106 Character 'A' displayed with Arial, size 12, bold: True
107 Character 'B' displayed with Times New Roman, size 14, bold: False
108 Character 'C' displayed with Arial, size 12, bold: True
109 Character 'D' displayed with Arial, size 12, bold: False
110 Character 'E' displayed with Times New Roman, size 14, bold: False

```

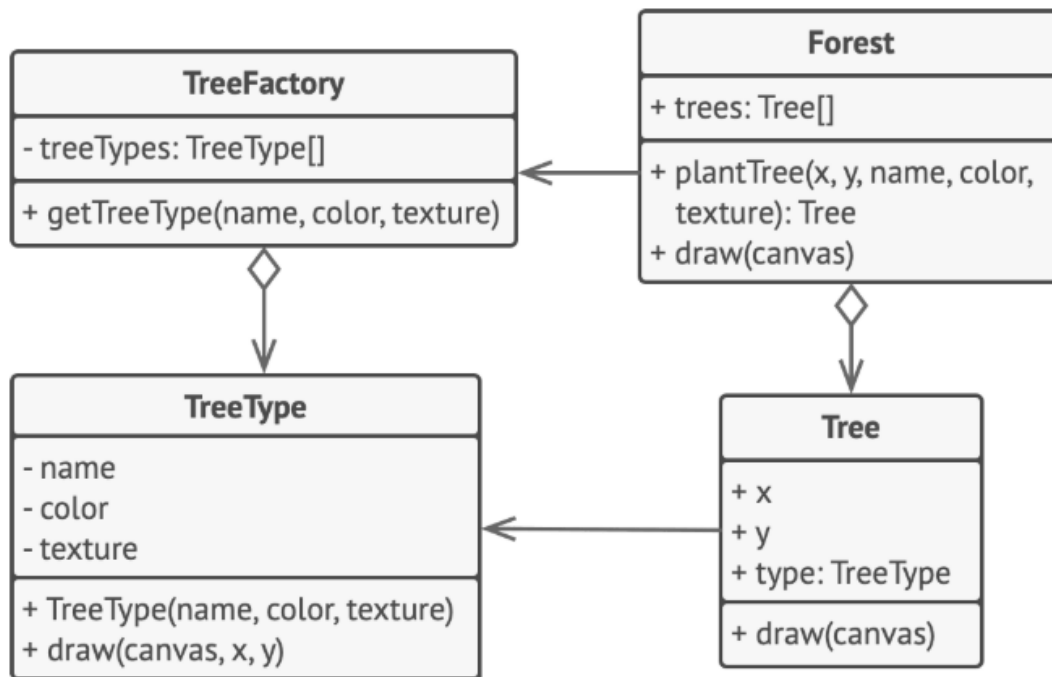


Figure 15: Flyweight

```

1  using System;
2  using System.Collections.Generic;
3
4  // Flyweight (intrinsic state)
5  public class TreeType
6  {
7      public string Name { get; }
8      public string Color { get; }
9      public string Texture { get; }
10
11     public TreeType(string name, string color, string texture)
12     {
13         Name = name;
14         Color = color;
15         Texture = texture;
16     }
17
18     public void Draw(string canvas, int x, int y)
19     {
20         Console.WriteLine($"Drawing {Name} tree at ({x},{y}) with {Color} color
21                             and {Texture} texture");
22     }
23 }
24
25 // Flyweight Factory
26 public class TreeFactory
27 {
28     private readonly Dictionary<string, TreeType> _treeTypes = new Dictionary<

```

```

28         string, TreeType>();
29     public TreeType GetTreeType(string name, string color, string texture)
30     {
31         string key = $"{name}_{color}_{texture}";
32
33         if (!_treeTypes.ContainsKey(key))
34         {
35             _treeTypes[key] = new TreeType(name, color, texture);
36             Console.WriteLine($"Created new TreeType: {key}");
37         }
38         else
39         {
40             Console.WriteLine($"Reused existing TreeType: {key}");
41         }
42
43         return _treeTypes[key];
44     }
45 }
46
47 // Context (extrinsic state)
48 public class Tree
49 {
50     public int X { get; }
51     public int Y { get; }
52     public TreeType Type { get; }
53
54     public Tree(int x, int y, TreeType type)
55     {
56         X = x;
57         Y = y;
58         Type = type;
59     }
60
61     public void Draw(string canvas)
62     {
63         Type.Draw(canvas, X, Y);
64     }
65 }
66
67 // Client (maintains trees)
68 public class Forest
69 {
70     private readonly List<Tree> _trees = new List<Tree>();
71     private readonly TreeFactory _treeFactory = new TreeFactory();
72
73     public void PlantTree(int x, int y, string name, string color, string texture)
74     {
75         TreeType type = _treeFactory.GetTreeType(name, color, texture);
76         Tree tree = new Tree(x, y, type);
77         _trees.Add(tree);
78     }
79
80     public void DrawForest(string canvas)
81     {
82         Console.WriteLine($"\\nDrawing entire forest on {canvas}:");
83         foreach (var tree in _trees)
84         {
85             tree.Draw(canvas);

```

```

86     }
87 }
88 }
89
90 // Usage
91 class Program
92 {
93     static void Main(string[] args)
94     {
95         Forest forest = new Forest();
96
97         // Plant trees - notice duplicate types
98         forest.PlantTree(1, 2, "Oak", "Green", "Rough");
99         forest.PlantTree(5, 3, "Maple", "Red", "Smooth");
100        forest.PlantTree(3, 1, "Oak", "Green", "Rough"); // Same type as first
            tree
101        forest.PlantTree(7, 5, "Pine", "Dark Green", "Needles");
102        forest.PlantTree(2, 4, "Maple", "Red", "Smooth"); // Same type as second
            tree
103
104        // Draw the forest
105        forest.DrawForest("MainCanvas");
106    }
107 }
108 //output
109 Created new TreeType: Oak_Green_Rough
110 Created new TreeType: Maple_Red_Smooth
111 Reused existing TreeType: Oak_Green_Rough
112 Created new TreeType: Pine_Dark Green_Needles
113 Reused existing TreeType: Maple_Red_Smooth
114
115 Drawing entire forest on MainCanvas:
116 Drawing Oak tree at (1,2) with Green color and Rough texture
117 Drawing Maple tree at (5,3) with Red color and Smooth texture
118 Drawing Oak tree at (3,1) with Green color and Rough texture
119 Drawing Pine tree at (7,5) with Dark Green color and Needles texture
120 Drawing Maple tree at (2,4) with Red color and Smooth texture

```

1.10 proxy Pattern - Structural

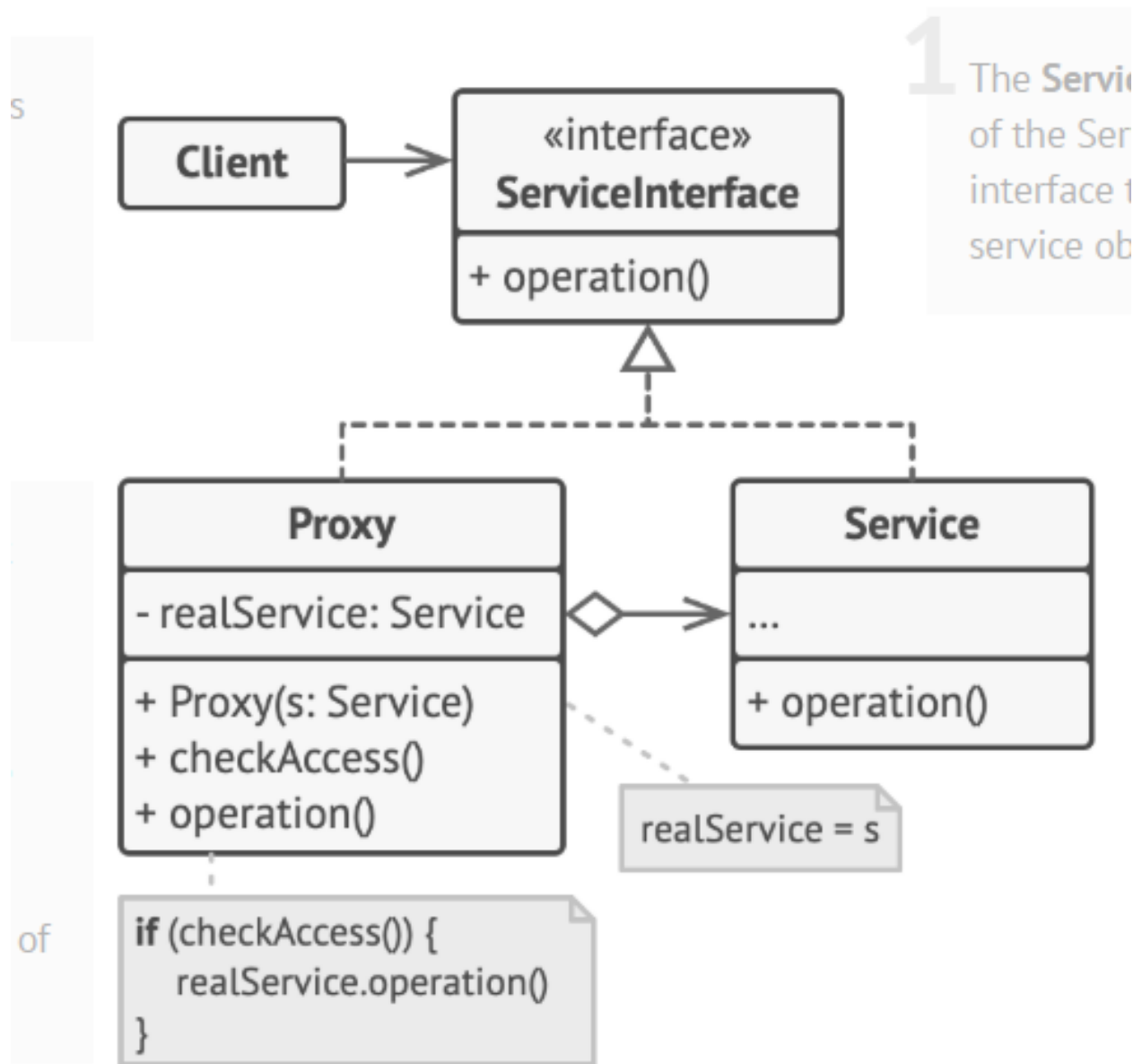


Figure 16: Proxy structure

```
1 using System;
2 using System.Collections.Generic;
3
4 // Subject interface
5 public interface IThirdPartyYouTubeLib
6 {
```



```

7     List<string> ListVideos();
8     string GetVideoInfo(int id);
9     void DownloadVideo(int id);
10 }
11
12 // RealSubject - The actual YouTube service
13 public class ThirdPartyYouTubeClass : IThirdPartyYouTubeLib
14 {
15     public List<string> ListVideos()
16     {
17         // Simulate network request
18         Console.WriteLine("Fetching video list from YouTube...");
19         return new List<string> { "Video1", "Video2", "Video3" };
20     }
21
22     public string GetVideoInfo(int id)
23     {
24         // Simulate network request
25         Console.WriteLine($"Fetching video info for ID {id} from YouTube...");
26         return $"Detailed info for video {id}";
27     }
28
29     public void DownloadVideo(int id)
30     {
31         // Simulate network request
32         Console.WriteLine($"Downloading video {id} from YouTube...");
33     }
34 }
35
36 // Proxy - Cached YouTube service
37 public class CachedYouTubeClass : IThirdPartyYouTubeLib
38 {
39     private readonly IThirdPartyYouTubeLib _service;
40     private List<string> _videoListCache;
41     private Dictionary<int, string> _videoInfoCache = new Dictionary<int, string>
42         >();
43     private bool _listNeedsRefresh = true;
44
45     public CachedYouTubeClass(IThirdPartyYouTubeLib service)
46     {
47         _service = service;
48     }
49
50     public List<string> ListVideos()
51     {
52         if (_listNeedsRefresh || _videoListCache == null)
53         {
54             _videoListCache = _service.ListVideos();
55             _listNeedsRefresh = false;
56             Console.WriteLine("Fetched video list from service and cached it.");
57         }
58         else
59         {
60             Console.WriteLine("Retrieved video list from cache.");
61         }
62         return _videoListCache;
63     }
64
65     public string GetVideoInfo(int id)

```

```

65     {
66         if (!_videoInfoCache.ContainsKey(id))
67         {
68             _videoInfoCache[id] = _service.GetVideoInfo(id);
69             Console.WriteLine($"Fetched video info for ID {id} from service and
70                             cached it.");
71         }
72         else
73         {
74             Console.WriteLine($"Retrieved video info for ID {id} from cache.");
75         }
76         return _videoInfoCache[id];
77     }
78
79     public void DownloadVideo(int id)
80     {
81         // Downloads can't be cached, always forward to service
82         _service.DownloadVideo(id);
83
84         // Invalidate cache since we downloaded a fresh copy
85         if (_videoInfoCache.ContainsKey(id))
86         {
87             _videoInfoCache.Remove(id);
88         }
89         _listNeedsRefresh = true;
90         Console.WriteLine($"Downloaded video {id} and invalidated related cache.");
91     }
92 }
93
94 // YouTube Manager (Client)
95 public class YouTubeManager
96 {
97     private readonly IThirdPartyYouTubeLib _service;
98
99     public YouTubeManager(IThirdPartyYouTubeLib service)
100     {
101         _service = service;
102     }
103
104     public void RenderVideoList()
105     {
106         var videos = _service.ListVideos();
107         Console.WriteLine("\nVideo List:");
108         foreach (var video in videos)
109         {
110             Console.WriteLine($"- {video}");
111         }
112     }
113
114     public void RenderVideoInfo(int id)
115     {
116         var info = _service.GetVideoInfo(id);
117         Console.WriteLine($"Video Info: {info}");
118     }
119
120     public void Download(int id)
121     {
122         _service.DownloadVideo(id);

```

```

122     }
123 }
124
125 class Program
126 {
127     static void Main(string[] args)
128     {
129         // Using the real service directly (no caching)
130         Console.WriteLine("Using real YouTube service:");
131         var realService = new ThirdPartyYouTubeClass();
132         var manager1 = new YouTubeManager(realService);
133
134         manager1.RenderVideoList();
135         manager1.RenderVideoInfo(1);
136         manager1.RenderVideoInfo(1); // Will fetch again
137         manager1.Download(1);
138
139         // Using the cached proxy
140         Console.WriteLine("\nUsing cached YouTube service:");
141         var cachedService = new CachedYouTubeClass(realService);
142         var manager2 = new YouTubeManager(cachedService);
143
144         manager2.RenderVideoList();
145         manager2.RenderVideoInfo(2);
146         manager2.RenderVideoInfo(2); // Will use cache
147         manager2.Download(2);
148         manager2.RenderVideoList(); // Cache was invalidated
149
150         /* Output:
151         Using real YouTube service:
152         Fetching video list from YouTube...
153
154         Video List:
155         - Video1
156         - Video2
157         - Video3
158         Fetching video info for ID 1 from YouTube...
159
160         Video Info: Detailed info for video 1
161         Fetching video info for ID 1 from YouTube...
162
163         Video Info: Detailed info for video 1
164         Downloading video 1 from YouTube...
165
166         Using cached YouTube service:
167         Fetching video list from YouTube...
168         Fetched video list from service and cached it.
169
170         Video List:
171         - Video1
172         - Video2
173         - Video3
174         Fetching video info for ID 2 from YouTube...
175         Fetched video info for ID 2 from service and cached it.
176
177         Video Info: Detailed info for video 2
178         Retrieved video info for ID 2 from cache.
179
180         Video Info: Detailed info for video 2

```

```

181         Downloading video 2 from YouTube...
182         Downloaded video 2 and invalidated related cache.
183         Fetching video list from YouTube...
184         Fetched video list from service and cached it.
185
186         Video List:
187         - Video1
188         - Video2
189         - Video3
190         */
191     }
192 }

```

Let's say you're creating a document management system that deals with large images (like high-resolution photos). Loading these images can be resource-intensive, so you might want to use a proxy to lazily load the image only when necessary, and keep the client from having to load the full image all the time.

```

1  using System;
2
3  // Image Interface
4  public interface IImage
5  {
6      void Display();
7      string GetMetadata();
8  }
9
10 // Real Image (expensive to load)
11 public class HighResolutionImage : IImage
12 {
13     private readonly string _filename;
14     private string _metadata;
15
16     public HighResolutionImage(string filename)
17     {
18         _filename = filename;
19         LoadImageFromDisk();
20     }
21
22     private void LoadImageFromDisk()
23     {
24         // Simulate expensive loading operation
25         Console.WriteLine($"Loading high-resolution image '{_filename}' from disk
26             ... (This takes time and memory)");
27         _metadata = $"Metadata for {_filename}: 6000x4000, 24MB, RGB";
28         // Actual image data would be loaded here
29     }
30
31     public void Display()
32     {
33         Console.WriteLine($"Displaying high-resolution image: {_filename}");
34     }
35
36     public string GetMetadata()
37     {
38         return _metadata;
39     }
40 }
41 // Proxy Image (lazy-loads the real image)

```

```

42 public class ImageProxy : IImage
43 {
44     private readonly string _filename;
45     private HighResolutionImage _realImage;
46     private string _placeholderMetadata;
47
48     public ImageProxy(string filename)
49     {
50         _filename = filename;
51         _placeholderMetadata = $"Placeholder for {_filename}: Not loaded yet";
52     }
53
54     public void Display()
55     {
56         // Lazy initialization: only load when actually needed
57         if (_realImage == null)
58         {
59             _realImage = new HighResolutionImage(_filename);
60         }
61         _realImage.Display();
62     }
63
64     public string GetMetadata()
65     {
66         // Return lightweight metadata without loading the full image
67         if (_realImage == null)
68         {
69             return _placeholderMetadata;
70         }
71         return _realImage.GetMetadata();
72     }
73 }
74
75 // Document Management System
76 class DocumentManager
77 {
78     public void ShowDocumentPreview(IImage image)
79     {
80         // For preview, we only need metadata
81         Console.WriteLine("Document Preview:");
82         Console.WriteLine(image.GetMetadata());
83     }
84
85     public void ViewFullDocument(IImage image)
86     {
87         // For full view, we need to display the image
88         Console.WriteLine("\nFull Document View:");
89         image.Display();
90     }
91 }
92
93 class Program
94 {
95     static void Main(string[] args)
96     {
97         DocumentManager manager = new DocumentManager();
98
99         // Client works with the proxy without knowing it's a proxy
100         IImage imageProxy = new ImageProxy("vacation_photo.jpg");

```

```

101 // Preview shows metadata without loading the full image
102 manager.ShowDocumentPreview(imageProxy);
103
104 // Only when actually viewing the document do we load the full image
105 manager.ViewFullDocument(imageProxy);
106
107 // Subsequent calls use the already-loaded image
108 Console.WriteLine("\nSecond view:");
109 manager.ViewFullDocument(imageProxy);
110 }
111 }
112
113 //output
114 Document Preview:
115 Placeholder for vacation_photo.jpg: Not loaded yet
116
117 Full Document View:
118 Loading high-resolution image 'vacation_photo.jpg' from disk... (This takes time
119 and memory)
120 Displaying high-resolution image: vacation_photo.jpg
121
122 Second view:
123 Displaying high-resolution image: vacation_photo.jpg

```

1.11 Command Pattern - behavioral

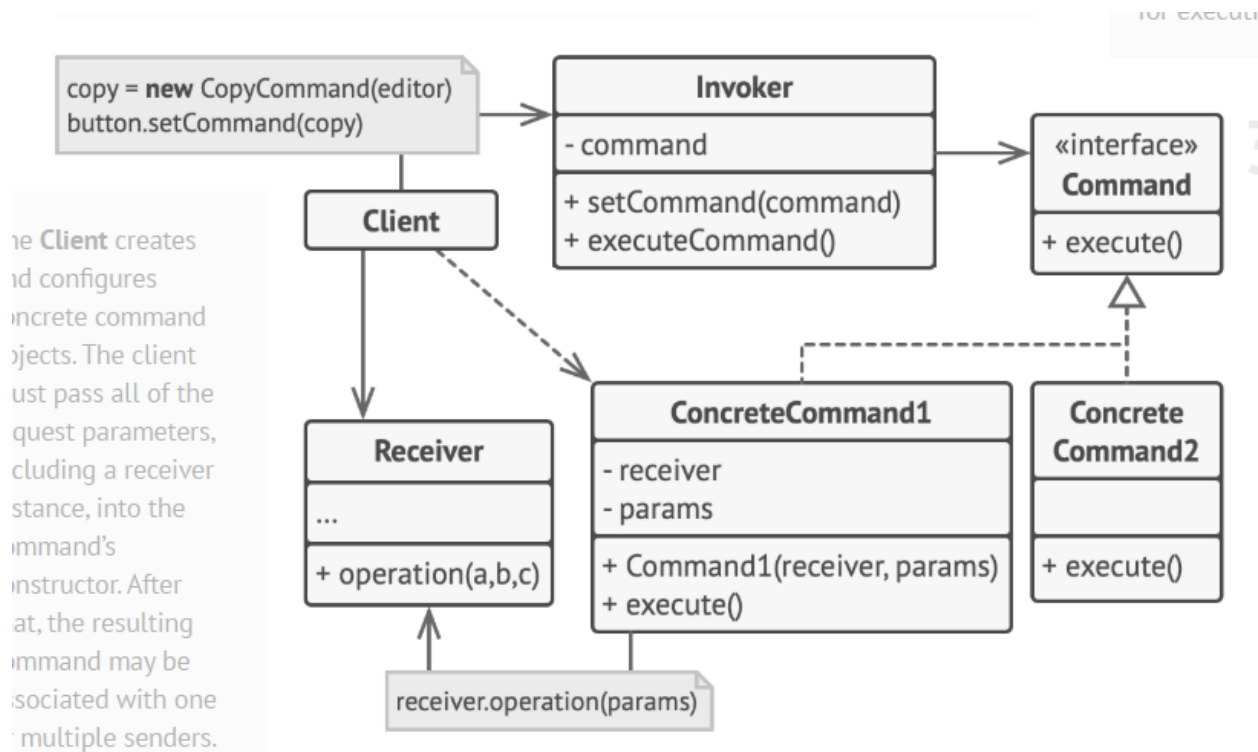


Figure 17: Command structure

```

1      using System;
2      using System.Collections.Generic;
3
4      // The Receiver class that performs the actual operations
5      public class Editor
6      {
7          public string Text { get; set; }
8
9          public string GetSelection()
10         {
11             // In a real implementation, this would return the currently selected text
12             return "selected text";
13         }
14
15         public void DeleteSelection()
16         {
17             // In a real implementation, this would delete the selected text
18             Console.WriteLine("Deleted selected text");
19         }
20
21         public void ReplaceSelection(string text)
22         {
23             // In a real implementation, this would replace the selected text
24             Console.WriteLine($"Replaced selection with: {text}");
25         }
26     }
27
28     // The Invoker class that manages commands
29     public class Application
30     {
31         private List<Editor> editors = new List<Editor>();
32         private Editor activeEditor;
33         private string clipboard;
34         private CommandHistory history = new CommandHistory();
35
36         public Application()
37         {
38             activeEditor = new Editor();
39             editors.Add(activeEditor);
40         }
41
42         public void CreateUI()
43         {
44             // UI creation logic would go here
45         }
46
47         public void ExecuteCommand(Command command)
48         {
49             command.Execute();
50             history.Push(command);
51         }
52
53         public void Undo()
54         {
55             Command command = history.Pop();
56             if (command != null)
57             {
58                 command.Undo();
59             }
60         }
61     }

```

```

60     }
61
62     public string Clipboard
63     {
64         get => clipboard;
65         set => clipboard = value;
66     }
67
68     public Editor ActiveEditor => activeEditor;
69 }
70
71 // The abstract Command class
72 public abstract class Command
73 {
74     protected Application app;
75     protected Editor editor;
76     protected string backup;
77
78     public Command(Application app, Editor editor)
79     {
80         this.app = app;
81         this.editor = editor;
82     }
83
84     public void SaveBackup()
85     {
86         backup = editor.Text;
87     }
88
89     public void Undo()
90     {
91         editor.Text = backup;
92     }
93
94     public abstract void Execute();
95 }
96
97 // Concrete Command classes
98 public class CopyCommand : Command
99 {
100     public CopyCommand(Application app, Editor editor) : base(app, editor) { }
101
102     public override void Execute()
103     {
104         app.Clipboard = editor.GetSelection();
105         Console.WriteLine("Copied to clipboard: " + app.Clipboard);
106     }
107 }
108
109 public class CutCommand : Command
110 {
111     public CutCommand(Application app, Editor editor) : base(app, editor) { }
112
113     public override void Execute()
114     {
115         SaveBackup();
116         app.Clipboard = editor.GetSelection();
117         editor.DeleteSelection();
118         Console.WriteLine("Cut to clipboard: " + app.Clipboard);

```



```

119     }
120 }
121
122 public class PasteCommand : Command
123 {
124     public PasteCommand(Application app, Editor editor) : base(app, editor) { }
125
126     public override void Execute()
127     {
128         SaveBackup();
129         editor.ReplaceSelection(app.Clipboard);
130         Console.WriteLine("Pasted from clipboard: " + app.Clipboard);
131     }
132 }
133
134 public class UndoCommand : Command
135 {
136     public UndoCommand(Application app, Editor editor) : base(app, editor) { }
137
138     public override void Execute()
139     {
140         app.Undo();
141         Console.WriteLine("Undo performed");
142     }
143 }
144
145 // The CommandHistory class manages the command stack
146 public class CommandHistory
147 {
148     private Stack<Command> history = new Stack<Command>();
149
150     public void Push(Command command)
151     {
152         history.Push(command);
153     }
154
155     public Command Pop()
156     {
157         if (history.Count > 0)
158         {
159             return history.Pop();
160         }
161         return null;
162     }
163 }
164
165 // Example usage
166 class Program
167 {
168     static void Main(string[] args)
169     {
170         var app = new Application();
171         var editor = app.ActiveEditor;
172         editor.Text = "Hello, world!";
173
174         // Execute some commands
175         var copyCommand = new CopyCommand(app, editor);
176         app.ExecuteCommand(copyCommand);
177

```

```

178     var cutCommand = new CutCommand(app, editor);
179     app.ExecuteCommand(cutCommand);
180
181     var pasteCommand = new PasteCommand(app, editor);
182     app.ExecuteCommand(pasteCommand);
183
184     // Undo the last command
185     var undoCommand = new UndoCommand(app, editor);
186     app.ExecuteCommand(undoCommand);
187 }
188 }

```

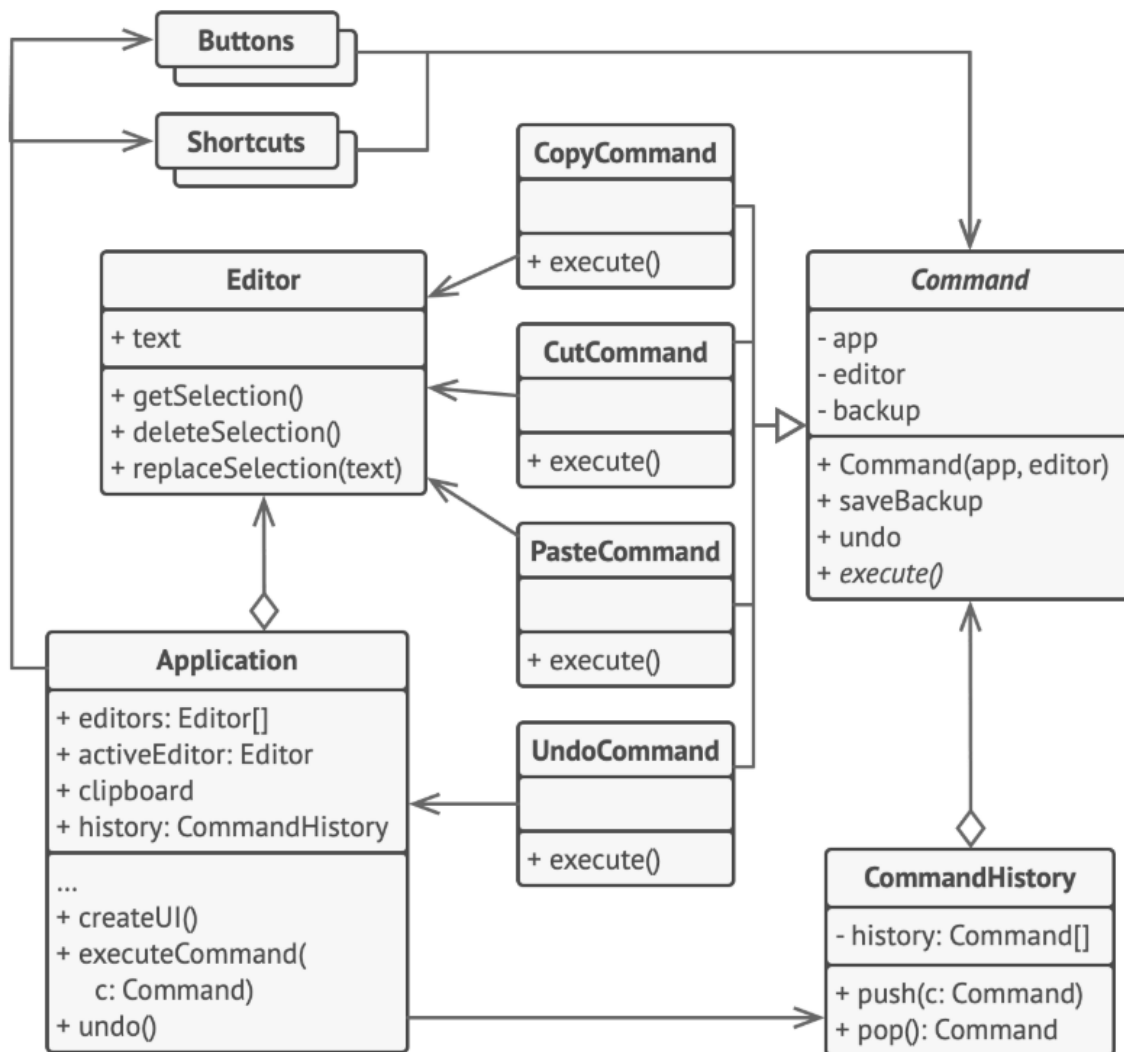


Figure 18: Command

```

1     using System;
2     using System.Collections.Generic;

```

```

3
4 // Receiver - knows how to perform the operations
5 public class Editor
6 {
7     public string Text { get; set; } = "";
8     private string _selectedText = "";
9     private int _selectionStart = 0;
10    private int _selectionLength = 0;
11
12    public string GetSelection()
13    {
14        if (_selectionLength > 0 && _selectionStart + _selectionLength <= Text.
            Length)
15        {
16            _selectedText = Text.Substring(_selectionStart, _selectionLength);
17            return _selectedText;
18        }
19        return "";
20    }
21
22    public void DeleteSelection()
23    {
24        if (_selectionLength > 0 && _selectionStart + _selectionLength <= Text.
            Length)
25        {
26            Text = Text.Remove(_selectionStart, _selectionLength);
27            _selectedText = "";
28            _selectionLength = 0;
29        }
30    }
31
32    public void ReplaceSelection(string text)
33    {
34        DeleteSelection();
35        Text = Text.Insert(_selectionStart, text);
36    }
37
38    public void SetSelection(int start, int length)
39    {
40        _selectionStart = start;
41        _selectionLength = length;
42    }
43
44    public void CreateBackup()
45    {
46        Console.WriteLine($"Editor backup created. Current text: {Text}");
47    }
48
49    public void RestoreFromBackup(string backup)
50    {
51        Text = backup;
52        Console.WriteLine($"Editor restored from backup. Current text: {Text}");
53    }
54 }
55
56 // Command interface
57 public interface ICommand
58 {
59     void Execute();

```

```

60     void Undo();
61 }
62
63 // Base Command class
64 public abstract class Command : ICommand
65 {
66     protected Application App;
67     protected Editor Editor;
68     protected string Backup;
69
70     protected Command(Application app, Editor editor)
71     {
72         App = app;
73         Editor = editor;
74     }
75
76     protected void SaveBackup()
77     {
78         Backup = Editor.Text;
79         Editor.CreateBackup();
80     }
81
82     public void Undo()
83     {
84         Editor.RestoreFromBackup(Backup);
85     }
86
87     public abstract void Execute();
88 }
89
90 // Concrete Commands
91 public class CopyCommand : Command
92 {
93     public CopyCommand(Application app, Editor editor) : base(app, editor) { }
94
95     public override void Execute()
96     {
97         App.Clipboard = Editor.GetSelection();
98         Console.WriteLine($"Copied to clipboard: {App.Clipboard}");
99     }
100 }
101
102 public class CutCommand : Command
103 {
104     public CutCommand(Application app, Editor editor) : base(app, editor) { }
105
106     public override void Execute()
107     {
108         SaveBackup();
109         App.Clipboard = Editor.GetSelection();
110         Editor.DeleteSelection();
111         Console.WriteLine($"Cut to clipboard: {App.Clipboard}");
112     }
113 }
114
115 public class PasteCommand : Command
116 {
117     public PasteCommand(Application app, Editor editor) : base(app, editor) { }
118 }

```

```

119     public override void Execute()
120     {
121         SaveBackup();
122         Editor.ReplaceSelection(App.Clipboard);
123         Console.WriteLine($"Pasted from clipboard: {App.Clipboard}");
124     }
125 }
126
127 public class UndoCommand : Command
128 {
129     public UndoCommand(Application app, Editor editor) : base(app, editor) { }
130
131     public override void Execute()
132     {
133         App.Undo();
134         Console.WriteLine("Undo performed");
135     }
136 }
137
138 // Command history
139 public class CommandHistory
140 {
141     private readonly Stack<Command> _history = new Stack<Command>();
142
143     public void Push(Command command)
144     {
145         _history.Push(command);
146     }
147
148     public Command Pop()
149     {
150         return _history.Count > 0 ? _history.Pop() : null;
151     }
152 }
153
154 // Invoker - executes commands
155 public class Application
156 {
157     public List<Editor> Editors { get; } = new List<Editor>();
158     public Editor ActiveEditor { get; private set; }
159     public string Clipboard { get; set; } = "";
160     public CommandHistory History { get; } = new CommandHistory();
161
162     public Application()
163     {
164         ActiveEditor = new Editor();
165         Editors.Add(ActiveEditor);
166     }
167
168     public void CreateUI()
169     {
170         Console.WriteLine("UI created with buttons for Copy, Cut, Paste, Undo");
171     }
172
173     public void ExecuteCommand(Command command)
174     {
175         command.Execute();
176         if (!(command is UndoCommand))
177         {

```

```

178         History.Push(command);
179     }
180 }
181
182 public void Undo()
183 {
184     var command = History.Pop();
185     command?.Undo();
186 }
187 }
188
189 // Client code
190 class Program
191 {
192     static void Main(string[] args)
193     {
194         var app = new Application();
195         app.CreateUI();
196
197         // Setup initial editor state
198         var editor = app.ActiveEditor;
199         editor.Text = "Hello, Command Pattern!";
200         editor.SetSelection(7, 7); // Select "Command"
201
202         // Execute commands
203         Console.WriteLine("\n--- Executing Commands ---");
204         app.ExecuteCommand(new CopyCommand(app, editor));
205         app.ExecuteCommand(new CutCommand(app, editor));
206
207         editor.SetSelection(0, 0); // Move cursor to start
208         app.ExecuteCommand(new PasteCommand(app, editor));
209
210         Console.WriteLine("\nCurrent editor text: " + editor.Text);
211
212         Console.WriteLine("\n--- Undoing Last Command ---");
213         app.ExecuteCommand(new UndoCommand(app, editor));
214         Console.WriteLine("Editor text after undo: " + editor.Text);
215
216         Console.WriteLine("\n--- Redoing by Executing Paste Again ---");
217         editor.SetSelection(0, 0);
218         app.ExecuteCommand(new PasteCommand(app, editor));
219         Console.WriteLine("Editor text after re-paste: " + editor.Text);
220
221         /* Output:
222         UI created with buttons for Copy, Cut, Paste, Undo
223
224         --- Executing Commands ---
225         Copied to clipboard: Command
226         Editor backup created. Current text: Hello, Command Pattern!
227         Cut to clipboard: Command
228         Editor backup created. Current text: Hello, Pattern!
229         Pasted from clipboard: Command
230
231         Current editor text: CommandHello, Pattern!
232
233         --- Undoing Last Command ---
234         Editor restored from backup. Current text: Hello, Pattern!
235         Undo performed
236         Editor text after undo: Hello, Pattern!

```

```

237     --- Redoing by Executing Paste Again ---
238     Editor backup created. Current text: Hello,  Pattern!
239     Pasted from clipboard: Command
240     Editor text after re-paste: CommandHello,  Pattern!
241     */
242 }
243 }
244

```

```

1
2 In this example, we'll simulate a remote control that can turn a light on and off.
  The structure includes:
3
4 Command Interface: Declares the Execute() method.
5
6 Concrete Commands: Implement the command interface (e.g., TurnOnCommand,
  TurnOffCommand).
7
8 Receiver: The object that performs the actual work (e.g., Light).
9
10 Invoker: The object that calls the command (e.g., RemoteControl).
11
12 Client: Configures the command objects and assigns them to the invoker.
13 \begin{lstlisting}
14     using System;
15
16 namespace CommandPatternExample
17 {
18     // Command Interface
19     public interface ICommand
20     {
21         void Execute();
22     }
23
24     // Receiver
25     public class Light
26     {
27         public void TurnOn()
28         {
29             Console.WriteLine("The light is ON.");
30         }
31
32         public void TurnOff()
33         {
34             Console.WriteLine("The light is OFF.");
35         }
36     }
37
38     // Concrete Command: Turn On the light
39     public class TurnOnCommand : ICommand
40     {
41         private readonly Light _light;
42
43         public TurnOnCommand(Light light)
44         {
45             _light = light;
46         }
47
48         public void Execute()

```

```

49     {
50         _light.TurnOn();
51     }
52 }
53
54 // Concrete Command: Turn Off the light
55 public class TurnOffCommand : ICommand
56 {
57     private readonly Light _light;
58
59     public TurnOffCommand(Light light)
60     {
61         _light = light;
62     }
63
64     public void Execute()
65     {
66         _light.TurnOff();
67     }
68 }
69
70 // Invoker: RemoteControl that can store and execute commands
71 public class RemoteControl
72 {
73     private ICommand _command;
74
75     // Set a command on the remote control
76     public void SetCommand(ICommand command)
77     {
78         _command = command;
79     }
80
81     // Simulate pressing a button on the remote control
82     public void PressButton()
83     {
84         if (_command != null)
85         {
86             _command.Execute();
87         }
88         else
89         {
90             Console.WriteLine("No command is set.");
91         }
92     }
93 }
94
95 // Client Code
96 class Program
97 {
98     static void Main(string[] args)
99     {
100         // Create the receiver
101         Light livingRoomLight = new Light();
102
103         // Create concrete commands with the receiver
104         ICommand turnOn = new TurnOnCommand(livingRoomLight);
105         ICommand turnOff = new TurnOffCommand(livingRoomLight);
106
107         // Create the invoker

```



```

108 RemoteControl remote = new RemoteControl();
109
110 // Set and execute the Turn On command
111 remote.SetCommand(turnOn);
112 remote.PressButton(); // Output: The light is ON.
113
114 // Set and execute the Turn Off command
115 remote.SetCommand(turnOff);
116 remote.PressButton(); // Output: The light is OFF.
117 }
118 }
119 }
120 //output

```

1.12 Prototype Pattern - creational

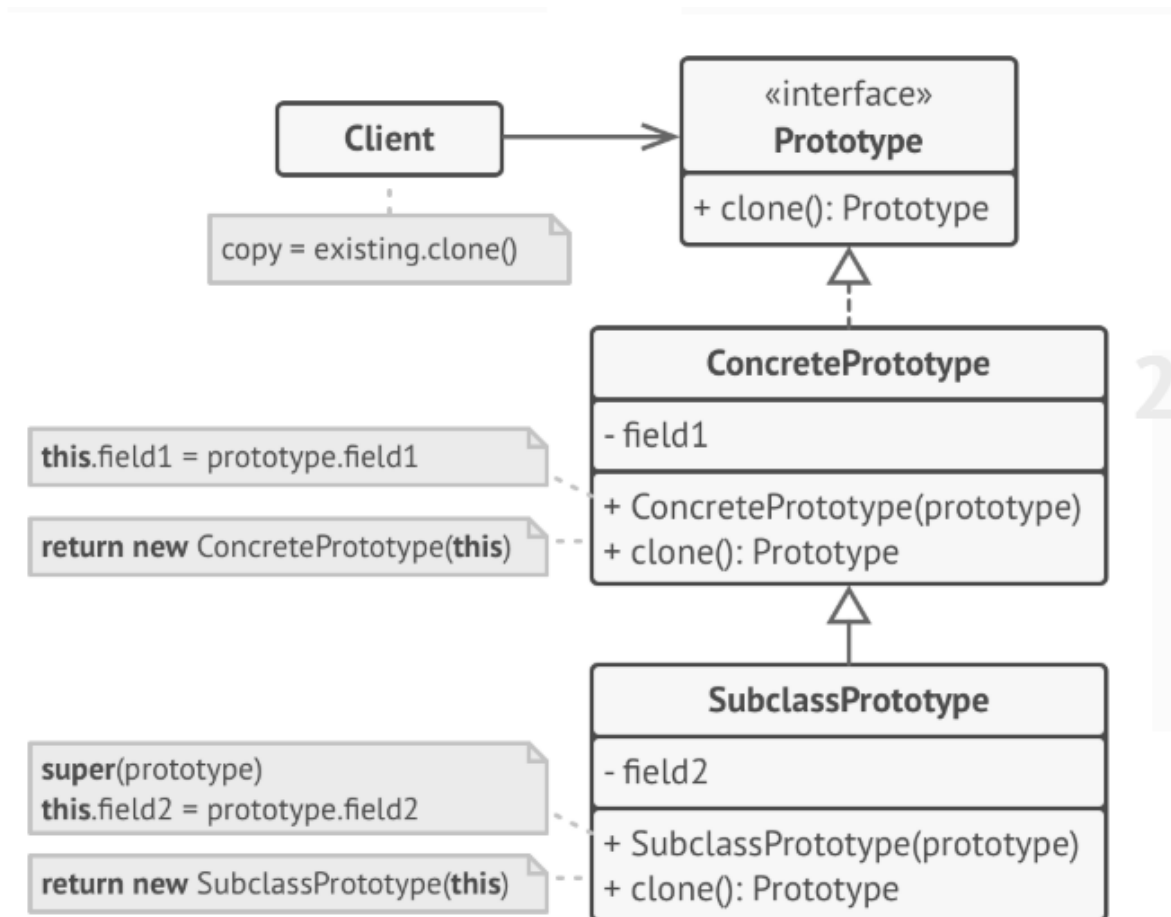


Figure 19: prototype structure

Imagine you're building a game where you have different types of characters (e.g., Warrior, Mage) that share similar properties. You want to create new characters quickly by cloning a prototype, instead of building them from scratch every time.

```

1      using System;
2
3      // Prototype interface
4      public interface ICharacter
5      {
6          ICharacter Clone();
7          void Display();
8      }
9
10     // Concrete Prototype: Warrior
11     public class Warrior : ICharacter
12     {
13         public string Weapon { get; set; }
14         public int Strength { get; set; }
15
16         public Warrior(string weapon, int strength)
17         {
18             Weapon = weapon;
19             Strength = strength;
20         }
21
22         public ICharacter Clone()
23         {
24             // Shallow copy (can use MemberwiseClone if more complex)
25             return new Warrior(this.Weapon, this.Strength);
26         }
27
28         public void Display()
29         {
30             Console.WriteLine($"Warrior with {Weapon}, Strength: {Strength}");
31         }
32     }
33
34     // Concrete Prototype: Mage
35     public class Mage : ICharacter
36     {
37         public string Spell { get; set; }
38         public int Mana { get; set; }
39
40         public Mage(string spell, int mana)
41         {
42             Spell = spell;
43             Mana = mana;
44         }
45
46         public ICharacter Clone()
47         {
48             return new Mage(this.Spell, this.Mana);
49         }
50
51         public void Display()
52         {
53             Console.WriteLine($"Mage casting {Spell}, Mana: {Mana}");
54         }
55     }
56
57     // Client code
58     class Program

```

```

59 {
60     static void Main(string[] args)
61     {
62         // Original prototypes
63         Warrior baseWarrior = new Warrior("Sword", 100);
64         Mage baseMage = new Mage("Fireball", 200);
65
66         // Clone characters
67         ICharacter warriorClone1 = baseWarrior.Clone();
68         ICharacter mageClone1 = baseMage.Clone();
69
70         // Modify clones (if needed)
71         ((Warrior)warriorClone1).Weapon = "Axe";
72         ((Mage)mageClone1).Mana = 150;
73
74         // Display all characters
75         baseWarrior.Display();
76         warriorClone1.Display();
77         baseMage.Display();
78         mageClone1.Display();
79     }
80 }
81
82 //output
83 Warrior with Sword, Strength: 100
84 Warrior with Axe, Strength: 100
85 Mage casting Fireball, Mana: 200
86 Mage casting Fireball, Mana: 150

```

We'll create a character that has an Equipment object. Then, we'll clone the character using shallow and deep copies to see the difference.

```

1     using System;
2
3     // Equipment class (reference type)
4     public class Equipment
5     {
6         public string Weapon { get; set; }
7         public string Armor { get; set; }
8     }
9
10    // Prototype interface
11    public interface ICharacter
12    {
13        ICharacter CloneShallow();
14        ICharacter CloneDeep();
15        void Display();
16    }
17
18    // Character class
19    public class GameCharacter : ICharacter
20    {
21        public string Name { get; set; }
22        public Equipment Gear { get; set; }
23
24        public GameCharacter(string name, Equipment gear)
25        {
26            Name = name;
27            Gear = gear;

```

```

28     }
29
30     // Shallow copy
31     public ICharacter CloneShallow()
32     {
33         return (ICharacter)this.MemberwiseClone();
34     }
35
36     // Deep copy
37     public ICharacter CloneDeep()
38     {
39         // Create new equipment instance manually
40         Equipment newGear = new Equipment
41         {
42             Weapon = this.Gear.Weapon,
43             Armor = this.Gear.Armor
44         };
45
46         return new GameCharacter(this.Name, newGear);
47     }
48
49     public void Display()
50     {
51         Console.WriteLine($"{Name} with Weapon: {Gear.Weapon}, Armor: {Gear.Armor}");
52     }
53 }
54
55 // Test
56 class Program
57 {
58     static void Main(string[] args)
59     {
60         var original = new GameCharacter("Knight", new Equipment { Weapon = "Sword", Armor = "Plate" });
61
62         var shallowClone = original.CloneShallow();
63         var deepClone = original.CloneDeep();
64
65         // Modify the original's equipment
66         original.Gear.Weapon = "Axe";
67
68         Console.WriteLine("After changing original's weapon:");
69
70         original.Display();           // Knight with Axe
71         shallowClone.Display();       // Weapon also changed (shared reference)
72         deepClone.Display();          // Weapon unchanged (independent)
73     }
74 }
75
76 //output
77 After changing original's weapon:
78 Knight with Weapon: Axe, Armor: Plate
79 Knight with Weapon: Axe, Armor: Plate // shallow copy - shared reference
80 Knight with Weapon: Sword, Armor: Plate // deep copy - independent

```

Imagine a video game where:

You have different types of characters (e.g., Warrior, Mage).

Each character has a lot of configuration (weapons, skills, stats).

You want to clone a pre-configured character instead of creating it from scratch every time (especially

for AI enemies, presets, NPCs, etc.).

Instead of building a character from scratch each time (expensive), we clone a prototype and modify it if needed.

```
1      using System;
2      using System.Collections.Generic;
3
4      #region Prototype Interface
5      interface IPrototype<T>
6      {
7          T Clone();
8      }
9      #endregion
10
11     #region Prototype Class
12     class GameCharacter : IPrototype<GameCharacter>
13     {
14         public string Name { get; set; }
15         public string Class { get; set; }
16         public int Health { get; set; }
17         public List<string> Skills { get; set; }
18         public string Weapon { get; set; }
19
20         public GameCharacter(string name, string @class, int health, List<string>
            skills, string weapon)
21         {
22             Name = name;
23             Class = @class;
24             Health = health;
25             Skills = new List<string>(skills);
26             Weapon = weapon;
27         }
28
29         public GameCharacter Clone()
30         {
31             // Deep copy to avoid shared references
32             return new GameCharacter(Name, Class, Health, new List<string>(Skills),
                Weapon);
33         }
34
35         public void ShowInfo()
36         {
37             Console.WriteLine($"[Character] {Name} - {Class}, HP: {Health}, Weapon: {
                Weapon}, Skills: {string.Join(", ", Skills)}");
38         }
39     }
40     #endregion
41
42     #region Usage
43     class Program
44     {
45         static void Main()
46         {
47             // Create a prototype for a Warrior
48             var warriorPrototype = new GameCharacter(
49                 name: "PrototypeWarrior",
50                 @class: "Warrior",
51                 health: 150,
52                 skills: new List<string> { "Slash", "Shield Block", "Battle Cry" },
```

```

53         weapon: "Sword"
54     );
55
56     // Clone a new enemy from prototype
57     var enemy1 = warriorPrototype.Clone();
58     enemy1.Name = "EnemyWarrior1";
59     enemy1.Weapon = "Axe"; // Customize clone
60
61     // Clone another one
62     var enemy2 = warriorPrototype.Clone();
63     enemy2.Name = "EnemyWarrior2";
64     enemy2.Skills.Add("Berserk");
65
66     // Display characters
67     warriorPrototype.ShowInfo();
68     enemy1.ShowInfo();
69     enemy2.ShowInfo();
70 }
71 }
72 #endregion
73
74 //output
75 [Character] PrototypeWarrior - Warrior, HP: 150, Weapon: Sword, Skills: Slash,
    Shield Block, Battle Cry
76 [Character] EnemyWarrior1 - Warrior, HP: 150, Weapon: Axe, Skills: Slash, Shield
    Block, Battle Cry
77 [Character] EnemyWarrior2 - Warrior, HP: 150, Weapon: Sword, Skills: Slash, Shield
    Block, Battle Cry, Berserk

```

```

1  using System;
2
3  #region Prototype Interface
4  interface ICharacterPrototype
5  {
6      Character Clone();
7  }
8  #endregion
9
10 #region Nested Object (Weapon)
11 class Weapon
12 {
13     public string Name { get; set; }
14     public int Damage { get; set; }
15
16     public Weapon Clone()
17     {
18         return new Weapon { Name = this.Name, Damage = this.Damage };
19     }
20 }
21 #endregion
22
23 #region Concrete Prototype
24 class Character : ICharacterPrototype
25 {
26     public string Name { get; set; }
27     public string ClassType { get; set; }
28     public int Level { get; set; }
29     public Weapon Weapon { get; set; }
30 }

```

```

31 public Character Clone()
32 {
33     return new Character
34     {
35         Name = this.Name,
36         ClassType = this.ClassType,
37         Level = this.Level,
38         Weapon = this.Weapon.Clone() // Deep copy of nested object
39     };
40 }
41
42 public void Display()
43 {
44     Console.WriteLine($"Name: {Name}, Class: {ClassType}, Level: {Level},
45                        Weapon: {Weapon.Name} ({Weapon.Damage} dmg)");
46 }
47 #endregion
48
49 #region Program
50 class Program
51 {
52     static void Main()
53     {
54         // Original prototype character
55         Character warriorPrototype = new Character
56         {
57             Name = "PrototypeWarrior",
58             ClassType = "Warrior",
59             Level = 1,
60             Weapon = new Weapon { Name = "Iron Sword", Damage = 10 }
61         };
62
63         Console.WriteLine("== Original Prototype ==");
64         warriorPrototype.Display();
65
66         // Clone and customize
67         Character hero1 = warriorPrototype.Clone();
68         hero1.Name = "Aragon";
69         hero1.Level = 5;
70         hero1.Weapon.Name = "Flame Sword";
71         hero1.Weapon.Damage = 25;
72
73         Character hero2 = warriorPrototype.Clone();
74         hero2.Name = "Brienne";
75         hero2.Level = 3;
76
77         Console.WriteLine("\n== Cloned Characters ==");
78         hero1.Display();
79         hero2.Display();
80
81         Console.WriteLine("\n== Check Prototype After Cloning ==");
82         warriorPrototype.Display(); // Should be unchanged
83     }
84 }
85 #endregion
86 //output
87 == Original Prototype ==
88 Name: PrototypeWarrior, Class: Warrior, Level: 1, Weapon: Iron Sword (10 dmg)

```

```

89
90 == Cloned Characters ==
91 Name: Aragon, Class: Warrior, Level: 5, Weapon: Flame Sword (25 dmg)
92 Name: Brienne, Class: Warrior, Level: 3, Weapon: Iron Sword (10 dmg)
93
94 == Check Prototype After Cloning ==
95 Name: PrototypeWarrior, Class: Warrior, Level: 1, Weapon: Iron Sword (10 dmg)

```

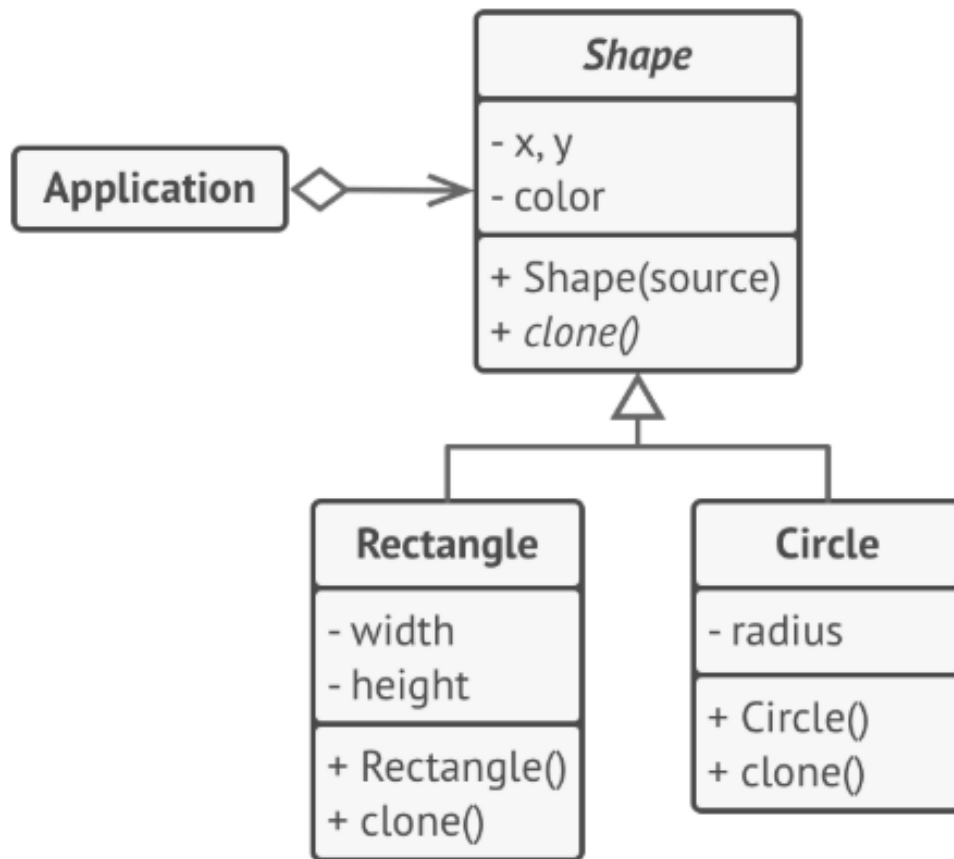


Figure 20: prototype

```

1      using System;
2
3      // Base prototype class
4      public abstract class Shape : ICloneable
5      {
6          public int X { get; set; }
7          public int Y { get; set; }
8          public string Color { get; set; }
9      }

```



```

10 // Regular constructor
11 public Shape()
12 {
13 }
14
15 // Copy constructor
16 public Shape(Shape source)
17 {
18     if (source != null)
19     {
20         X = source.X;
21         Y = source.Y;
22         Color = source.Color;
23     }
24 }
25
26 // The clone method that all concrete prototypes must implement
27 public abstract Shape Clone();
28
29 // Explicit interface implementation
30 object ICloneable.Clone()
31 {
32     return Clone();
33 }
34
35 public override string ToString()
36 {
37     return $"{GetType().Name} at ({X},{Y}), Color: {Color}";
38 }
39 }
40
41 // Concrete prototype - Rectangle
42 public class Rectangle : Shape
43 {
44     public int Width { get; set; }
45     public int Height { get; set; }
46
47     public Rectangle()
48     {
49     }
50
51     // Copy constructor
52     public Rectangle(Rectangle source) : base(source)
53     {
54         if (source != null)
55         {
56             Width = source.Width;
57             Height = source.Height;
58         }
59     }
60
61     public override Shape Clone()
62     {
63         return new Rectangle(this);
64     }
65
66     public override string ToString()
67     {
68         return base.ToString() + $", Width: {Width}, Height: {Height}";

```

```

69     }
70 }
71
72 // Concrete prototype - Circle
73 public class Circle : Shape
74 {
75     public int Radius { get; set; }
76
77     public Circle()
78     {
79     }
80
81     // Copy constructor
82     public Circle(Circle source) : base(source)
83     {
84         if (source != null)
85         {
86             Radius = source.Radius;
87         }
88     }
89
90     public override Shape Clone()
91     {
92         return new Circle(this);
93     }
94
95     public override string ToString()
96     {
97         return base.ToString() + $", Radius: {Radius}";
98     }
99 }
100
101 // Client code
102 class Program
103 {
104     static void Main(string[] args)
105     {
106         Console.WriteLine("Original Shapes:");
107
108         // Create original shapes
109         var originalRectangle = new Rectangle
110         {
111             X = 10,
112             Y = 20,
113             Color = "Red",
114             Width = 100,
115             Height = 50
116         };
117         Console.WriteLine(originalRectangle);
118
119         var originalCircle = new Circle
120         {
121             X = 30,
122             Y = 40,
123             Color = "Blue",
124             Radius = 25
125         };
126         Console.WriteLine(originalCircle);
127

```

```

128     // Clone the shapes
129     Console.WriteLine("\nCloned Shapes:");
130     var clonedRectangle = (Rectangle)originalRectangle.Clone();
131     clonedRectangle.X += 5; // Modify position to demonstrate it's a different
        object
132     Console.WriteLine(clonedRectangle);
133
134     var clonedCircle = (Circle)originalCircle.Clone();
135     clonedCircle.Color = "Green"; // Modify color to demonstrate it's a
        different object
136     Console.WriteLine(clonedCircle);
137
138     // Verify the originals weren't modified
139     Console.WriteLine("\nOriginal Shapes After Cloning:");
140     Console.WriteLine(originalRectangle);
141     Console.WriteLine(originalCircle);
142
143     /* Output:
144     Original Shapes:
145     Rectangle at (10,20), Color: Red, Width: 100, Height: 50
146     Circle at (30,40), Color: Blue, Radius: 25
147
148     Cloned Shapes:
149     Rectangle at (15,20), Color: Red, Width: 100, Height: 50
150     Circle at (30,40), Color: Green, Radius: 25
151
152     Original Shapes After Cloning:
153     Rectangle at (10,20), Color: Red, Width: 100, Height: 50
154     Circle at (30,40), Color: Blue, Radius: 25
155     */
156 }
157 }

```

1.13 Observer Pattern - behavioral

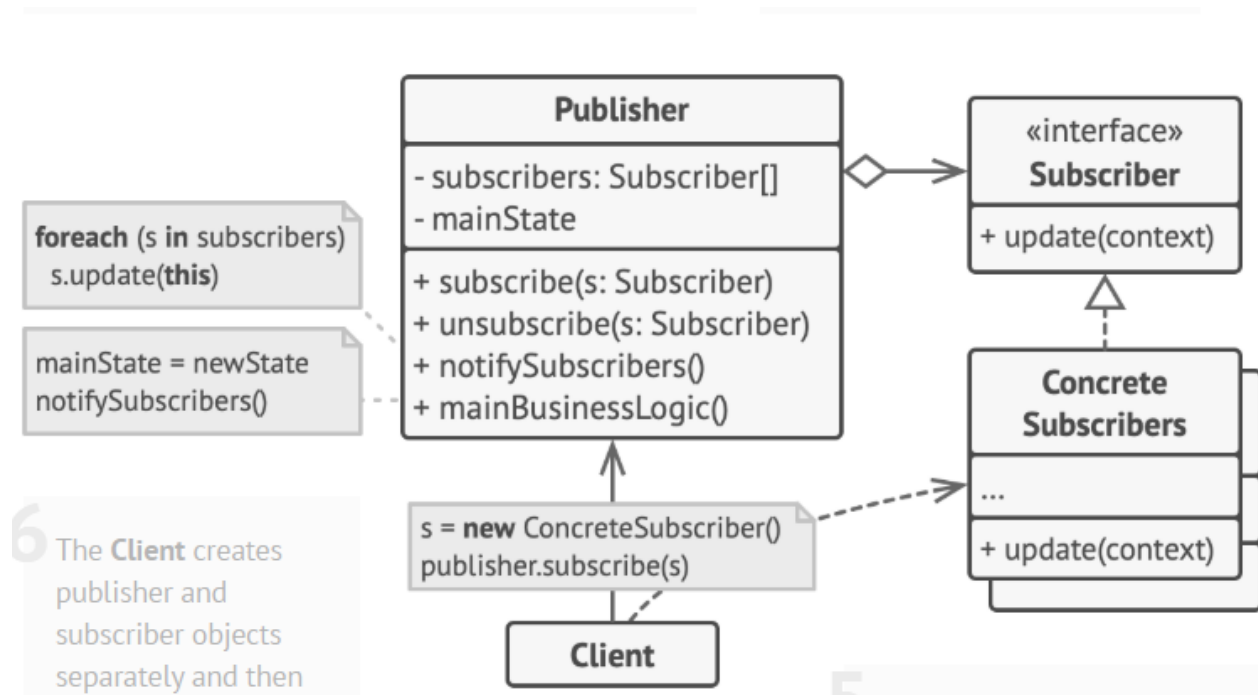


Figure 21: Observer structure

We have a WeatherStation that measures temperature, and multiple displays (like PhoneDisplay and WindowDisplay) that want to be notified when the temperature changes.

```

1  using System;
2  using System.Collections.Generic;
3
4  // The Observer interface
5  public interface IObserver
6  {
7      void Update(float temperature);
8  }
9
10 // The Subject interface
11 public interface ISubject
12 {
13     void RegisterObserver(IObserver observer);
14     void RemoveObserver(IObserver observer);
15     void NotifyObservers();
16 }
17
18 // Concrete Subject
19 public class WeatherStation : ISubject
20 {
21     private List<IObserver> observers = new List<IObserver>();
22     private float temperature;
23
24     public void RegisterObserver(IObserver observer)
25     {

```

```

26     observers.Add(observer);
27 }
28
29 public void RemoveObserver(IObserver observer)
30 {
31     observers.Remove(observer);
32 }
33
34 public void NotifyObservers()
35 {
36     foreach (var observer in observers)
37     {
38         observer.Update(temperature);
39     }
40 }
41
42 public void SetTemperature(float temp)
43 {
44     Console.WriteLine($"\\nWeatherStation: new temperature = {temp} C ");
45     this.temperature = temp;
46     NotifyObservers();
47 }
48 }
49
50 // Concrete Observer 1
51 public class PhoneDisplay : IObserver
52 {
53     public void Update(float temperature)
54     {
55         Console.WriteLine($"PhoneDisplay: temperature updated to {temperature} C ");
56     }
57 }
58
59 // Concrete Observer 2
60 public class WindowDisplay : IObserver
61 {
62     public void Update(float temperature)
63     {
64         Console.WriteLine($"WindowDisplay: temperature updated to {temperature} C ");
65     }
66 }
67
68 // Test the observer pattern
69 class Program
70 {
71     static void Main(string[] args)
72     {
73         WeatherStation station = new WeatherStation();
74
75         PhoneDisplay phone = new PhoneDisplay();
76         WindowDisplay window = new WindowDisplay();
77
78         station.RegisterObserver(phone);
79         station.RegisterObserver(window);
80
81         station.SetTemperature(22.5f);
82         station.SetTemperature(25.0f);

```

```

83         station.RemoveObserver(window);
84         station.SetTemperature(26.5f);
85     }
86 }
87
88 //output
89 WeatherStation: new temperature = 22.5 C
90 PhoneDisplay: temperature updated to 22.5 C
91 WindowDisplay: temperature updated to 22.5 C
92
93 WeatherStation: new temperature = 25 C
94 PhoneDisplay: temperature updated to 25 C
95 WindowDisplay: temperature updated to 25 C
96
97 WeatherStation: new temperature = 26.5 C
98 PhoneDisplay: temperature updated to 26.5 C

```

We have a Stock Market (Subject) where:
 Multiple investors (Observers) subscribe to updates.
 Some investors only care about specific stocks.
 When a stock price changes, only the interested investors should be notified.
 We'll also include:
 Multiple stock types (e.g., AAPL, GOOG, etc.)
 Dynamic subscription/unsubscription
 Custom reactions to stock changes

```

1     using System;
2     using System.Collections.Generic;
3
4     #region Observer Interface
5     interface IInvestor
6     {
7         void Update(string stockSymbol, decimal newPrice);
8     }
9     #endregion
10
11    #region Subject
12    class StockMarket
13    {
14        // Stock Symbol -> Price
15        private Dictionary<string, decimal> _stockPrices = new();
16
17        // Stock Symbol -> List of Observers
18        private Dictionary<string, List<IInvestor>> _observers = new();
19
20        public void AddStock(string symbol, decimal initialPrice)
21        {
22            _stockPrices[symbol] = initialPrice;
23            _observers[symbol] = new List<IInvestor>();
24        }
25
26        public void Subscribe(string symbol, IInvestor investor)
27        {
28            if (!_observers.ContainsKey(symbol))
29                _observers[symbol].Add(investor);
30        }
31
32        public void Unsubscribe(string symbol, IInvestor investor)
33        {

```

```

34         if (_observers.ContainsKey(symbol))
35             _observers[symbol].Remove(investor);
36     }
37
38     public void SetPrice(string symbol, decimal newPrice)
39     {
40         if (!_stockPrices.ContainsKey(symbol)) return;
41
42         _stockPrices[symbol] = newPrice;
43         Notify(symbol, newPrice);
44     }
45
46     private void Notify(string symbol, decimal newPrice)
47     {
48         foreach (var observer in _observers[symbol])
49         {
50             observer.Update(symbol, newPrice);
51         }
52     }
53 }
54 #endregion
55
56 #region Concrete Observers
57 class RegularInvestor : IInvestor
58 {
59     public string Name { get; }
60
61     public RegularInvestor(string name)
62     {
63         Name = name;
64     }
65
66     public void Update(string stockSymbol, decimal newPrice)
67     {
68         Console.WriteLine($"{Name} notified: {stockSymbol} is now {newPrice:C}");
69     }
70 }
71
72 class AlertingInvestor : IInvestor
73 {
74     private readonly string _name;
75     private readonly decimal _threshold;
76
77     public AlertingInvestor(string name, decimal threshold)
78     {
79         _name = name;
80         _threshold = threshold;
81     }
82
83     public void Update(string stockSymbol, decimal newPrice)
84     {
85         if (newPrice > _threshold)
86         {
87             Console.WriteLine($"          ALERT for {_name}: {stockSymbol} crossed {
88                 _threshold:C} (Now: {newPrice:C})");
89         }
90     }
91 }
92 #endregion

```

```

92
93 #region Demo
94 class Program
95 {
96     static void Main()
97     {
98         var market = new StockMarket();
99
100         market.AddStock("AAPL", 150.00m);
101         market.AddStock("GOOG", 2800.00m);
102
103         var john = new RegularInvestor("John");
104         var emma = new AlertingInvestor("Emma", 155.00m);
105         var li = new AlertingInvestor("Li", 3000.00m);
106
107         market.Subscribe("AAPL", john);
108         market.Subscribe("AAPL", emma);
109         market.Subscribe("GOOG", li);
110
111         Console.WriteLine("== Stock Price Updates ==");
112         market.SetPrice("AAPL", 152.00m); // Notify John and Emma
113         market.SetPrice("AAPL", 156.00m); // Emma gets alert
114         market.SetPrice("GOOG", 3100.00m); // Li gets alert
115     }
116 }
117 #endregion
118
119 //output
120 == Stock Price Updates ==
121 John notified: AAPL is now $152.00
122 Emma notified: AAPL is now $152.00
123 John notified: AAPL is now $156.00
124 Emma notified: AAPL is now $156.00
125     ALERT for Emma: AAPL crossed $155.00 (Now: $156.00)
126 Li notified: GOOG is now $3,100.00
127     ALERT for Li: GOOG crossed $3,000.00 (Now: $3,100.00)

```

We have a Weather Station (Subject) that gathers temperature, humidity, and pressure data. There are multiple displays (Observers), like:

- Current Conditions Display

- Statistics Display

- Forecast Display

When the weather station updates its measurements, all displays should automatically update.

```

1     using System;
2     using System.Collections.Generic;
3
4     #region Observer Interface
5     interface IWeatherObserver
6     {
7         void Update(float temperature, float humidity, float pressure);
8     }
9     #endregion
10
11     #region Subject
12     class WeatherStation
13     {
14         private List<IWeatherObserver> observers = new();
15         private float temperature, humidity, pressure;

```



```

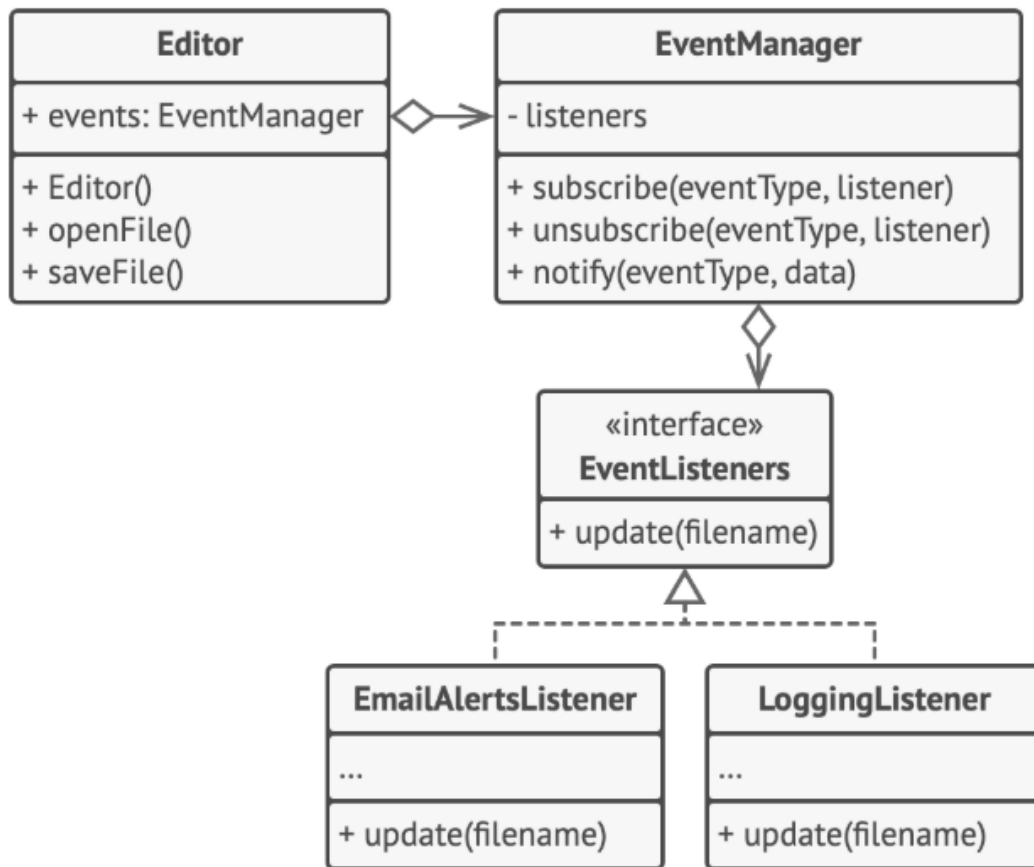
16     public void Register(IWeatherObserver observer)
17     {
18         observers.Add(observer);
19     }
20
21     public void Unregister(IWeatherObserver observer)
22     {
23         observers.Remove(observer);
24     }
25
26     public void SetMeasurements(float temp, float hum, float pres)
27     {
28         temperature = temp;
29         humidity = hum;
30         pressure = pres;
31         NotifyObservers();
32     }
33
34     private void NotifyObservers()
35     {
36         foreach (var observer in observers)
37         {
38             observer.Update(temperature, humidity, pressure);
39         }
40     }
41 }
42 #endregion
43
44 #region Concrete Observers
45
46 class CurrentConditionsDisplay : IWeatherObserver
47 {
48     public void Update(float temperature, float humidity, float pressure)
49     {
50         Console.WriteLine($"[CurrentConditions] Temp: {temperature} C , Humidity:
51             {humidity}%, Pressure: {pressure}hPa");
52     }
53 }
54
55 class StatisticsDisplay : IWeatherObserver
56 {
57     private List<float> tempReadings = new();
58
59     public void Update(float temperature, float humidity, float pressure)
60     {
61         tempReadings.Add(temperature);
62         float avg = (float)Math.Round(tempReadings.Average(), 2);
63         Console.WriteLine($"[Statistics] Avg Temp: {avg} C (Last {tempReadings.
64             Count} readings)");
65     }
66 }
67
68 class ForecastDisplay : IWeatherObserver
69 {
70     private float lastPressure = 1013;
71
72     public void Update(float temperature, float humidity, float pressure)
73     {

```

```

73         string forecast = pressure > lastPressure ? "Improving weather" : "
           Possible rain";
74         Console.WriteLine($"[Forecast] {forecast}");
75         lastPressure = pressure;
76     }
77 }
78
79 #endregion
80
81 #region Program
82 class Program
83 {
84     static void Main()
85     {
86         var weatherStation = new WeatherStation();
87
88         var currentDisplay = new CurrentConditionsDisplay();
89         var statsDisplay = new StatisticsDisplay();
90         var forecastDisplay = new ForecastDisplay();
91
92         weatherStation.Register(currentDisplay);
93         weatherStation.Register(statsDisplay);
94         weatherStation.Register(forecastDisplay);
95
96         Console.WriteLine("== Weather Update 1 ==");
97         weatherStation.SetMeasurements(25.5f, 65f, 1012f);
98
99         Console.WriteLine("\n== Weather Update 2 ==");
100        weatherStation.SetMeasurements(26.8f, 70f, 1014f);
101
102        Console.WriteLine("\n== Unregister ForecastDisplay and Update ==");
103        weatherStation.Unregister(forecastDisplay);
104        weatherStation.SetMeasurements(28f, 72f, 1011f);
105    }
106 }
107 #endregion
108
109 //output
110 == Weather Update 1 ==
111 [CurrentConditions] Temp: 25.5 C , Humidity: 65%, Pressure: 1012hPa
112 [Statistics] Avg Temp: 25.5 C (Last 1 readings)
113 [Forecast] Possible rain
114
115 == Weather Update 2 ==
116 [CurrentConditions] Temp: 26.8 C , Humidity: 70%, Pressure: 1014hPa
117 [Statistics] Avg Temp: 26.15 C (Last 2 readings)
118 [Forecast] Improving weather
119
120 == Unregister ForecastDisplay and Update ==
121 [CurrentConditions] Temp: 28 C , Humidity: 72%, Pressure: 1011hPa
122 [Statistics] Avg Temp: 26.77 C (Last 3 readings)

```



Notifying objects about events that happen to other objects

Figure 22: Observer

```

1  using System;
2  using System.Collections.Generic;
3
4  // Observer interface
5  public interface IEventListener
6  {
7      void Update(string filename);
8  }
9
10 // Subject (Observable)
11 public class EventManager
12 {
13     private readonly Dictionary<string, List<IEventListener>> _listeners = new
        Dictionary<string, List<IEventListener>>();
14
15     public void Subscribe(string eventType, IEventListener listener)
16     {
17         if (!_listeners.ContainsKey(eventType))
18         {

```

```

19         _listeners[eventType] = new List<IEventListener>();
20     }
21     _listeners[eventType].Add(listener);
22     Console.WriteLine($"Subscribed {listener.GetType().Name} to {eventType}
23         events");
24 }
25 public void Unsubscribe(string eventType, IEventListener listener)
26 {
27     if (_listeners.ContainsKey(eventType))
28     {
29         _listeners[eventType].Remove(listener);
30         Console.WriteLine($"Unsubscribed {listener.GetType().Name} from {
31             eventType} events");
32     }
33 }
34 public void Notify(string eventType, string filename)
35 {
36     if (_listeners.ContainsKey(eventType))
37     {
38         Console.WriteLine($"\\nNotifying all {eventType} subscribers...");
39         foreach (var listener in _listeners[eventType])
40         {
41             listener.Update(filename);
42         }
43     }
44 }
45 }
46
47 // Concrete Observers
48 public class EmailAlertsListener : IEventListener
49 {
50     private readonly string _email;
51
52     public EmailAlertsListener(string email)
53     {
54         _email = email;
55     }
56
57     public void Update(string filename)
58     {
59         Console.WriteLine($"Email to {_email}: Someone has performed an operation
60             with the file {filename}");
61     }
62 }
63 public class LoggingListener : IEventListener
64 {
65     private readonly string _logFile;
66
67     public LoggingListener(string logFile)
68     {
69         _logFile = logFile;
70     }
71
72     public void Update(string filename)
73     {
74         Console.WriteLine($"Log to {_logFile}: File operation occurred on {

```

```

75         filename}");
76     }
77
78     // Publisher
79     public class Editor
80     {
81         public EventManager Events { get; } = new EventManager();
82
83         public void OpenFile(string filename)
84         {
85             Console.WriteLine($"\\nOpening file {filename}");
86             Events.Notify("open", filename);
87         }
88
89         public void SaveFile(string filename)
90         {
91             Console.WriteLine($"\\nSaving file {filename}");
92             Events.Notify("save", filename);
93         }
94     }
95
96     // Client code
97     class Program
98     {
99         static void Main(string[] args)
100         {
101             var editor = new Editor();
102
103             // Create subscribers
104             var emailAlerts = new EmailAlertsListener("admin@example.com");
105             var logger = new LoggingListener("editor.log");
106
107             // Subscribe to events
108             editor.Events.Subscribe("open", emailAlerts);
109             editor.Events.Subscribe("save", emailAlerts);
110             editor.Events.Subscribe("open", logger);
111             editor.Events.Subscribe("save", logger);
112
113             // Perform operations
114             editor.OpenFile("test.txt");
115             editor.SaveFile("test.txt");
116
117             // Unsubscribe logger from save events
118             editor.Events.Unsubscribe("save", logger);
119
120             // Perform another operation
121             editor.SaveFile("test.txt");
122
123             /* Output:
124             Subscribed EmailAlertsListener to open events
125             Subscribed EmailAlertsListener to save events
126             Subscribed LoggingListener to open events
127             Subscribed LoggingListener to save events
128
129             Opening file test.txt
130             Notifying all open subscribers...
131             Email to admin@example.com: Someone has performed an operation with the
                file test.txt

```

```

132     Log to editor.log: File operation occurred on test.txt
133
134     Saving file test.txt
135     Notifying all save subscribers...
136     Email to admin@example.com: Someone has performed an operation with the
137         file test.txt
138     Log to editor.log: File operation occurred on test.txt
139
140     Unsubscribed LoggingListener from save events
141
142     Saving file test.txt
143     Notifying all save subscribers...
144     Email to admin@example.com: Someone has performed an operation with the
145         file test.txt
146     */
147 }
148 }

```

2 Combinations

2.1 Strategy and Factory

We have different transport types (e.g., RoadTransport, SeaTransport), and each transport can have different pricing strategies (FixedRatePricing, DistanceBasedPricing). The Factory Method will create transport objects, and the Strategy Pattern will determine how the cost is calculated.

```

1  //Step 1: Define the Strategy Interface
2  public interface IPricingStrategy
3  {
4      decimal CalculateCost(decimal distance);
5  }
6  //Step 2: Create Concrete Strategies
7  public class FixedRatePricing : IPricingStrategy
8  {
9      private readonly decimal _rate;
10
11     public FixedRatePricing(decimal rate)
12     {
13         _rate = rate;
14     }
15
16     public decimal CalculateCost(decimal distance)
17     {
18         return _rate; // Fixed cost, regardless of distance
19     }
20 }
21
22 public class DistanceBasedPricing : IPricingStrategy
23 {
24     private readonly decimal _ratePerKm;
25
26     public DistanceBasedPricing(decimal ratePerKm)
27     {
28         _ratePerKm = ratePerKm;
29     }
30
31     public decimal CalculateCost(decimal distance)

```

```

32     {
33         return distance * _ratePerKm; // Cost based on distance
34     }
35 }
36 //Step 3: Define the Transport Product Interface
37 public interface ITransport
38 {
39     void Deliver();
40     void SetPricingStrategy(IPricingStrategy pricingStrategy);
41     decimal GetCost(decimal distance);
42 }
43 //Step 4: Create Concrete Transport Products
44 public class RoadTransport : ITransport
45 {
46     private IPricingStrategy _pricingStrategy;
47
48     public void Deliver()
49     {
50         Console.WriteLine("Delivering by road in a truck.");
51     }
52
53     public void SetPricingStrategy(IPricingStrategy pricingStrategy)
54     {
55         _pricingStrategy = pricingStrategy;
56     }
57
58     public decimal GetCost(decimal distance)
59     {
60         return _pricingStrategy?.CalculateCost(distance) ?? 0;
61     }
62 }
63
64 public class SeaTransport : ITransport
65 {
66     private IPricingStrategy _pricingStrategy;
67
68     public void Deliver()
69     {
70         Console.WriteLine("Delivering by sea in a ship.");
71     }
72
73     public void SetPricingStrategy(IPricingStrategy pricingStrategy)
74     {
75         _pricingStrategy = pricingStrategy;
76     }
77
78     public decimal GetCost(decimal distance)
79     {
80         return _pricingStrategy?.CalculateCost(distance) ?? 0;
81     }
82 }
83 //Step 5: Define the Creator Abstract Class (Factory Method)
84 public abstract class Logistics
85 {
86     public abstract ITransport CreateTransport();
87
88     public void PlanDelivery(IPricingStrategy pricingStrategy, decimal distance)
89     {
90         ITransport transport = CreateTransport();

```

```

91         transport.SetPricingStrategy(pricingStrategy);
92         transport.Deliver();
93         decimal cost = transport.GetCost(distance);
94         Console.WriteLine($"Total cost for delivery: {cost:C}");
95     }
96 }
97 //output
98 Delivering by road in a truck.
99 Total cost for delivery: $100.00
100 Delivering by sea in a ship.
101 Total cost for delivery: $400.00

```

2.2 Strategy and Template

Imagine a payment processing system where you need to handle different types of payments (e.g., Credit-Card, PayPal, BankTransfer). The process has a general structure (e.g., validate, authenticate, and process payment) but different payment methods may have varying implementation details. Here, the Template pattern can define the overall workflow, and the Strategy pattern can handle the specific implementation of each step.

```

1  //Step 1: Define the Strategy Interfaces
2  // Strategy Interface for Payment Authentication
3  public interface IAuthenticationStrategy
4  {
5      void Authenticate();
6  }
7
8  // Strategy Interface for Payment Processing
9  public interface IPaymentStrategy
10 {
11     void ProcessPayment(decimal amount);
12 }
13 //Step 2: Implement Concrete Strategies
14 // Concrete Strategy for Credit Card Authentication
15 public class CreditCardAuthentication : IAuthenticationStrategy
16 {
17     public void Authenticate()
18     {
19         Console.WriteLine("Authenticating via Credit Card");
20     }
21 }
22
23 // Concrete Strategy for PayPal Authentication
24 public class PayPalAuthentication : IAuthenticationStrategy
25 {
26     public void Authenticate()
27     {
28         Console.WriteLine("Authenticating via PayPal");
29     }
30 }
31
32 // Concrete Strategy for Credit Card Payment Processing
33 public class CreditCardPayment : IPaymentStrategy
34 {
35     public void ProcessPayment(decimal amount)
36     {
37         Console.WriteLine($"Processing Credit Card payment of {amount:C}");
38     }

```



```

39 }
40
41 // Concrete Strategy for PayPal Payment Processing
42 public class PayPalPayment : IPaymentStrategy
43 {
44     public void ProcessPayment(decimal amount)
45     {
46         Console.WriteLine($"Processing PayPal payment of {amount:C}");
47     }
48 }
49 //Step 3: Define the Template Base Class
50 // Template Base Class
51 public abstract class PaymentProcessor
52 {
53     private IAuthenticationStrategy _authStrategy;
54     private IPaymentStrategy _paymentStrategy;
55
56     protected PaymentProcessor(IAuthenticationStrategy authStrategy,
57                                IPaymentStrategy paymentStrategy)
58     {
59         _authStrategy = authStrategy;
60         _paymentStrategy = paymentStrategy;
61     }
62
63     // Template method defining the general steps
64     public void ExecutePayment(decimal amount)
65     {
66         Validate();
67         _authStrategy.Authenticate();
68         _paymentStrategy.ProcessPayment(amount);
69         Complete();
70     }
71
72     // Common to all types of payments, but can be overridden if needed
73     protected virtual void Validate()
74     {
75         Console.WriteLine("Validating payment details...");
76     }
77
78     // Optional step for all payments
79     protected virtual void Complete()
80     {
81         Console.WriteLine("Payment completed successfully!");
82     }
83 }
84 //Step 4: Implement Concrete Template Classes
85 // Concrete Class for Credit Card Payment Processor
86 public class CreditCardPaymentProcessor : PaymentProcessor
87 {
88     public CreditCardPaymentProcessor()
89         : base(new CreditCardAuthentication(), new CreditCardPayment())
90     {
91     }
92
93     // Override Validate if needed
94     protected override void Validate()
95     {
96         Console.WriteLine("Validating Credit Card payment details...");
97     }
98 }

```

```

97 }
98
99 // Concrete Class for PayPal Payment Processor
100 public class PayPalPaymentProcessor : PaymentProcessor
101 {
102     public PayPalPaymentProcessor()
103         : base(new PayPalAuthentication(), new PayPalPayment())
104     {
105     }
106
107     // Override Complete if needed
108     protected override void Complete()
109     {
110         Console.WriteLine("PayPal payment completed successfully with a thank you
111                             note!");
112     }
113 }
114 //Step 5: Usage Example
115 class Program
116 {
117     static void Main(string[] args)
118     {
119         PaymentProcessor creditCardProcessor = new CreditCardPaymentProcessor();
120         creditCardProcessor.ExecutePayment(150.00m);
121
122         Console.WriteLine();
123
124         PaymentProcessor paypalProcessor = new PayPalPaymentProcessor();
125         paypalProcessor.ExecutePayment(75.50m);
126     }
127 }
128 //output
129 Validating Credit Card payment details...
130 Authenticating via Credit Card
131 Processing Credit Card payment of $150.00
132 Payment completed successfully!
133
134 Validating payment details...
135 Authenticating via PayPal
136 Processing PayPal payment of $75.50
137 PayPal payment completed successfully with a thank you note!

```

2.3 Strategy and Decorator

Imagine a text editor that applies different formatting strategies (e.g., PlainText, HTML, Markdown). Additionally, you can add features like spell checking, grammar checking, and auto-correction to the text. The Strategy pattern allows you to switch between different text formatting strategies, while the Decorator pattern lets you dynamically add text-processing features.

```

1 //Step 1: Define the Strategy Interface
2 // Strategy Interface for Text Formatting
3 public interface ITextFormatter
4 {
5     string Format(string text);
6 }
7 //Step 2: Implement Concrete Strategies
8 // Concrete Strategy for Plain Text Formatting
9 public class PlainTextFormatter : ITextFormatter

```

```

10 {
11     public string Format(string text)
12     {
13         return text;
14     }
15 }
16
17 // Concrete Strategy for HTML Text Formatting
18 public class HtmlTextFormatter : ITextFormatter
19 {
20     public string Format(string text)
21     {
22         return $"<p>{text}</p>";
23     }
24 }
25
26 // Concrete Strategy for Markdown Text Formatting
27 public class MarkdownTextFormatter : ITextFormatter
28 {
29     public string Format(string text)
30     {
31         return $"**{text}**"; // Bold text in Markdown
32     }
33 }
34 //Step 3: Define the Decorator Base Class
35 // Base Decorator for Text Processing
36 public abstract class TextDecorator : ITextFormatter
37 {
38     protected ITextFormatter _textFormatter;
39
40     public TextDecorator(ITextFormatter textFormatter)
41     {
42         _textFormatter = textFormatter;
43     }
44
45     public virtual string Format(string text)
46     {
47         return _textFormatter.Format(text);
48     }
49 }
50 //Step 4: Implement Concrete Decorators
51 // Concrete Decorator for Spell Checking
52 public class SpellCheckDecorator : TextDecorator
53 {
54     public SpellCheckDecorator(ITextFormatter textFormatter) : base(textFormatter)
55     {
56     }
57
58     public override string Format(string text)
59     {
60         text = base.Format(text);
61         // Simulate spell-checking
62         return $"{text} (Spell-checked)";
63     }
64 }
65
66 // Concrete Decorator for Grammar Checking
67 public class GrammarCheckDecorator : TextDecorator
68 {

```

```

69     public GrammarCheckDecorator(ITextFormatter textFormatter) : base(
70         textFormatter)
71     {
72     }
73
74     public override string Format(string text)
75     {
76         text = base.Format(text);
77         // Simulate grammar-checking
78         return $"{text} (Grammar-checked)";
79     }
80
81     // Concrete Decorator for Auto-Correction
82     public class AutoCorrectDecorator : TextDecorator
83     {
84         public AutoCorrectDecorator(ITextFormatter textFormatter) : base(textFormatter)
85         {
86         }
87
88         public override string Format(string text)
89         {
90             text = base.Format(text);
91             // Simulate auto-correction
92             return $"{text} (Auto-corrected)";
93         }
94     }
95
96     //Step 5: Usage Example
97     class Program
98     {
99         static void Main(string[] args)
100         {
101             string text = "This is a sample text.";
102
103             // Using Plain Text Formatter
104             ITextFormatter plainTextFormatter = new PlainTextFormatter();
105             Console.WriteLine(plainTextFormatter.Format(text));
106
107             // Adding Spell Check
108             ITextFormatter spellCheckedText = new SpellCheckDecorator(
109                 plainTextFormatter);
110             Console.WriteLine(spellCheckedText.Format(text));
111
112             // Using HTML Formatter with Spell Check and Grammar Check
113             ITextFormatter htmlFormatter = new HtmlTextFormatter();
114             ITextFormatter enhancedHtmlFormatter = new GrammarCheckDecorator(new
115                 SpellCheckDecorator(htmlFormatter));
116             Console.WriteLine(enhancedHtmlFormatter.Format(text));
117
118             // Using Markdown Formatter with all features: Spell Check, Grammar Check,
119             // and Auto-Correction
120             ITextFormatter markdownFormatter = new MarkdownTextFormatter();
121             ITextFormatter fullyEnhancedMarkdown = new AutoCorrectDecorator(
122                 new GrammarCheckDecorator(
123                     new SpellCheckDecorator(markdownFormatter)
124                 )
125             );
126             Console.WriteLine(fullyEnhancedMarkdown.Format(text));

```

```

123     }
124 }
125 //output
126 This is a sample text.
127 This is a sample text. (Spell-checked)
128 <p>This is a sample text.</p> (Spell-checked) (Grammar-checked)
129 **This is a sample text.** (Spell-checked) (Grammar-checked) (Auto-corrected)

```

2.4 Strategy and Composite

Consider a Drawing Application where you can create various shapes (Circle, Rectangle, etc.) and group them into composite shapes (Group). Each shape can be drawn on the screen using different drawing strategies (e.g., Solid, Dotted, Dashed). The Composite pattern lets you treat individual shapes and groups of shapes uniformly, and the Strategy pattern allows you to apply different drawing behaviors to them.

```

1 //Step 1: Define the Strategy Interface
2 // Strategy Interface for Drawing
3 public interface IDrawStrategy
4 {
5     void Draw(string shape);
6 }
7 //Step 2: Implement Concrete Strategies
8 // Concrete Strategy for Solid Drawing
9 public class SolidDrawStrategy : IDrawStrategy
10 {
11     public void Draw(string shape)
12     {
13         Console.WriteLine($"Drawing {shape} with a solid line.");
14     }
15 }
16
17 // Concrete Strategy for Dotted Drawing
18 public class DottedDrawStrategy : IDrawStrategy
19 {
20     public void Draw(string shape)
21     {
22         Console.WriteLine($"Drawing {shape} with a dotted line.");
23     }
24 }
25
26 // Concrete Strategy for Dashed Drawing
27 public class DashedDrawStrategy : IDrawStrategy
28 {
29     public void Draw(string shape)
30     {
31         Console.WriteLine($"Drawing {shape} with a dashed line.");
32     }
33 }
34 //Step 3: Define the Component Base Class for Composite Pattern
35 // Component Base Class
36 public abstract class Shape
37 {
38     protected IDrawStrategy _drawStrategy;
39
40     public Shape(IDrawStrategy drawStrategy)
41     {
42         _drawStrategy = drawStrategy;
43     }

```

```

44     public abstract void Draw();
45     public abstract void Add(Shape shape);
46     public abstract void Remove(Shape shape);
47 }
48
49 //Step 4: Implement Leaf Classes
50 // Leaf Class for Circle
51 public class Circle : Shape
52 {
53     public Circle(IDrawStrategy drawStrategy) : base(drawStrategy) { }
54
55     public override void Draw()
56     {
57         _drawStrategy.Draw("Circle");
58     }
59
60     public override void Add(Shape shape)
61     {
62         // Cannot add to a leaf
63         throw new NotSupportedException("Cannot add to a leaf");
64     }
65
66     public override void Remove(Shape shape)
67     {
68         // Cannot remove from a leaf
69         throw new NotSupportedException("Cannot remove from a leaf");
70     }
71 }
72
73 // Leaf Class for Rectangle
74 public class Rectangle : Shape
75 {
76     public Rectangle(IDrawStrategy drawStrategy) : base(drawStrategy) { }
77
78     public override void Draw()
79     {
80         _drawStrategy.Draw("Rectangle");
81     }
82
83     public override void Add(Shape shape)
84     {
85         // Cannot add to a leaf
86         throw new NotSupportedException("Cannot add to a leaf");
87     }
88
89     public override void Remove(Shape shape)
90     {
91         // Cannot remove from a leaf
92         throw new NotSupportedException("Cannot remove from a leaf");
93     }
94 }
95 //Step 5: Implement Composite Class
96 // Composite Class for Grouping Shapes
97 public class Group : Shape
98 {
99     private List<Shape> _shapes = new List<Shape>();
100
101     public Group(IDrawStrategy drawStrategy) : base(drawStrategy) { }
102

```

```

103     public override void Draw()
104     {
105         Console.WriteLine("Drawing a group of shapes:");
106         foreach (var shape in _shapes)
107         {
108             shape.Draw();
109         }
110     }
111
112     public override void Add(Shape shape)
113     {
114         _shapes.Add(shape);
115     }
116
117     public override void Remove(Shape shape)
118     {
119         _shapes.Remove(shape);
120     }
121 }
122 //Step 6: Usage Example
123 class Program
124 {
125     static void Main(string[] args)
126     {
127         // Creating individual shapes with different drawing strategies
128         Shape circleSolid = new Circle(new SolidDrawStrategy());
129         Shape rectangleDotted = new Rectangle(new DottedDrawStrategy());
130
131         // Drawing individual shapes
132         circleSolid.Draw();
133         rectangleDotted.Draw();
134
135         // Creating a group and adding shapes to it
136         Shape group = new Group(new SolidDrawStrategy());
137         group.Add(circleSolid);
138         group.Add(rectangleDotted);
139
140         // Drawing the group of shapes
141         group.Draw();
142
143         // Adding another group within a group
144         Shape subGroup = new Group(new DashedDrawStrategy());
145         subGroup.Add(new Circle(new DashedDrawStrategy()));
146         subGroup.Add(new Rectangle(new SolidDrawStrategy()));
147
148         group.Add(subGroup);
149
150         Console.WriteLine("\nDrawing the complete group:");
151         group.Draw();
152     }
153 }
154 //output
155 Drawing Circle with a solid line.
156 Drawing Rectangle with a dotted line.
157 Drawing a group of shapes:
158 Drawing Circle with a solid line.
159 Drawing Rectangle with a dotted line.
160
161 Drawing the complete group:

```

```

162 Drawing a group of shapes:
163 Drawing Circle with a solid line.
164 Drawing Rectangle with a dotted line.
165 Drawing a group of shapes:
166 Drawing Circle with a dashed line.
167 Drawing Rectangle with a solid line.

```

2.5 Strategy and Adapter

Suppose you are building a Payment Gateway system that allows you to process payments using multiple external payment processors (e.g., PayPal, Stripe, Square). Each external processor has its own unique interface that doesn't match your system's expected interface. You can use Adapters to wrap each external processor and make them compatible, while the Strategy pattern enables you to switch between different payment processing methods seamlessly.

```

1  //Step 1: Define the Strategy Interface
2  // Strategy Interface for Payment Processing
3  public interface IPaymentProcessor
4  {
5      void ProcessPayment(decimal amount);
6  }
7  //Step 2: Implement Concrete Strategies
8  // Concrete Strategy for processing payment via PayPal
9  public class PayPalPaymentProcessor : IPaymentProcessor
10 {
11     public void ProcessPayment(decimal amount)
12     {
13         Console.WriteLine($"Processing {amount:C} via PayPal.");
14     }
15 }
16
17 // Concrete Strategy for processing payment via Stripe
18 public class StripePaymentProcessor : IPaymentProcessor
19 {
20     public void ProcessPayment(decimal amount)
21     {
22         Console.WriteLine($"Processing {amount:C} via Stripe.");
23     }
24 }
25 //Step 3: Define an Existing Service (Incompatible Interface)
26 // Existing Service for Square Payment (Incompatible Interface)
27 public class SquareService
28 {
29     public void MakePayment(double totalAmount)
30     {
31         Console.WriteLine($"Processing {totalAmount:C} via Square.");
32     }
33 }
34 //Step 4: Implement the Adapter
35 // Adapter to make SquareService compatible with IPaymentProcessor
36 public class SquarePaymentAdapter : IPaymentProcessor
37 {
38     private SquareService _squareService;
39
40     public SquarePaymentAdapter(SquareService squareService)
41     {
42         _squareService = squareService;
43     }

```



```

44     public void ProcessPayment(decimal amount)
45     {
46         // Adapt the method to match the expected interface
47         _squareService.MakePayment((double)amount);
48     }
49 }
50
51 //Step 5: Payment Context Class (Optional)
52 // Payment Context that uses a strategy
53 public class PaymentContext
54 {
55     private IPaymentProcessor _paymentProcessor;
56
57     public PaymentContext(IPaymentProcessor paymentProcessor)
58     {
59         _paymentProcessor = paymentProcessor;
60     }
61
62     public void SetPaymentProcessor(IPaymentProcessor paymentProcessor)
63     {
64         _paymentProcessor = paymentProcessor;
65     }
66
67     public void ExecutePayment(decimal amount)
68     {
69         _paymentProcessor.ProcessPayment(amount);
70     }
71 }
72 //Step 6: Usage Example
73 class Program
74 {
75     static void Main(string[] args)
76     {
77         // Using PayPal as the payment processor
78         IPaymentProcessor paypalProcessor = new PayPalPaymentProcessor();
79         PaymentContext paymentContext = new PaymentContext(paypalProcessor);
80         paymentContext.ExecutePayment(100.00m);
81
82         // Switch to Stripe
83         IPaymentProcessor stripeProcessor = new StripePaymentProcessor();
84         paymentContext.SetPaymentProcessor(stripeProcessor);
85         paymentContext.ExecutePayment(150.00m);
86
87         // Use Square via Adapter
88         SquareService squareService = new SquareService();
89         IPaymentProcessor squareAdapter = new SquarePaymentAdapter(squareService);
90         paymentContext.SetPaymentProcessor(squareAdapter);
91         paymentContext.ExecutePayment(200.00m);
92     }
93 }
94
95 \\output
96 Processing $100.00 via PayPal.
97 Processing $150.00 via Stripe.
98 Processing $200.00 via Square.

```

2.6 Strategy and Singleton

Let's consider a Logger system where different logging strategies (e.g., Console Logging, File Logging, Database Logging) can be used. The application should have a single, globally accessible Logger instance to manage the logs, and you should be able to switch between different logging strategies easily.

```
1      //Step 1: Define the Strategy Interface
2      // Strategy Interface for Logging
3  public interface ILoggerStrategy
4  {
5      void Log(string message);
6  }
7  //Step 2: Implement Concrete Strategies
8  // Concrete Strategy for Console Logging
9  public class ConsoleLogger : ILoggerStrategy
10 {
11     public void Log(string message)
12     {
13         Console.WriteLine($"[Console] {message}");
14     }
15 }
16
17 // Concrete Strategy for File Logging
18 public class FileLogger : ILoggerStrategy
19 {
20     public void Log(string message)
21     {
22         // Simulate writing to a file
23         Console.WriteLine($"[File] {message} - Logged to a file.");
24     }
25 }
26
27 // Concrete Strategy for Database Logging
28 public class DatabaseLogger : ILoggerStrategy
29 {
30     public void Log(string message)
31     {
32         // Simulate writing to a database
33         Console.WriteLine($"[Database] {message} - Logged to a database.");
34     }
35 }
36 //Step 3: Implement the Singleton Context Class
37 // Singleton Logger Context
38 public class Logger
39 {
40     private static Logger _instance;
41     private ILoggerStrategy _loggerStrategy;
42
43     // Private constructor to prevent instantiation
44     private Logger()
45     {
46         // Default to Console Logger
47         _loggerStrategy = new ConsoleLogger();
48     }
49
50     // Public method to get the singleton instance
51     public static Logger GetInstance()
52     {
53         if (_instance == null)
```

```

54     {
55         _instance = new Logger();
56     }
57     return _instance;
58 }
59
60 // Method to set the logging strategy
61 public void SetLoggerStrategy(ILoggerStrategy loggerStrategy)
62 {
63     _loggerStrategy = loggerStrategy;
64 }
65
66 // Method to log the message using the current strategy
67 public void Log(string message)
68 {
69     _loggerStrategy.Log(message);
70 }
71 }
72 //Step 4: Usage Example
73 class Program
74 {
75     static void Main(string[] args)
76     {
77         // Get the singleton Logger instance
78         Logger logger = Logger.GetInstance();
79
80         // Use the default Console Logger
81         logger.Log("This is a test message.");
82
83         // Change to File Logger
84         logger.SetLoggerStrategy(new FileLogger());
85         logger.Log("Logging to a file now.");
86
87         // Change to Database Logger
88         logger.SetLoggerStrategy(new DatabaseLogger());
89         logger.Log("Logging to a database now.");
90
91         // Demonstrate that it's the same singleton instance
92         Logger anotherLoggerReference = Logger.GetInstance();
93         anotherLoggerReference.Log("Using the same logger instance.");
94     }
95 }
96 //output
97 [Console] This is a test message.
98 [File] Logging to a file now. - Logged to a file.
99 [Database] Logging to a database now. - Logged to a database.
100 [Database] Using the same logger instance. - Logged to a database.

```

2.7 Strategy, Factory and Template

Suppose you are building a Report Generation System that generates reports in different formats (e.g., PDF, Excel, HTML). The process for generating these reports follows a common workflow but may have slight variations depending on the format. Additionally, the system should allow for dynamic creation of the report generators without directly instantiating them, so it can easily add or switch formats.

- Strategy Pattern: To define interchangeable ways to generate content for the report.
- Template Pattern: To outline the skeleton of the report generation process and allow subclasses to

customize parts of the process.

- Factory Pattern: To create specific instances of report generators dynamically.

```
1  //Step 1: Define the Strategy Interface
2  // Strategy Interface for Content Generation
3  public interface IContentStrategy
4  {
5      string GenerateContent();
6  }
7  //Step 2: Implement Concrete Strategies
8  // Concrete Strategy for Financial Report Content
9  public class FinancialContentStrategy : IContentStrategy
10 {
11     public string GenerateContent()
12     {
13         return "Financial Report Content";
14     }
15 }
16
17 // Concrete Strategy for Marketing Report Content
18 public class MarketingContentStrategy : IContentStrategy
19 {
20     public string GenerateContent()
21     {
22         return "Marketing Report Content";
23     }
24 }
25 //Step 3: Define the Template Base Class
26 // Template Base Class for Report Generation
27 public abstract class ReportGenerator
28 {
29     protected IContentStrategy _contentStrategy;
30
31     public ReportGenerator(IContentStrategy contentStrategy)
32     {
33         _contentStrategy = contentStrategy;
34     }
35
36     // Template method defining the general process
37     public void GenerateReport()
38     {
39         PrepareData();
40         string content = _contentStrategy.GenerateContent();
41         Export(content);
42         FinalizeReport();
43     }
44
45     // Common step for all report generation
46     protected virtual void PrepareData()
47     {
48         Console.WriteLine("Preparing data for report...");
49     }
50
51     // Abstract method for exporting the report, must be implemented by subclasses
52     protected abstract void Export(string content);
53
54     // Optional step that can be overridden
55     protected virtual void FinalizeReport()
```

```

56     {
57         Console.WriteLine("Finalizing the report...");
58     }
59 }
60 //Step 4: Implement Concrete Template Classes
61 // Concrete Class for PDF Report Generation
62 public class PdfReportGenerator : ReportGenerator
63 {
64     public PdfReportGenerator(IContentStrategy contentStrategy) : base(
65         contentStrategy) { }
66
67     protected override void Export(string content)
68     {
69         Console.WriteLine($"Exporting {content} to PDF.");
70     }
71 }
72 // Concrete Class for Excel Report Generation
73 public class ExcelReportGenerator : ReportGenerator
74 {
75     public ExcelReportGenerator(IContentStrategy contentStrategy) : base(
76         contentStrategy) { }
77
78     protected override void Export(string content)
79     {
80         Console.WriteLine($"Exporting {content} to Excel.");
81     }
82 }
83 // Concrete Class for HTML Report Generation
84 public class HtmlReportGenerator : ReportGenerator
85 {
86     public HtmlReportGenerator(IContentStrategy contentStrategy) : base(
87         contentStrategy) { }
88
89     protected override void Export(string content)
90     {
91         Console.WriteLine($"Exporting {content} to HTML.");
92     }
93 }
94 //Step 5: Implement the Factory Class
95 // Factory Class for Creating Report Generators
96 public class ReportGeneratorFactory
97 {
98     public static ReportGenerator CreateReportGenerator(string format,
99         IContentStrategy contentStrategy)
100     {
101         switch (format.ToLower())
102         {
103             case "pdf":
104                 return new PdfReportGenerator(contentStrategy);
105             case "excel":
106                 return new ExcelReportGenerator(contentStrategy);
107             case "html":
108                 return new HtmlReportGenerator(contentStrategy);
109             default:
110                 throw new ArgumentException("Unknown report format.");
111         }
112     }
113 }

```

```

111 }
112 //Step 6: Usage Example
113 class Program
114 {
115     static void Main(string[] args)
116     {
117         // Use Factory to create a PDF report generator with financial content
118         IContentStrategy financialStrategy = new FinancialContentStrategy();
119         ReportGenerator pdfReport = ReportGeneratorFactory.CreateReportGenerator("
120             pdf", financialStrategy);
121         pdfReport.GenerateReport();
122
123         Console.WriteLine();
124
125         // Use Factory to create an Excel report generator with marketing content
126         IContentStrategy marketingStrategy = new MarketingContentStrategy();
127         ReportGenerator excelReport = ReportGeneratorFactory.CreateReportGenerator(
128             "excel", marketingStrategy);
129         excelReport.GenerateReport();
130
131         Console.WriteLine();
132
133         // Use Factory to create an HTML report generator with marketing content
134         ReportGenerator htmlReport = ReportGeneratorFactory.CreateReportGenerator(
135             "html", marketingStrategy);
136         htmlReport.GenerateReport();
137     }
138 }
139
140 //output
141 Preparing data for report...
142 Exporting Financial Report Content to PDF.
143 Finalizing the report...
144
145 Preparing data for report...
146 Exporting Marketing Report Content to Excel.
147 Finalizing the report...
148
149 Preparing data for report...
150 Exporting Marketing Report Content to HTML.
151 Finalizing the report...

```

2.8 Decorator, Composite and Adapter

Suppose you are building a Graphics Editor where different types of shapes (like circles, rectangles, etc.) can be combined into groups, and additional features (like color, border, shadow) can be added to individual shapes or groups. Moreover, some external shapes might come from a different system with a different interface, which requires adaptation.

- Decorator Pattern: To add features dynamically to individual shapes (e.g., adding color, border, shadow).
- Composite Pattern: To treat individual shapes and groups of shapes uniformly, allowing for nested groups.
- Adapter Pattern: To adapt external shapes to be compatible with the existing system interface.

```

1 //Step 1: Define the Component Interface
2 // Component Interface

```

```

3 public interface IShape
4 {
5     void Draw();
6 }
7 //Step 2: Implement Leaf Classes
8 // Concrete Class for Circle
9 public class Circle : IShape
10 {
11     public void Draw()
12     {
13         Console.WriteLine("Drawing a Circle");
14     }
15 }
16
17 // Concrete Class for Rectangle
18 public class Rectangle : IShape
19 {
20     public void Draw()
21     {
22         Console.WriteLine("Drawing a Rectangle");
23     }
24 }
25 //Step 3: Implement Composite Class
26 // Composite Class for Group of Shapes
27 public class ShapeGroup : IShape
28 {
29     private List<IShape> _shapes = new List<IShape>();
30
31     public void Add(IShape shape)
32     {
33         _shapes.Add(shape);
34     }
35
36     public void Remove(IShape shape)
37     {
38         _shapes.Remove(shape);
39     }
40
41     public void Draw()
42     {
43         Console.WriteLine("Drawing a Group of Shapes:");
44         foreach (var shape in _shapes)
45         {
46             shape.Draw();
47         }
48     }
49 }
50 //Step 4: Implement Decorator Base Class
51 // Decorator Base Class
52 public abstract class ShapeDecorator : IShape
53 {
54     protected IShape _shape;
55
56     public ShapeDecorator(IShape shape)
57     {
58         _shape = shape;
59     }
60
61     public virtual void Draw()

```

```

62     {
63         _shape.Draw();
64     }
65 }
66 //Step 5: Implement Concrete Decorators
67 // Concrete Decorator for Color
68 public class ColorDecorator : ShapeDecorator
69 {
70     private string _color;
71
72     public ColorDecorator(IShape shape, string color) : base(shape)
73     {
74         _color = color;
75     }
76
77     public override void Draw()
78     {
79         Console.WriteLine($"Adding {_color} Color");
80         _shape.Draw();
81     }
82 }
83
84 // Concrete Decorator for Border
85 public class BorderDecorator : ShapeDecorator
86 {
87     public BorderDecorator(IShape shape) : base(shape) { }
88
89     public override void Draw()
90     {
91         Console.WriteLine("Adding Border");
92         _shape.Draw();
93     }
94 }
95
96 // Concrete Decorator for Shadow
97 public class ShadowDecorator : ShapeDecorator
98 {
99     public ShadowDecorator(IShape shape) : base(shape) { }
100
101     public override void Draw()
102     {
103         Console.WriteLine("Adding Shadow");
104         _shape.Draw();
105     }
106 }
107 //Step 6: Implement External Shape (Incompatible Interface)
108 // External Shape (Incompatible Interface)
109 public class LegacyTriangle
110 {
111     public void Render()
112     {
113         Console.WriteLine("Rendering a Triangle (from Legacy System)");
114     }
115 }
116 //Step 7: Implement Adapter
117 // Adapter to make LegacyTriangle compatible with IShape
118 public class TriangleAdapter : IShape
119 {
120     private LegacyTriangle _legacyTriangle;

```



```

121     public TriangleAdapter(LegacyTriangle legacyTriangle)
122     {
123         _legacyTriangle = legacyTriangle;
124     }
125
126     public void Draw()
127     {
128         // Adapting the method to fit the IShape interface
129         _legacyTriangle.Render();
130     }
131 }
132
133 //Step 8: Usage Example
134 class Program
135 {
136     static void Main(string[] args)
137     {
138         // Creating individual shapes
139         IShape circle = new Circle();
140         IShape rectangle = new Rectangle();
141
142         // Adding features using Decorators
143         IShape coloredCircle = new ColorDecorator(circle, "Red");
144         IShape borderedRectangle = new BorderDecorator(rectangle);
145
146         // Using the Adapter to integrate a legacy shape
147         LegacyTriangle legacyTriangle = new LegacyTriangle();
148         IShape adaptedTriangle = new TriangleAdapter(legacyTriangle);
149
150         // Creating a group of shapes using Composite
151         ShapeGroup shapeGroup = new ShapeGroup();
152         shapeGroup.Add(coloredCircle);
153         shapeGroup.Add(borderedRectangle);
154         shapeGroup.Add(adaptedTriangle);
155
156         // Drawing all shapes in the group
157         shapeGroup.Draw();
158
159         Console.WriteLine();
160
161         // Adding more decorations to the group
162         IShape shadowedGroup = new ShadowDecorator(shapeGroup);
163         shadowedGroup.Draw();
164     }
165 }
166
167 //output
168 Adding Red Color
169 Drawing a Circle
170 Adding Border
171 Drawing a Rectangle
172 Rendering a Triangle (from Legacy System)
173
174 Adding Shadow
175 Drawing a Group of Shapes:
176 Adding Red Color
177 Drawing a Circle
178 Adding Border
179 Drawing a Rectangle
180 Rendering a Triangle (from Legacy System)

```

2.9 Factory, Singleton and Adapter

Suppose you are building a Database Connection Management System where your application supports multiple databases (e.g., MySQL, SQL Server, PostgreSQL). Each database connection has a different interface, and you need to ensure that there is a single instance of each connection type across the application (to save resources). You also need to adapt the existing database client libraries to a common interface.

- Factory Pattern: To create database connection objects dynamically based on the type of database (MySQL, SQL Server, etc.).
- Singleton Pattern: To ensure only one instance of each database connection exists across the application.
- Adapter Pattern: To adapt existing database client libraries (which may not match the required interface) to a common interface.

```
1      //Step 1: Define the Target Interface
2      // Target Interface for Database Connections
3      public interface IDatabaseConnection
4      {
5          void Connect();
6          void Disconnect();
7      }
8      //Step 2: Implement External Database Clients (Incompatible Interfaces)
9      // External MySQL Client (Incompatible Interface)
10     public class MySQLClient
11     {
12         public void OpenConnection()
13         {
14             Console.WriteLine("Connecting to MySQL Database...");
15         }
16
17         public void CloseConnection()
18         {
19             Console.WriteLine("Disconnecting from MySQL Database...");
20         }
21     }
22
23     // External SQL Server Client (Incompatible Interface)
24     public class SQLServerClient
25     {
26         public void ConnectToServer()
27         {
28             Console.WriteLine("Connecting to SQL Server Database...");
29         }
30
31         public void DisconnectFromServer()
32         {
33             Console.WriteLine("Disconnecting from SQL Server Database...");
34         }
35     }
36     //Step 3: Implement Adapters
37     // Adapter for MySQL Client
38     public class MySQLAdapter : IDatabaseConnection
39     {
40         private MySQLClient _mysqlClient;
```

```

41
42     public MySQLAdapter(MySQLClient mysqlClient)
43     {
44         _mysqlClient = mysqlClient;
45     }
46
47     public void Connect()
48     {
49         _mysqlClient.OpenConnection();
50     }
51
52     public void Disconnect()
53     {
54         _mysqlClient.CloseConnection();
55     }
56 }
57
58 // Adapter for SQL Server Client
59 public class SQLServerAdapter : IDatabaseConnection
60 {
61     private SQLServerClient _sqlServerClient;
62
63     public SQLServerAdapter(SQLServerClient sqlServerClient)
64     {
65         _sqlServerClient = sqlServerClient;
66     }
67
68     public void Connect()
69     {
70         _sqlServerClient.ConnectToServer();
71     }
72
73     public void Disconnect()
74     {
75         _sqlServerClient.DisconnectFromServer();
76     }
77 }
78 //Step 4: Implement the Singleton Class
79 // Singleton Class for Managing Database Connections
80 public class DatabaseConnectionManager
81 {
82     private static Dictionary<string, IDatabaseConnection> _connections = new
83         Dictionary<string, IDatabaseConnection>();
84
85     private DatabaseConnectionManager() { }
86
87     public static IDatabaseConnection GetConnection(string type)
88     {
89         if (!_connections.ContainsKey(type))
90         {
91             // Use Factory to create the connection
92             _connections[type] = DatabaseConnectionFactory.CreateConnection(type);
93         }
94         return _connections[type];
95     }
96 }
97 //Step 5: Implement the Factory Class
98 // Factory Class for Creating Database Connections
99 public class DatabaseConnectionFactory

```

```

99 {
100     public static IDatabaseConnection CreateConnection(string type)
101     {
102         switch (type.ToLower())
103         {
104             case "mysql":
105                 return new MySQLAdapter(new MySQLClient());
106             case "sqlserver":
107                 return new SQLServerAdapter(new SQLServerClient());
108             default:
109                 throw new ArgumentException("Unknown database type.");
110         }
111     }
112 }
113 //Step 6: Usage Example
114 class Program
115 {
116     static void Main(string[] args)
117     {
118         // Get MySQL connection using Singleton and Adapter
119         IDatabaseConnection mysqlConnection = DatabaseConnectionManager.
120             GetConnection("mysql");
121         mysqlConnection.Connect();
122         mysqlConnection.Disconnect();
123
124         Console.WriteLine();
125
126         // Get SQL Server connection using Singleton and Adapter
127         IDatabaseConnection sqlServerConnection = DatabaseConnectionManager.
128             GetConnection("sqlserver");
129         sqlServerConnection.Connect();
130         sqlServerConnection.Disconnect();
131
132         Console.WriteLine();
133
134         // Try to get the MySQL connection again (it will be the same instance)
135         IDatabaseConnection mysqlConnection2 = DatabaseConnectionManager.
136             GetConnection("mysql");
137         Console.WriteLine("Are both MySQL connections the same instance? " + (
138             mysqlConnection == mysqlConnection2));
139     }
140 }
141 //output
142 Connecting to MySQL Database...
143 Disconnecting from MySQL Database...
144
145 Connecting to SQL Server Database...
146 Disconnecting from SQL Server Database...
147
148 Are both MySQL connections the same instance? True

```

2.10 Template, Factory and Strategy

Imagine you are building a Data Processing System that can handle different types of data sources (e.g., CSV, JSON, XML). The system should have a general workflow for processing data (e.g., loading data, transforming it, and saving it). Different data sources might require different processing strategies (e.g., parsing logic), and new data sources should be easily addable.

- Template Pattern: To outline the general workflow for processing data.
- Factory Pattern: To create instances of data processors dynamically.
- Strategy Pattern: To define interchangeable parsing strategies that can be swapped out based on the type of data.

```

1      //Step 1: Define the Strategy Interface
2
3      // Strategy Interface for Parsing Data
4      public interface IDataParser
5      {
6          void ParseData(string rawData);
7      }
8      //Step 2: Implement Concrete Strategies
9
10     // Concrete Strategy for Parsing CSV Data
11     public class CsvParser : IDataParser
12     {
13         public void ParseData(string rawData)
14         {
15             Console.WriteLine("Parsing CSV data...");
16             // Simulated parsing logic
17         }
18     }
19
20     // Concrete Strategy for Parsing JSON Data
21     public class JsonParser : IDataParser
22     {
23         public void ParseData(string rawData)
24         {
25             Console.WriteLine("Parsing JSON data...");
26             // Simulated parsing logic
27         }
28     }
29
30     // Concrete Strategy for Parsing XML Data
31     public class XmlParser : IDataParser
32     {
33         public void ParseData(string rawData)
34         {
35             Console.WriteLine("Parsing XML data...");
36             // Simulated parsing logic
37         }
38     }
39     //Step 3: Define the Template Base Class
40
41     // Template Base Class for Data Processing
42     public abstract class DataProcessor
43     {
44         protected IDataParser _parser;
45
46         public DataProcessor(IDataParser parser)
47         {
48             _parser = parser;
49         }
50
51         // Template method defining the workflow
52         public void Process(string rawData)

```

```

53     {
54         LoadData(rawData);
55         _parser.ParseData(rawData); // Delegate parsing to the strategy
56         SaveData();
57     }
58
59     // Common step for all data processing
60     protected virtual void LoadData(string rawData)
61     {
62         Console.WriteLine("Loading raw data...");
63     }
64
65     // Abstract method for saving data, to be implemented by subclasses
66     protected abstract void SaveData();
67 }
68 //Step 4: Implement Concrete Template Classes
69
70 // Concrete Class for CSV Data Processor
71 public class CsvDataProcessor : DataProcessor
72 {
73     public CsvDataProcessor(IDataParser parser) : base(parser) { }
74
75     protected override void SaveData()
76     {
77         Console.WriteLine("Saving data in CSV format...");
78     }
79 }
80
81 // Concrete Class for JSON Data Processor
82 public class JsonDataProcessor : DataProcessor
83 {
84     public JsonDataProcessor(IDataParser parser) : base(parser) { }
85
86     protected override void SaveData()
87     {
88         Console.WriteLine("Saving data in JSON format...");
89     }
90 }
91
92 // Concrete Class for XML Data Processor
93 public class XmlDataProcessor : DataProcessor
94 {
95     public XmlDataProcessor(IDataParser parser) : base(parser) { }
96
97     protected override void SaveData()
98     {
99         Console.WriteLine("Saving data in XML format...");
100     }
101 }
102 //Step 5: Implement the Factory Class
103
104 // Factory Class for Creating Data Processors
105 public class DataProcessorFactory
106 {
107     public static DataProcessor CreateDataProcessor(string type, IDataParser
        parser)
108     {
109         switch (type.ToLower())
110         {

```

```

111         case "csv":
112             return new CsvDataProcessor(parser);
113         case "json":
114             return new JsonDataProcessor(parser);
115         case "xml":
116             return new XmlDataProcessor(parser);
117         default:
118             throw new ArgumentException("Unknown data processor type.");
119     }
120 }
121 }
122 //Step 6: Usage Example
123
124 class Program
125 {
126     static void Main(string[] args)
127     {
128         // Use Factory to create a CSV Data Processor with CSV Parsing Strategy
129         IDataParser csvParser = new CsvParser();
130         DataProcessor csvProcessor = DataProcessorFactory.CreateDataProcessor("csv",
131                                     csvParser);
132         csvProcessor.Process("Sample CSV Data");
133
134         Console.WriteLine();
135
136         // Use Factory to create a JSON Data Processor with JSON Parsing Strategy
137         IDataParser jsonParser = new JsonParser();
138         DataProcessor jsonProcessor = DataProcessorFactory.CreateDataProcessor("json",
139                                     jsonParser);
140         jsonProcessor.Process("Sample JSON Data");
141
142         Console.WriteLine();
143
144         // Use Factory to create an XML Data Processor with XML Parsing Strategy
145         IDataParser xmlParser = new XmlParser();
146         DataProcessor xmlProcessor = DataProcessorFactory.CreateDataProcessor("xml",
147                                     xmlParser);
148         xmlProcessor.Process("Sample XML Data");
149     }
150 }
151 //Output
152
153 Loading raw data...
154 Parsing CSV data...
155 Saving data in CSV format...
156
157 Loading raw data...
158 Parsing JSON data...
159 Saving data in JSON format...
160
161 Loading raw data...
162 Parsing XML data...
163 Saving data in XML format...

```

2.11 Strategy, Decorator and Factory

Imagine you are building a Coffee Shop System where customers can order different types of beverages (e.g., Coffee, Tea, Hot Chocolate), and they can add various add-ons (e.g., Milk, Sugar, Whipped Cream) to their

drinks. The system should allow:

- Dynamic selection of drink types (e.g., Coffee, Tea) using the Strategy pattern.
- Enhancing beverages with add-ons like Milk or Sugar using the Decorator pattern.
- Creating beverage objects efficiently using the Factory pattern.

```
1      //Step 1: Define the Strategy Interface
2
3      // Strategy Interface for Beverages
4      public interface IBeverage
5      {
6          string GetDescription();
7          double GetCost();
8      }
9      //Step 2: Implement Concrete Strategies
10
11     // Concrete Strategy for Coffee
12     public class Coffee : IBeverage
13     {
14         public string GetDescription()
15         {
16             return "Coffee";
17         }
18
19         public double GetCost()
20         {
21             return 2.00; // Cost in USD
22         }
23     }
24
25     // Concrete Strategy for Tea
26     public class Tea : IBeverage
27     {
28         public string GetDescription()
29         {
30             return "Tea";
31         }
32
33         public double GetCost()
34         {
35             return 1.50; // Cost in USD
36         }
37     }
38
39     // Concrete Strategy for Hot Chocolate
40     public class HotChocolate : IBeverage
41     {
42         public string GetDescription()
43         {
44             return "Hot Chocolate";
45         }
46
47         public double GetCost()
48         {
49             return 2.50; // Cost in USD
50         }
51     }
```



```

52 //Step 3: Implement the Decorator Base Class
53
54 // Decorator Base Class
55 public abstract class BeverageDecorator : IBeverage
56 {
57     protected IBeverage _beverage;
58
59     public BeverageDecorator(IBeverage beverage)
60     {
61         _beverage = beverage;
62     }
63
64     public virtual string GetDescription()
65     {
66         return _beverage.GetDescription();
67     }
68
69     public virtual double GetCost()
70     {
71         return _beverage.GetCost();
72     }
73 }
74 //Step 4: Implement Concrete Decorators
75
76 // Concrete Decorator for Milk
77 public class MilkDecorator : BeverageDecorator
78 {
79     public MilkDecorator(IBeverage beverage) : base(beverage) { }
80
81     public override string GetDescription()
82     {
83         return _beverage.GetDescription() + ", Milk";
84     }
85
86     public override double GetCost()
87     {
88         return _beverage.GetCost() + 0.50; // Adding milk cost
89     }
90 }
91
92 // Concrete Decorator for Sugar
93 public class SugarDecorator : BeverageDecorator
94 {
95     public SugarDecorator(IBeverage beverage) : base(beverage) { }
96
97     public override string GetDescription()
98     {
99         return _beverage.GetDescription() + ", Sugar";
100     }
101
102     public override double GetCost()
103     {
104         return _beverage.GetCost() + 0.20; // Adding sugar cost
105     }
106 }
107
108 // Concrete Decorator for Whipped Cream
109 public class WhippedCreamDecorator : BeverageDecorator
110 {

```

```

111     public WhippedCreamDecorator(IBeverage beverage) : base(beverage) { }
112
113     public override string GetDescription()
114     {
115         return _beverage.GetDescription() + ", Whipped Cream";
116     }
117
118     public override double GetCost()
119     {
120         return _beverage.GetCost() + 0.70; // Adding whipped cream cost
121     }
122 }
123 //Step 5: Implement the Factory Class
124
125 // Factory Class for Creating Beverages
126 public class BeverageFactory
127 {
128     public static IBeverage CreateBeverage(string type)
129     {
130         switch (type.ToLower())
131         {
132             case "coffee":
133                 return new Coffee();
134             case "tea":
135                 return new Tea();
136             case "hotchocolate":
137                 return new HotChocolate();
138             default:
139                 throw new ArgumentException("Unknown beverage type.");
140         }
141     }
142 }
143 //Step 6: Usage Example
144
145 class Program
146 {
147     static void Main(string[] args)
148     {
149         // Use Factory to create a Coffee beverage
150         IBeverage coffee = BeverageFactory.CreateBeverage("coffee");
151
152         // Decorate the Coffee with Milk and Sugar
153         IBeverage coffeeWithAddOns = new SugarDecorator(new MilkDecorator(coffee));
154
155         Console.WriteLine($"{coffeeWithAddOns.GetDescription()} - ${
156             coffeeWithAddOns.GetCost():F2}");
157
158         Console.WriteLine();
159
160         // Use Factory to create a Tea beverage
161         IBeverage tea = BeverageFactory.CreateBeverage("tea");
162
163         // Decorate the Tea with Whipped Cream
164         IBeverage teaWithWhippedCream = new WhippedCreamDecorator(tea);
165         Console.WriteLine($"{teaWithWhippedCream.GetDescription()} - ${
166             teaWithWhippedCream.GetCost():F2}");
167     }
168 }
169 //Output

```

```
167
168 Coffee, Milk, Sugar - $2.70
169
170 Tea, Whipped Cream - $2.20
```

2.12 Observer + Command

Command Pattern: We use it to encapsulate requests (turn on/off the light).

Observer Pattern: We notify observers when the light's state changes.

```
1      using System;
2      using System.Collections.Generic;
3
4      // ----- Command Pattern -----
5
6      public interface ICommand
7      {
8          void Execute();
9      }
10
11     public class Light
12     {
13         private bool isOn = false;
14         private List<ILightObserver> observers = new List<ILightObserver>();
15
16         public void AddObserver(ILightObserver observer)
17         {
18             observers.Add(observer);
19         }
20
21         public void RemoveObserver(ILightObserver observer)
22         {
23             observers.Remove(observer);
24         }
25
26         private void NotifyObservers()
27         {
28             foreach (var observer in observers)
29             {
30                 observer.Update(isOn);
31             }
32         }
33
34         public void TurnOn()
35         {
36             isOn = true;
37             Console.WriteLine("Light turned ON");
38             NotifyObservers();
39         }
40
41         public void TurnOff()
42         {
43             isOn = false;
44             Console.WriteLine("Light turned OFF");
45             NotifyObservers();
46         }
47     }
48
```

```

49 public class LightOnCommand : ICommand
50 {
51     private Light _light;
52
53     public LightOnCommand(Light light)
54     {
55         _light = light;
56     }
57
58     public void Execute()
59     {
60         _light.TurnOn();
61     }
62 }
63
64 public class LightOffCommand : ICommand
65 {
66     private Light _light;
67
68     public LightOffCommand(Light light)
69     {
70         _light = light;
71     }
72
73     public void Execute()
74     {
75         _light.TurnOff();
76     }
77 }
78
79 public class RemoteControl
80 {
81     private ICommand _command;
82
83     public void SetCommand(ICommand command)
84     {
85         _command = command;
86     }
87
88     public void PressButton()
89     {
90         _command.Execute();
91     }
92 }
93
94 // ----- Observer Pattern -----
95
96 public interface ILightObserver
97 {
98     void Update(bool isOn);
99 }
100
101 public class LightLogger : ILightObserver
102 {
103     public void Update(bool isOn)
104     {
105         Console.WriteLine($"[Logger] Light is now {(isOn ? "ON" : "OFF")}");
106     }
107 }

```

```

108
109 public class LightDashboard : ILightObserver
110 {
111     public void Update(bool isOn)
112     {
113         Console.WriteLine($"[Dashboard] Display updated: Light is {(isOn ? "ON" :
114             "OFF")}");
115     }
116 }
117 // ----- Main Program -----
118
119 class Program
120 {
121     static void Main(string[] args)
122     {
123         Light livingRoomLight = new Light();
124
125         // Attach observers
126         livingRoomLight.AddObserver(new LightLogger());
127         livingRoomLight.AddObserver(new LightDashboard());
128
129         // Create commands
130         ICommand turnOn = new LightOnCommand(livingRoomLight);
131         ICommand turnOff = new LightOffCommand(livingRoomLight);
132
133         // Use remote control
134         RemoteControl remote = new RemoteControl();
135
136         remote.SetCommand(turnOn);
137         remote.PressButton();
138
139         remote.SetCommand(turnOff);
140         remote.PressButton();
141     }
142 }
143 //output
144 Light turned ON
145 [Logger] Light is now ON
146 [Dashboard] Display updated: Light is ON
147 Light turned OFF
148 [Logger] Light is now OFF
149 [Dashboard] Display updated: Light is OFF

```

2.13 proxy + Flyweight

A document viewer displays many images.

Images are large, so we share them using Flyweight to avoid loading the same image multiple times.

A Proxy controls access to the Flyweight, e.g., lazy loading or access logging.

```

1     using System;
2     using System.Collections.Generic;
3
4     // ----- Flyweight (Shared Object) -----
5     public class Image
6     {
7         private string _fileName;
8

```

```

9      public Image(string fileName)
10     {
11         _fileName = fileName;
12         LoadImage();
13     }
14
15     private void LoadImage()
16     {
17         Console.WriteLine($"[Loading] Image: {_fileName}");
18         // Simulate heavy loading
19     }
20
21     public void Display(int x, int y)
22     {
23         Console.WriteLine($"Displaying '{_fileName}' at ({x},{y})");
24     }
25 }
26
27 // ----- Flyweight Factory -----
28 public class ImageFactory
29 {
30     private Dictionary<string, Image> _imagePool = new Dictionary<string, Image>()
31     ;
32
33     public Image GetImage(string fileName)
34     {
35         if (!_imagePool.ContainsKey(fileName))
36         {
37             _imagePool[fileName] = new Image(fileName);
38         }
39         return _imagePool[fileName];
40     }
41 }
42
43 // ----- Proxy (Virtual Proxy for Lazy Loading) -----
44 public class ImageProxy
45 {
46     private string _fileName;
47     private Image _realImage;
48     private ImageFactory _factory;
49
50     public ImageProxy(string fileName, ImageFactory factory)
51     {
52         _fileName = fileName;
53         _factory = factory;
54     }
55
56     public void Display(int x, int y)
57     {
58         if (_realImage == null)
59         {
60             Console.WriteLine($"[Proxy] Loading image via factory for: {_fileName}");
61             _realImage = _factory.GetImage(_fileName);
62         }
63         _realImage.Display(x, y);
64     }
65 }

```

```

66 // ----- Client -----
67 class Program
68 {
69     static void Main(string[] args)
70     {
71         ImageFactory factory = new ImageFactory();
72
73         // Proxies for images (simulates multiple uses)
74         var image1 = new ImageProxy("tree.png", factory);
75         var image2 = new ImageProxy("house.png", factory);
76         var image3 = new ImageProxy("tree.png", factory); // Same as image1 (
            shared)
77
78         image1.Display(10, 20);
79         image2.Display(30, 40);
80         image3.Display(50, 60); // Should NOT reload "tree.png"
81     }
82 }
83 //output
84 [Proxy] Loading image via factory for: tree.png
85 [Loading] Image: tree.png
86 Displaying 'tree.png' at (10,20)
87 [Proxy] Loading image via factory for: house.png
88 [Loading] Image: house.png
89 Displaying 'house.png' at (30,40)
90 [Proxy] Loading image via factory for: tree.png
91 Displaying 'tree.png' at (50,60)

```

2.14 Facade + Prototype

The game has a complex system for building characters with weapons, armor, and skills.

A Facade hides this complexity behind a simple interface.

A Prototype allows cloning a fully built character as a template for new players or enemies.

```

1     using System;
2
3 // ----- Prototype -----
4 public interface IPrototype<T>
5 {
6     T Clone();
7 }
8
9 public class Character : IPrototype<Character>
10 {
11     public string Name { get; set; }
12     public string Class { get; set; }
13     public string Weapon { get; set; }
14     public string Armor { get; set; }
15
16     public Character Clone()
17     {
18         return (Character)this.MemberwiseClone(); // Shallow copy is enough here
19     }
20
21     public void Display()
22     {
23         Console.WriteLine($"Name: {Name}, Class: {Class}, Weapon: {Weapon}, Armor:
            {Armor}");
24     }
25 }

```

```

24     }
25 }
26
27 // ----- Subsystems Hidden by Facade -----
28 public class WeaponSystem
29 {
30     public string GetWeaponForClass(string characterClass)
31     {
32         return characterClass switch
33         {
34             "Warrior" => "Sword",
35             "Mage" => "Staff",
36             "Archer" => "Bow",
37             _ => "Fist"
38         };
39     }
40 }
41
42 public class ArmorSystem
43 {
44     public string GetArmorForClass(string characterClass)
45     {
46         return characterClass switch
47         {
48             "Warrior" => "Heavy Armor",
49             "Mage" => "Robe",
50             "Archer" => "Leather Armor",
51             _ => "Cloth"
52         };
53     }
54 }
55
56 // ----- Facade -----
57 public class CharacterBuilderFacade
58 {
59     private WeaponSystem _weaponSystem = new WeaponSystem();
60     private ArmorSystem _armorSystem = new ArmorSystem();
61
62     public Character CreateCharacter(string name, string characterClass)
63     {
64         return new Character
65         {
66             Name = name,
67             Class = characterClass,
68             Weapon = _weaponSystem.GetWeaponForClass(characterClass),
69             Armor = _armorSystem.GetArmorForClass(characterClass)
70         };
71     }
72 }
73
74 // ----- Client -----
75 class Program
76 {
77     static void Main(string[] args)
78     {
79         CharacterBuilderFacade builder = new CharacterBuilderFacade();
80
81         // Use facade to build a Warrior
82         Character warrior = builder.CreateCharacter("Arthas", "Warrior");

```



```

83     warrior.Display();
84
85     // Clone the Warrior to create a new enemy
86     Character enemyWarrior = warrior.Clone();
87     enemyWarrior.Name = "Enemy Clone";
88     enemyWarrior.Display();
89 }
90 }
91 //output
92 Name: Arthas, Class: Warrior, Weapon: Sword, Armor: Heavy Armor
93 Name: Enemy Clone, Class: Warrior, Weapon: Sword, Armor: Heavy Armor

```

2.15 Command + Prototype

Each action (like typing, deleting) is encapsulated as a Command.

To support undo/redo or repeat, we clone the commands using Prototype.

```

1     using System;
2     using System.Collections.Generic;
3
4     // ----- Prototype Interface -----
5     public interface IPrototype<T>
6     {
7         T Clone();
8     }
9
10    // ----- Command Interface -----
11    public interface ICommand : IPrototype<ICommand>
12    {
13        void Execute();
14    }
15
16    // ----- Receiver -----
17    public class TextEditor
18    {
19        public string Text { get; private set; } = "";
20
21        public void Append(string text)
22        {
23            Text += text;
24        }
25
26        public void RemoveLast(int length)
27        {
28            if (length <= Text.Length)
29                Text = Text.Substring(0, Text.Length - length);
30        }
31
32        public void Show()
33        {
34            Console.WriteLine($"[Editor] Current Text: \"{Text}\"");
35        }
36    }
37
38    // ----- Concrete Command -----
39    public class AddTextCommand : ICommand
40    {
41        private TextEditor _editor;

```

```

42     private string _text;
43
44     public AddTextCommand(TextEditor editor, string text)
45     {
46         _editor = editor;
47         _text = text;
48     }
49
50     public void Execute()
51     {
52         _editor.Append(_text);
53     }
54
55     public ICommand Clone()
56     {
57         // Cloning with same target and data
58         return new AddTextCommand(_editor, _text);
59     }
60
61     public string GetText() => _text;
62 }
63
64 // ----- Invoker (Command Manager) -----
65 public class CommandManager
66 {
67     private List<ICommand> _history = new List<ICommand>();
68
69     public void ExecuteCommand(ICommand command)
70     {
71         command.Execute();
72         _history.Add(command.Clone()); // Store clone for history
73     }
74
75     public void Replay()
76     {
77         Console.WriteLine("\n[Replaying Commands]");
78         foreach (var cmd in _history)
79         {
80             cmd.Execute();
81         }
82     }
83 }
84
85 // ----- Client Code -----
86 class Program
87 {
88     static void Main(string[] args)
89     {
90         TextEditor editor = new TextEditor();
91         CommandManager manager = new CommandManager();
92
93         manager.ExecuteCommand(new AddTextCommand(editor, "Hello "));
94         editor.Show();
95
96         manager.ExecuteCommand(new AddTextCommand(editor, "World"));
97         editor.Show();
98
99         manager.ExecuteCommand(new AddTextCommand(editor, "!"));
100        editor.Show();

```

```

101         // Let's replay the command history (like retyping everything)
102         editor = new TextEditor(); // reset editor
103         manager.ExecuteCommand(new AddTextCommand(editor, "\n--- Replay Start ---\n"));
104         manager.Replay();
105         editor.Show();
106     }
107 }
108
109 //output
110 [Editor] Current Text: "Hello "
111 [Editor] Current Text: "Hello World"
112 [Editor] Current Text: "Hello World!"
113
114 [Replaying Commands]
115 [Editor] Current Text: "
116 --- Replay Start ---
117 Hello World!"

```

2.16 Proxy + Observer

A document can be viewed or edited, but access is controlled via a Proxy.

Observers (like SecurityLogger, UI updates, etc.) are notified when someone accesses or changes the document.

```

1     using System;
2     using System.Collections.Generic;
3
4     // ----- Observer Pattern -----
5
6     public interface IDocumentObserver
7     {
8         void OnDocumentChanged(string content);
9     }
10
11    // ----- Subject -----
12    public interface IDocument
13    {
14        void Edit(string newContent);
15        string Read();
16        void AddObserver(IDocumentObserver observer);
17    }
18
19    // ----- Real Subject -----
20    public class RealDocument : IDocument
21    {
22        private string _content = "Initial content";
23        private List<IDocumentObserver> _observers = new List<IDocumentObserver>();
24
25        public void Edit(string newContent)
26        {
27            _content = newContent;
28            NotifyObservers();
29        }
30
31        public string Read()
32        {
33            return _content;

```

```

34     }
35
36     public void AddObserver(IDocumentObserver observer)
37     {
38         _observers.Add(observer);
39     }
40
41     private void NotifyObservers()
42     {
43         foreach (var observer in _observers)
44         {
45             observer.OnDocumentChanged(_content);
46         }
47     }
48 }
49
50 // ----- Proxy -----
51 public class DocumentProxy : IDocument
52 {
53     private RealDocument _realDocument;
54     private string _userRole; // e.g., "admin" or "viewer"
55
56     public DocumentProxy(string userRole)
57     {
58         _realDocument = new RealDocument();
59         _userRole = userRole;
60     }
61
62     public void Edit(string newContent)
63     {
64         if (_userRole == "admin")
65         {
66             Console.WriteLine($"[Proxy] Edit allowed for role: {_userRole}");
67             _realDocument.Edit(newContent);
68         }
69         else
70         {
71             Console.WriteLine($"[Proxy] Edit DENIED for role: {_userRole}");
72         }
73     }
74
75     public string Read()
76     {
77         Console.WriteLine($"[Proxy] Document read by role: {_userRole}");
78         return _realDocument.Read();
79     }
80
81     public void AddObserver(IDocumentObserver observer)
82     {
83         _realDocument.AddObserver(observer);
84     }
85 }
86
87 // ----- Observers -----
88 public class SecurityLogger : IDocumentObserver
89 {
90     public void OnDocumentChanged(string content)
91     {
92         Console.WriteLine($"[Logger] Document changed. New content: '{content}'");

```

```

93     }
94 }
95
96 public class UIUpdater : IDocumentObserver
97 {
98     public void OnDocumentChanged(string content)
99     {
100         Console.WriteLine($"[UI] Updating editor view with: '{content}'");
101     }
102 }
103
104 // ----- Client -----
105 class Program
106 {
107     static void Main(string[] args)
108     {
109         DocumentProxy adminProxy = new DocumentProxy("admin");
110         adminProxy.AddObserver(new SecurityLogger());
111         adminProxy.AddObserver(new UIUpdater());
112
113         adminProxy.Edit("Confidential Document - Version 1");
114         Console.WriteLine(adminProxy.Read());
115
116         Console.WriteLine();
117
118         DocumentProxy viewerProxy = new DocumentProxy("viewer");
119         viewerProxy.AddObserver(new SecurityLogger());
120
121         viewerProxy.Edit("Trying to hack the doc");
122         Console.WriteLine(viewerProxy.Read());
123     }
124 }
125 //output
126 [Proxy] Edit allowed for role: admin
127 [Logger] Document changed. New content: 'Confidential Document - Version 1'
128 [UI] Updating editor view with: 'Confidential Document - Version 1'
129 [Proxy] Document read by role: admin
130 Confidential Document - Version 1
131
132 [Proxy] Edit DENIED for role: viewer
133 [Proxy] Document read by role: viewer
134 Confidential Document - Version 1

```

2.17 Factory + Decorator

A Factory creates basic coffee types (Espresso, Latte, etc.).

A Decorator adds extra features dynamically (Milk, Sugar, WhippedCream, etc.).

Result: A flexible way to build customizable coffee drinks.

```

1     using System;
2
3     // ----- Component Interface -----
4     public interface ICoffee
5     {
6         string GetDescription();
7         double GetCost();
8     }
9

```

```

10 // ----- Concrete Components -----
11 public class Espresso : ICoffee
12 {
13     public string GetDescription() => "Espresso";
14     public double GetCost() => 2.0;
15 }
16
17 public class Latte : ICoffee
18 {
19     public string GetDescription() => "Latte";
20     public double GetCost() => 3.0;
21 }
22
23 // ----- Decorator Base Class -----
24 public abstract class CoffeeDecorator : ICoffee
25 {
26     protected ICoffee _coffee;
27
28     public CoffeeDecorator(ICoffee coffee)
29     {
30         _coffee = coffee;
31     }
32
33     public virtual string GetDescription() => _coffee.GetDescription();
34     public virtual double GetCost() => _coffee.GetCost();
35 }
36
37 // ----- Concrete Decorators -----
38 public class Milk : CoffeeDecorator
39 {
40     public Milk(ICoffee coffee) : base(coffee) { }
41
42     public override string GetDescription() => _coffee.GetDescription() + ", Milk"
43     ;
44     public override double GetCost() => _coffee.GetCost() + 0.5;
45 }
46
47 public class Sugar : CoffeeDecorator
48 {
49     public Sugar(ICoffee coffee) : base(coffee) { }
50
51     public override string GetDescription() => _coffee.GetDescription() + ", Sugar"
52     ;
53     public override double GetCost() => _coffee.GetCost() + 0.2;
54 }
55
56 public class WhippedCream : CoffeeDecorator
57 {
58     public WhippedCream(ICoffee coffee) : base(coffee) { }
59
60     public override string GetDescription() => _coffee.GetDescription() + ",
61     Whipped Cream";
62     public override double GetCost() => _coffee.GetCost() + 0.7;
63 }
64
65 // ----- Factory -----
66 public class CoffeeFactory
67 {
68     public ICoffee CreateCoffee(string type)

```

```

66     {
67         return type.ToLower() switch
68         {
69             "espresso" => new Espresso(),
70             "latte" => new Latte(),
71             _ => throw new ArgumentException("Unknown coffee type"),
72         };
73     }
74 }
75
76 // ----- Client -----
77 class Program
78 {
79     static void Main(string[] args)
80     {
81         CoffeeFactory factory = new CoffeeFactory();
82
83         // Create a base coffee using factory
84         ICoffee coffee = factory.CreateCoffee("espresso");
85
86         // Decorate it with milk and sugar
87         coffee = new Milk(coffee);
88         coffee = new Sugar(coffee);
89         coffee = new WhippedCream(coffee);
90
91         Console.WriteLine($"Order: {coffee.GetDescription()}");
92         Console.WriteLine($"Total Cost: ${coffee.GetCost():0.00}");
93     }
94 }
95 //output
96 Order: Espresso, Milk, Sugar, Whipped Cream
97 Total Cost: $3.40

```

2.18 Facade + Singleton

You have a complex logging system (file writer, console, cloud logger).

You create a Facade to simplify interaction.

You make the Facade a Singleton, so the entire app uses the same logger instance.

```

1     using System;
2
3     // ----- Subsystems -----
4     public class FileLogger
5     {
6         public void LogToFile(string message)
7         {
8             Console.WriteLine($"[FileLogger] {message}");
9         }
10    }
11
12    public class ConsoleLogger
13    {
14        public void LogToConsole(string message)
15        {
16            Console.WriteLine($"[ConsoleLogger] {message}");
17        }
18    }
19

```

```

20 public class CloudLogger
21 {
22     public void LogToCloud(string message)
23     {
24         Console.WriteLine($"[CloudLogger] {message}");
25     }
26 }
27
28 // ----- Facade + Singleton -----
29 public class LoggerFacade
30 {
31     private static LoggerFacade _instance;
32     private static readonly object _lock = new object();
33
34     private FileLogger _fileLogger;
35     private ConsoleLogger _consoleLogger;
36     private CloudLogger _cloudLogger;
37
38     private LoggerFacade() // private constructor for singleton
39     {
40         _fileLogger = new FileLogger();
41         _consoleLogger = new ConsoleLogger();
42         _cloudLogger = new CloudLogger();
43     }
44
45     public static LoggerFacade Instance
46     {
47         get
48         {
49             lock (_lock)
50             {
51                 return _instance ??= new LoggerFacade();
52             }
53         }
54     }
55
56     // Unified interface for logging
57     public void Log(string message)
58     {
59         _fileLogger.LogToFile(message);
60         _consoleLogger.LogToConsole(message);
61         _cloudLogger.LogToCloud(message);
62     }
63 }
64
65 // ----- Client Code -----
66 class Program
67 {
68     static void Main(string[] args)
69     {
70         LoggerFacade logger = LoggerFacade.Instance;
71
72         logger.Log("App started");
73         logger.Log("User clicked a button");
74
75         // Another part of the app
76         LoggerFacade.Instance.Log("Database connected");
77     }
78 }

```



```

79 //output
80 [FileLogger] App started
81 [ConsoleLogger] App started
82 [CloudLogger] App started
83 [FileLogger] User clicked a button
84 [ConsoleLogger] User clicked a button
85 [CloudLogger] User clicked a button
86 [FileLogger] Database connected
87 [ConsoleLogger] Database connected
88 [CloudLogger] Database connected

```

2.19 Command + Composite

We have a robot that can perform actions like Move, Turn, PickUp.

Each action is a Command.

A group of commands (like a mission or macro) can be composed as a single command using Composite.

```

1  using System;
2  using System.Collections.Generic;
3
4  // ----- Command Interface -----
5  public interface ICommand
6  {
7      void Execute();
8  }
9
10 // ----- Receiver -----
11 public class Robot
12 {
13     public void MoveForward() => Console.WriteLine("Robot moves forward.");
14     public void TurnLeft() => Console.WriteLine("Robot turns left.");
15     public void PickUpItem() => Console.WriteLine("Robot picks up the item.");
16 }
17
18 // ----- Concrete Commands -----
19 public class MoveCommand : ICommand
20 {
21     private Robot _robot;
22     public MoveCommand(Robot robot) => _robot = robot;
23     public void Execute() => _robot.MoveForward();
24 }
25
26 public class TurnLeftCommand : ICommand
27 {
28     private Robot _robot;
29     public TurnLeftCommand(Robot robot) => _robot = robot;
30     public void Execute() => _robot.TurnLeft();
31 }
32
33 public class PickUpCommand : ICommand
34 {
35     private Robot _robot;
36     public PickUpCommand(Robot robot) => _robot = robot;
37     public void Execute() => _robot.PickUpItem();
38 }
39
40 // ----- Composite Command -----
41 public class MacroCommand : ICommand

```

```

42 {
43     private List<ICommand> _commands = new List<ICommand>();
44
45     public void Add(ICommand command)
46     {
47         _commands.Add(command);
48     }
49
50     public void Execute()
51     {
52         Console.WriteLine("Executing macro command:");
53         foreach (var cmd in _commands)
54         {
55             cmd.Execute();
56         }
57     }
58 }
59
60 // ----- Client Code -----
61 class Program
62 {
63     static void Main(string[] args)
64     {
65         Robot robot = new Robot();
66
67         // Individual commands
68         ICommand move = new MoveCommand(robot);
69         ICommand turn = new TurnLeftCommand(robot);
70         ICommand pickUp = new PickUpCommand(robot);
71
72         // Composite command (macro)
73         MacroCommand mission = new MacroCommand();
74         mission.Add(move);
75         mission.Add(turn);
76         mission.Add(move);
77         mission.Add(pickUp);
78
79         // Execute single macro
80         mission.Execute();
81     }
82 }
83 //output
84 Executing macro command:
85 Robot moves forward.
86 Robot turns left.
87 Robot moves forward.
88 Robot picks up the item.

```

2.20 Composite + Decorator

You have UI components like Button, TextField, Panel (a Panel can hold multiple components — this is Composite).

You want to decorate individual UI elements with behaviors like scrollbars, borders, shadows, etc. (Decorator).

```

1     using System;
2     using System.Collections.Generic;
3

```

```

4  // ----- Component Interface -----
5  public interface IUIComponent
6  {
7      void Render();
8  }
9
10 // ----- Leaf Components -----
11 public class Button : IUIComponent
12 {
13     public void Render() => Console.WriteLine("Rendering a Button");
14 }
15
16 public class TextField : IUIComponent
17 {
18     public void Render() => Console.WriteLine("Rendering a TextField");
19 }
20
21 // ----- Composite -----
22 public class Panel : IUIComponent
23 {
24     private List<IUIComponent> _children = new List<IUIComponent>();
25
26     public void Add(IUIComponent component) => _children.Add(component);
27
28     public void Render()
29     {
30         Console.WriteLine("Rendering Panel:");
31         foreach (var child in _children)
32         {
33             child.Render();
34         }
35     }
36 }
37
38 // ----- Decorator Base -----
39 public abstract class UIComponentDecorator : IUIComponent
40 {
41     protected IUIComponent _component;
42
43     public UIComponentDecorator(IUIComponent component)
44     {
45         _component = component;
46     }
47
48     public virtual void Render() => _component.Render();
49 }
50
51 // ----- Concrete Decorators -----
52 public class BorderDecorator : UIComponentDecorator
53 {
54     public BorderDecorator(IUIComponent component) : base(component) { }
55
56     public override void Render()
57     {
58         _component.Render();
59         Console.WriteLine(" -> with Border");
60     }
61 }
62

```

```

63 public class ShadowDecorator : UIComponentDecorator
64 {
65     public ShadowDecorator(UIComponent component) : base(component) { }
66
67     public override void Render()
68     {
69         _component.Render();
70         Console.WriteLine(" -> with Shadow");
71     }
72 }
73
74 // ----- Client Code -----
75 class Program
76 {
77     static void Main(string[] args)
78     {
79         // Create individual components
80         UIComponent button = new Button();
81         UIComponent textField = new TextField();
82
83         // Decorate button with border and shadow
84         UIComponent decoratedButton = new ShadowDecorator(new BorderDecorator(
            button));
85
86         // Create a panel and add components
87         Panel panel = new Panel();
88         panel.Add(decoratedButton);
89         panel.Add(textField);
90
91         // Decorate the whole panel with a border
92         UIComponent decoratedPanel = new BorderDecorator(panel);
93
94         // Render everything
95         decoratedPanel.Render();
96     }
97 }
98 //output
99 Rendering Panel:
100 Rendering a Button
101     -> with Border
102     -> with Shadow
103 Rendering a TextField
104     -> with Border

```

3 Differences

Aspect	Factory	Singleton
Purpose	Creates objects without specifying the exact class to be created.	Ensures only one instance of a class exists globally.
Object Creation	Provides a method to instantiate different objects based on inputs.	Controls the instantiation of a single object.
Focus	Encapsulates the creation logic.	Controls the number of instances to be one.
When to Use	When you need to create objects that may vary based on input or configuration.	When you need a single, shared resource across the application.
Use Case	Creating different types of documents (e.g., PDFs, Word files).	Managing a global configuration object.

Table 1: Factory vs. Singleton

Aspect	Decorator	Composite
Purpose	Adds additional responsibilities to an object dynamically.	Allows treating individual objects and compositions uniformly.
Structure	Wraps an object to extend its functionality.	Represents a part-whole hierarchy (tree structure).
Flexibility	High flexibility; can add multiple decorators dynamically.	Medium flexibility; adds uniformity in handling objects and groups.
Use Case	When you need to add dynamic features without altering the core class.	When you need to treat a group of objects as a single entity.
Examples	Adding scrollbars to windows, adding toppings to coffee.	GUI components like windows, panels, buttons.

Table 2: Decorator vs. Composite

Aspect	Adapter	Decorator
Purpose	Converts one interface to another, making incompatible interfaces compatible.	Adds new responsibilities or behaviors to an object dynamically.
Focus	Focuses on compatibility and interface adaptation.	Focuses on extending functionality without altering the original class.
When to Use	When you need to integrate or reuse existing classes with incompatible interfaces.	When you need to add features or behavior to individual objects dynamically.
Use Case	Integrating a legacy system or third-party library.	Adding features like borders or colors to graphical objects.

Table 3: Adapter vs. Decorator

Aspect	Factory	Singleton
Purpose	Creates objects without specifying the exact class to be created.	Ensures only one instance of a class exists globally.
Object Creation	Provides a method to instantiate different objects based on inputs.	Controls the instantiation of a single object.
Focus	Encapsulates the creation logic.	Controls the number of instances to be one.
When to Use	When you need to create objects that may vary based on input or configuration.	When you need a single, shared resource across the application.
Use Case	Creating different types of documents (e.g., PDFs, Word files).	Managing a global configuration object.

Table 4: Factory vs. Singleton

Aspect	Strategy	Adapter
Purpose	Encapsulates interchangeable behaviors.	Converts one interface to another to make classes compatible.
Flexibility	Allows swapping out algorithms at runtime.	Focuses on making two incompatible interfaces work together.
When to Use	When you want to replace one behavior with another dynamically.	When you need to integrate with existing systems that have different interfaces.
Use Case	Payment processing with different payment methods.	Integrating legacy code with a modern system.

Table 5: Strategy vs. Adapter

Aspect	Composite	Template
Purpose	Allows uniform treatment of individual objects and collections.	Defines a fixed algorithm structure, allowing customization of certain steps.
Hierarchy	Part-whole hierarchy; tree structure of objects.	No hierarchy; uses inheritance to customize behavior.
When to Use	When you want to treat individual objects and groups uniformly.	When you have a general algorithm but need flexibility in specific parts.
Use Case	GUI elements like windows, panels, and controls.	Document generation with customizable steps for formatting.

Table 6: Composite vs. Template

Aspect	Mediator	Observer
Purpose	Centralizes complex communication between multiple objects to reduce direct dependencies.	Defines a one-to-many dependency between objects, where changes in one object notify dependents automatically.
Structure	Introduces a mediator object that coordinates interactions between components.	Involves a subject (observable) and multiple observers (subscribers) that react to changes.
Flexibility	Reduces coupling between interacting objects but may centralize too much logic in the mediator.	Decouples subjects and observers, allowing dynamic subscription/unsubscription.
Use Case	When many objects communicate in complex ways, leading to tangled dependencies.	When an object's state change requires automatic updates to other dependent objects.
Examples	Air traffic control system, chatroom server coordinating messages.	Event handling systems, UI updates on data changes.

Table 7: Mediator vs. Observer

Aspect	Strategy	Template Method
Purpose	Enables selecting algorithms at runtime by encapsulating them as interchangeable objects.	Defines the skeleton of an algorithm in a base class, allowing subclasses to override specific steps.
Structure	Uses composition (strategy objects are injected).	Uses inheritance (algorithm structure is inherited).
Flexibility	High runtime flexibility - strategies can be swapped dynamically.	Compile-time flexibility - only specific steps can be modified.
Use Case	When multiple variants of an algorithm are needed and must be selectable during execution.	When the algorithm's structure is fixed but certain steps need customization.
Examples	Sorting algorithms (QuickSort, MergeSort), payment methods.	Framework hooks (init(), render()), document parsing pipelines.

Table 8: Strategy vs. Template Method

Aspect	Decorator	Proxy
Purpose	Dynamically adds responsibilities to objects without subclassing.	Controls access to an object, acting as its surrogate or placeholder.
Structure	Can stack multiple decorators recursively.	Typically has a one-to-one relationship with the target.
Flexibility	Highly flexible for adding combinations of behaviors.	More rigid, focused on access control.
Use Case	When responsibilities should be added/removed at runtime.	When object access needs management (lazy loading, protection).
Examples	I/O streams (BufferedInputStream, GZIP-InputStream).	Virtual proxies (lazy-loaded images), protection proxies.

Table 9: Decorator vs. Proxy

Aspect	Adapter	Facade
Purpose	Bridges incompatible interfaces to make them work together.	Provides a simplified interface to a complex subsystem.
Structure	Wraps a single class/interface.	Wraps multiple components.
Flexibility	Focused on interface translation.	Focused on abstraction and simplification.
Use Case	Integrating legacy code or third-party libraries.	Reducing subsystem complexity for clients.
Examples	Database connectors, device drivers.	Home theater systems, API gateways.

Table 10: Adapter vs. Facade

Aspect	Composite	Flyweight
Purpose	Treats individual objects and compositions uniformly in tree structures.	Shares state among objects to minimize memory usage.
Structure	Recursive tree structure (parent/child relationships).	Shared object pool with intrinsic/extrinsic state.
Flexibility	Flexible hierarchy management.	Optimized for memory efficiency.
Use Case	Representing part-whole hierarchies (e.g., GUI components).	When many similar objects must be managed efficiently.
Examples	File systems, organization charts.	Game engines (particle systems), text editors (character formatting).

Table 11: Composite vs. Flyweight

Aspect	Command	Observer
Purpose	Encapsulates requests as objects, enabling parameterization and queuing of operations.	Defines a one-to-many dependency between objects for automatic notifications.
Structure	Involves Invoker, Command, and Receiver components.	Comprises Subject (Observable) and Observer interfaces.
Flexibility	Supports undo/redo operations and command composition.	Allows dynamic subscription management at runtime.
Use Case	When operations need to be queued, logged, or undone.	When state changes require automatic propagation to dependents.
Examples	GUI button actions, macro recording.	Event handling, stock market updates.

Table 12: Command vs. Observer

Aspect	Prototype	Factory Method
Purpose	Creates new objects by cloning existing prototypes.	Defers instantiation to subclasses through virtual construction.
Structure	Uses clone() method on prototype instances.	Uses abstract creator class with factory methods.
Flexibility	Dynamic object configuration via cloning.	Static type variation through inheritance.
Use Case	When object creation is expensive or requires complex initialization.	When a class can't anticipate the objects it needs to create.
Examples	Game entity spawning, CAD object duplication.	Framework class libraries, cross-platform UI components.

Table 13: Prototype vs. Factory Method

Aspect	Adapter	Proxy
Purpose	Bridges interface incompatibilities between objects.	Controls access to an object without changing its interface.
Structure	Wraps an adaptee with different interface.	Maintains same interface as subject.
Flexibility	Interface translation focused.	Access management focused.
Use Case	Integrating incompatible components.	Managing expensive object access.
Examples	Legacy system wrappers, device drivers.	Remote proxies, protection proxies.

Table 14: Adapter vs. Proxy